

# nlp-final-assignment2

February 17, 2025

```
[2]: import nltk
import string
import gensim
import numpy as np
import time
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc
from sklearn.preprocessing import label_binarize
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import seaborn as sns
import pandas as pd
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from nltk.stem import PorterStemmer
import lime
from lime.lime_tabular import LimeTabularExplainer
from sklearn.inspection import permutation_importance
```

chatgpt prompt :- access file for tweet data

```
[5]: df = pd.read_csv(r"D:\Dataset\Sentiment_Data.csv", encoding="ISO-8859-1")
```

```
[173]: df.shape
```

```
[173]: (451332, 2)
```

```
[175]: df.describe()
```

```
[175]:
```

|        | Tweet  | Sentiment  |
|--------|--------|------------|
| count  | 451331 | 451332     |
| unique | 451279 | 5          |
| top    | #NAME? | Strong_Pos |
| freq   | 5      | 233700     |

```
[11]: df
```

```
[11]:
```

|        |                                                   | Tweet                                | Sentiment  |
|--------|---------------------------------------------------|--------------------------------------|------------|
| 0      | @angelica_toy                                     | Happy Anniversary!!!!...The Day...   | Mild_Pos   |
| 1      | @McfarlaneGlenda                                  | Happy Anniversary!!!!...The D...     | Mild_Pos   |
| 2      | @thevivafrei                                      | @JustinTrudeau Happy Anniversary!... | Mild_Pos   |
| 3      | @NChartierET                                      | Happy Anniversary!!!!...The Day t... | Mild_Pos   |
| 4      | @tabithapeters05                                  | Happy Anniversary!!!!...The D...     | Mild_Pos   |
| ...    |                                                   | ...                                  | ...        |
| 451327 | Gaza; Peace n' Freedom - Viva Palestina convoy... |                                      | Strong_Pos |
| 451328 | Face of Defense: Soldier Finds Freedom in U.S...  |                                      | Strong_Pos |
| 451329 | Face of Defense: Soldier Finds Freedom in U.S...  |                                      | Strong_Pos |
| 451330 | Gaza; Peace n' Freedom - "Israel stops aid con... |                                      | Strong_Pos |
| 451331 | @convoy 83 yes! get on freedom server!            |                                      | Strong_Pos |

[451332 rows x 2 columns]

```
[13]: df['Sentiment'].unique()
```

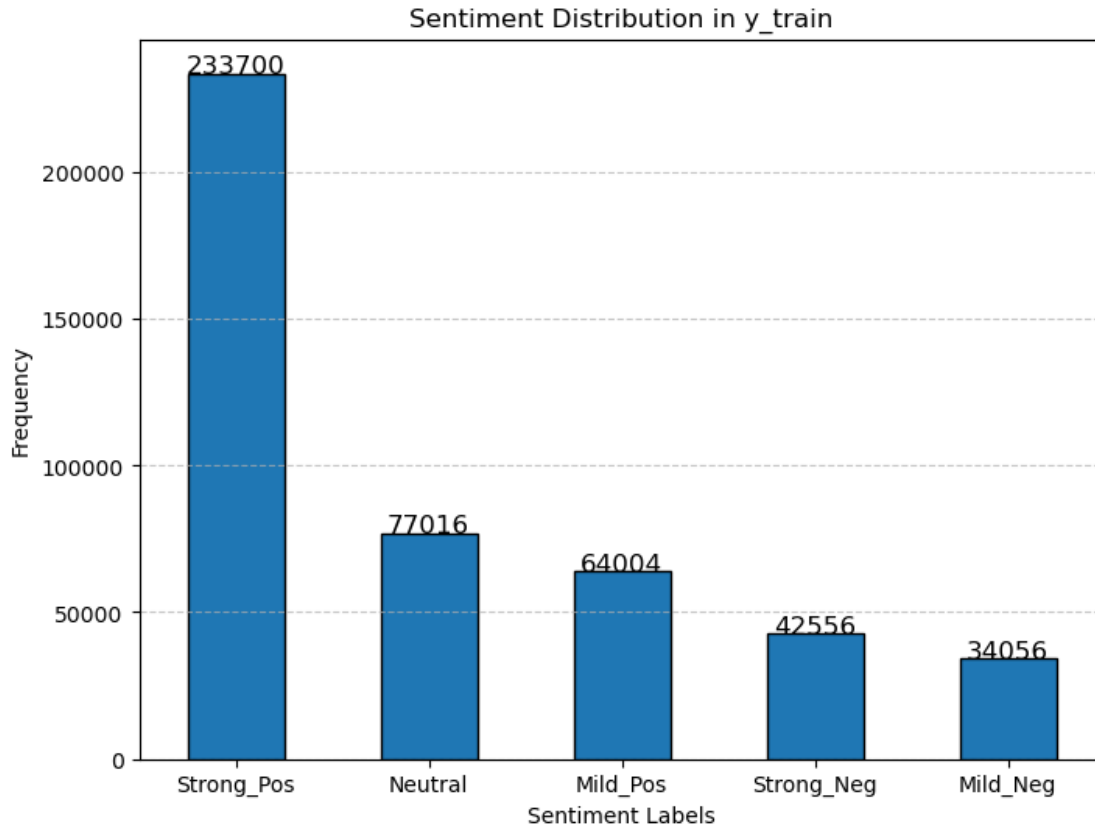
```
[13]: array(['Mild_Pos', 'Strong_Pos', 'Neutral', 'Strong_Neg', 'Mild_Neg'],  
      dtype=object)
```

```
[15]: def bar_plot(data, label_map=None):  
    # Count occurrences of each label  
    class_counts = pd.Series(data).value_counts()  
  
    # Plot bar chart  
    class_counts.plot(kind="bar", edgecolor="black", figsize=(8, 6))  
  
    # Add values on top of each bar  
    for i, v in enumerate(class_counts):  
        plt.text(i, v + 0.2, str(v), ha="center", fontsize=12)  
  
    # Customize plot  
    plt.xlabel("Sentiment Labels")  
    plt.ylabel("Frequency")  
    plt.title("Sentiment Distribution in y_train")  
  
    # If label_map is provided, replace numeric labels with customer labels  
    if label_map is not None:  
        # Use the label_map to replace numeric labels  
        plt.xticks(range(len(class_counts)), [label_map.get(i) for i in_  
↪ class_counts.index], rotation=0)  
    else:  
        # If no label_map, show numeric labels  
        plt.xticks(range(len(class_counts)), class_counts.index, rotation=0)
```

```
plt.grid(axis="y", linestyle="--", alpha=0.7)

# Show the plot
plt.show()
```

```
[17]: bar_plot(df['Sentiment'])
```



```
[19]: # 'Negative' -> 0, 'Neutral' -> 1, 'Positive' -> 2
label_map = {'Strong_Pos': 0, 'Mild_Pos': 1, 'Neutral': 2, 'Strong_Neg': 3,
            ↪ 'Mild_Neg': 4}
reverse_label_map = {0: 'Strong_Pos', 1: 'Mild_Pos', 2: 'Neutral', 3:
            ↪ 'Strong_Neg', 4: 'Mild_Neg'}
```

SVM Model using word2vec (CBOW or Skipgram and any vector size from 64 to 300)

chatgpt prompt :- how to implement random undersampling for each label in the target class

Under Sampling

```
[24]: # Count the number of samples per class
class_counts = df['Sentiment'].value_counts()
print("\nBefore undersampling:\n", class_counts)
```

```

# Define the number of samples to take for each class
sample_sizes = {
    'Strong_Pos': 6000,
    'Mild_Pos': 6000,
    'Neutral': 6000, # Adjust the sample size for Negative
    'Strong_Neg': 6000, # Adjust the sample size for Neutral
    'Mild_Neg': 6000 # Adjust the sample size for Positive
}

# Perform undersampling without replacement
df_under = pd.concat([
    df[df['Sentiment'] == label].sample(n=min(sample_sizes[label],
    ↪len(df[df['Sentiment'] == label])),
                                random_state=42,
                                replace=False)
    for label in sample_sizes.keys()
])

# Print class distribution after undersampling
print("\nAfter undersampling:\n", df_under['Sentiment'].value_counts())

```

Before undersampling:

| Sentiment  |        |
|------------|--------|
| Strong_Pos | 233700 |
| Neutral    | 77016  |
| Mild_Pos   | 64004  |
| Strong_Neg | 42556  |
| Mild_Neg   | 34056  |

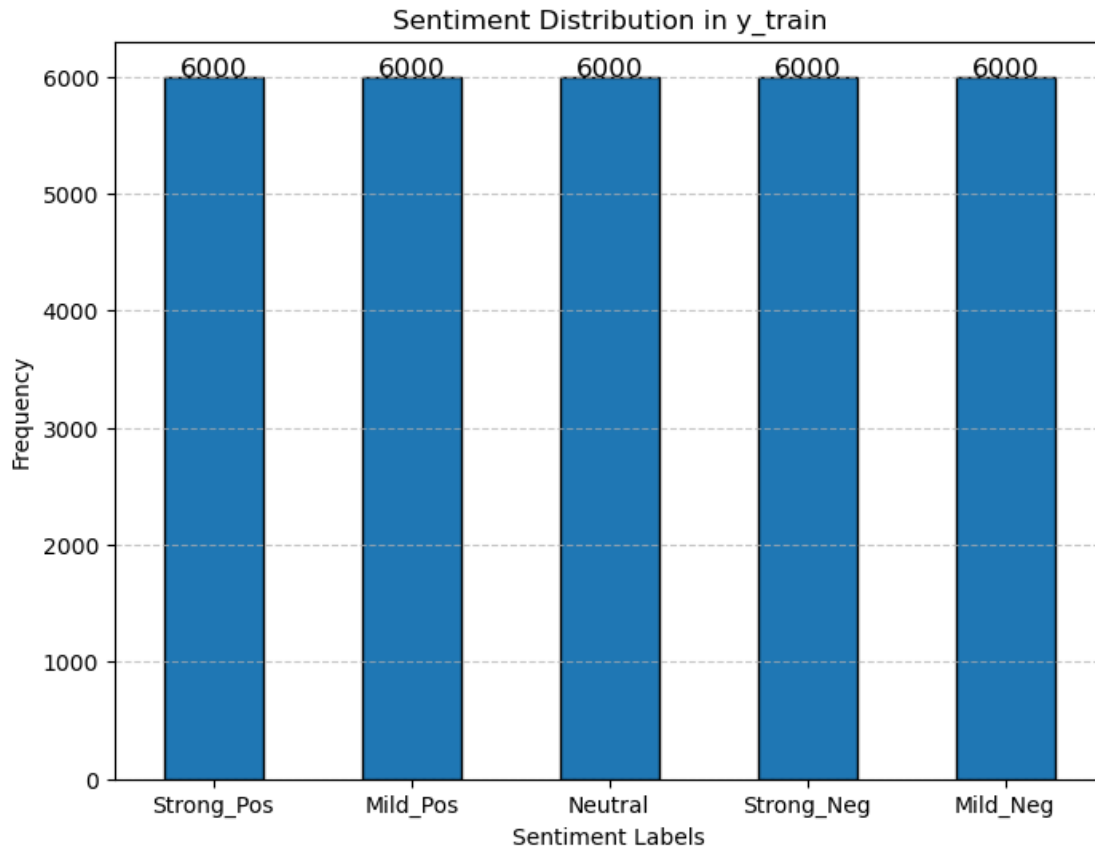
Name: count, dtype: int64

After undersampling:

| Sentiment  |      |
|------------|------|
| Strong_Pos | 6000 |
| Mild_Pos   | 6000 |
| Neutral    | 6000 |
| Strong_Neg | 6000 |
| Mild_Neg   | 6000 |

Name: count, dtype: int64

```
[25]: bar_plot(df_under['Sentiment'])
```



tokenization

```
[29]: df_under['Sentiment'] = df_under['Sentiment'].map(label_map)
```

```
[31]: df_under['Sentiment']
```

```
[31]: 406658    0
      90803    0
      75490    0
      431292  0
      150871  0
      ..
      285982  4
      409454  4
      397305  4
      195898  4
      101315  4
      Name: Sentiment, Length: 30000, dtype: int64
```

chatgpt prompt :- how to implement word2vector with svm

```
[34]: # Function to get the tweet vector by averaging word vectors
def get_tweet_vector(tweet_tokens, model):
    vector = np.zeros(model.vector_size) # Initialize a zero vector
    valid_words = 0
    for word in tweet_tokens:
        if word in model.wv:
            vector += model.wv[word] # Add the word vector to the total sum
            valid_words += 1
    if valid_words > 0:
        vector /= valid_words # Return the average word vector for the tweet
    return vector
```

chatgpt prompt :- lime for svm using word2vector

Lime

```
[38]: def lime_explanation_single_record(model, X_train_vectors, X_test_vector,
    ↪reverse_label_map, word2vec_model):
    """
    Generates a LIME explanation for a single instance using an SVM model and
    ↪precomputed Word2Vec vectors.

    Parameters:
    - model: Trained SVM classifier with probability=True
    - X_train_vectors: Numpy array of precomputed Word2Vec vectors (training
    ↪set)
    - X_test_vector: A single test instance (Word2Vec vector)
    - reverse_label_map: Dictionary mapping class indices to labels
    - word2vec_model: Trained Word2Vec model to get feature (word) names
    """

    # Extract class names from reverse_label_map
    class_names = [reverse_label_map[i] for i in range(len(reverse_label_map))]

    # Get feature (word) names from the Word2Vec vocabulary
    feature_names = list(word2vec_model.wv.index_to_key)[:X_train_vectors.
    ↪shape[1]] # Use only available features

    # Initialize LIME Tabular Explainer
    explainer = LimeTabularExplainer(
        training_data=X_train_vectors,
        feature_names=feature_names, # Use actual words as feature names
        class_names=class_names,
        mode="classification"
    )

    # Reshape the test vector to match LIME input format
    instance = X_test_vector.reshape(1, -1)
```

```

# Define prediction function for LIME
predict_fn = lambda x: model.predict_proba(x)

# Generate LIME explanation
exp = explainer.explain_instance(instance[0], predict_fn, num_features=10,
↳top_labels=5)

# Show explanation
exp.show_in_notebook()

return exp

```

chatgpt prompt :- modife the give code to include slang handling, abbreviations handling & emoji handling

stemming

```

[42]: import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import emoji

# Download necessary NLTK data
nltk.download('stopwords')
nltk.download('punkt')

# Slang and abbreviation dictionaries
slang_dict = {
    "brb": "be right back",
    "lol": "laugh out loud",
    "idk": "I don't know",
    "smh": "shaking my head",
    "pls": "please",
    "btw": "by the way",
    "u": "you",
    "r": "are"
}

abbreviations_dict = {
    "u": "you",
    "r": "are",
    "pls": "please",
    "btw": "by the way"
}

# Function to handle emojis

```

```

def handle_emojis(text):
    return emoji.demojize(text) # Converts emojis to text descriptions like ":
    ↪️smiling_face_with_sunglasses:"

# Function to handle slang words
def handle_slang(text):
    words = text.split()
    return " ".join([slang_dict.get(word, word) for word in words])

# Function to handle abbreviations
def handle_abbreviations(text):
    words = text.split()
    return " ".join([abbreviations_dict.get(word, word) for word in words])

# Function to preprocess the text
def preprocess_stopwords_stemming(text):
    # Step 1: Handle emojis
    text = handle_emojis(str(text))

    # Step 2: Handle slang and abbreviations
    text = handle_slang(text)
    text = handle_abbreviations(str(text))

    # Step 3: Tokenize the text and convert it to lowercase
    tokens = word_tokenize(str(text).lower()) # Convert to lowercase and
    ↪️tokenize

    # Step 4: Get the set of stopwords
    stop_words = set(stopwords.words('english'))

    # Step 5: Initialize the stemmer
    stemmer = PorterStemmer()

    # Step 6: Remove stopwords, non-alphabetic and non-numeric tokens, and
    ↪️apply stemming
    return [stemmer.stem(word) for word in tokens if word.isalpha() or word.
    ↪️isdigit() and word not in stop_words]

```

```

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!

```



# 1 SVM

chatgpt prompt :- how to implement svm using word2vecotr with skipgram

```
[46]: # Apply preprocessing to the text data
df_under['stem_tokens'] = df_under['Tweet'].apply(preprocess_stopwords_stemming)
```

```
[47]: df_under['stem_tokens']
```

```
[47]: 406658      [viva, frei, live, now, live, in, ottawa, free...
90803      [laurajede, thi, is, exactli, the, convoy, typ...
75490      [hollyanndoan, q, to, freeland, whi, did, you,...
431292      [freedom, convoy, rais, on, gofundm]
150871      [coupl, goe, to, freedom, convoy, to, see, for...
...
285982      [bullshit, i, am, a, free, canadian, bcz, of, ...
409454      [lvrrwatt, so, what, your, say, is, that, these...
397305      [paul, alexand, amp, apo, s, speech, at, freed...
195898      [the, person, email, of, a, hamont, infecti, d...
101315      [realandyleeshow, fascism, and, the, democrat,...
Name: stem_tokens, Length: 30000, dtype: object
```

chatgpt prompt:- Class Separability for the give tweet data

```
[51]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.manifold import TSNE
from sklearn.feature_selection import mutual_info_classif

# Assuming your DataFrame `df` is loaded and `stem_tokens` contains the
↳ tokenized words

# Step 1: Join the tokens into a single string for each row
df_under['cleaned_tweets'] = df_under['stem_tokens'].apply(lambda x: ' '.
↳ join(x))

# Step 2: Vectorize the cleaned tweets using TfidfVectorizer
vectorizer = TfidfVectorizer(max_features=5000) # Adjust max_features to your
↳ preference
X = vectorizer.fit_transform(df_under['cleaned_tweets'])

# Step 3: Define the target variable (sentiment)
y = df_under['Sentiment'] # Assuming you have a sentiment column

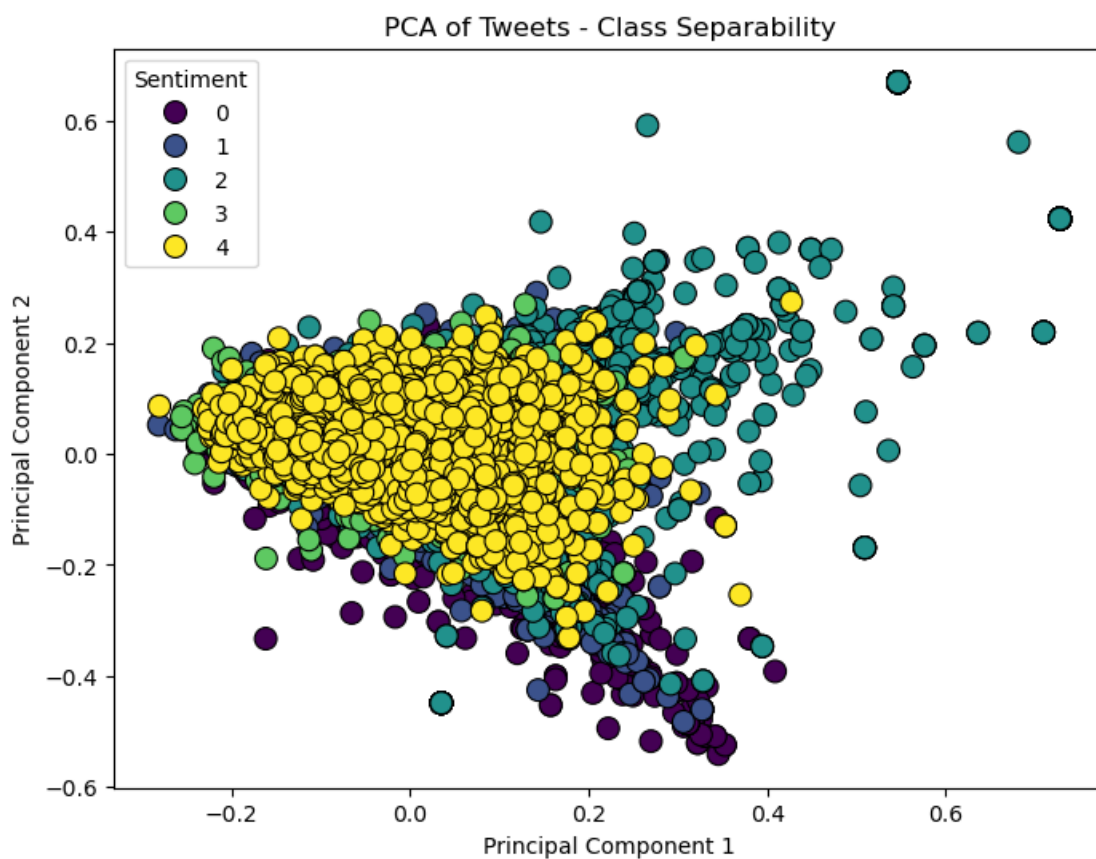
# Step 4: Dimensionality Reduction using PCA for visualization
```

```

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X.toarray()) # Convert sparse matrix to dense array,
    ↪ for PCA

# Visualizing PCA results
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y, palette='viridis', s=100,
    ↪ edgecolor='k')
plt.title("PCA of Tweets - Class Separability")
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

```



```

[53]: x_train, x_test, y_train, y_test = train_test_split(df_under['stem_tokens'],
    ↪ df_under['Sentiment'], test_size=0.2, random_state=42)
print("\nTraining and Testing Split:")
print(f"\nTraining Data:\n shape: {x_train.shape} \n {x_train},\n\n Test Data:
    ↪ \n shape: {x_test.shape} \n {x_test}")

```

```
print(f"\nTraining Target Data:\n shape: {y_train.shape} \n {y_train},\n\n Test_
↳Target Data:\n shape: {y_test.shape} \n {y_test}")
```

Training and Testing Split:

Training Data:

```
shape: (24000,)
114009      [it, no, consol, but, it, the, same, up, here,...
73798      [tmcgaghran, and, thi, about, conserv, mp, inc...
407200      [angri, that, so, mani, turn, a, blind, eye, t...
308627      [ottawa, freedom, convoy, live, 2022, raw, foo...
114305      [sorri, freedom, convoy, you, steal, our, flag...

...
279334      [justintrudeau, jimwatsonottawa, still, battl,...
219575      [freedom, convoy, freedomweek, start, 21, do, ...
94202      [7, liber, claim, about, the, freedom, convoy,...
296535      [ottawa, protest, continu, despit, new, emerg,...
418321      [mizzchanta, peopl, in, thi, convoy, are, figh...
Name: stem_tokens, Length: 24000, dtype: object,
```

Test Data:

```
shape: (6000,)
120426      [newfiescumbag, one, of, the, he, use, wa, tho...
237177      [evanlsolomon, ha, been, do, face, to, face, i...
368533      [i, spent, last, night, have, talk, with, my, ...
320595      [whi, are, trucker, protest, in, ottawa, our, ...
276222      [opp, hous, call, we, been, watch, the, freedo...

...
202501      [gggreenwald, where, is, thi, document, i, thin...
332923      [winter, of, discount, freedom, convoy, support...
299708      [goplead, laurenboebert, hawleymo, mattgaetz, ...
71179      [read, my, mind, trudeau, is, guilti, trudeaun...
378918      [i, just, got, restrict, becaus, i, told, seve...
Name: stem_tokens, Length: 6000, dtype: object
```

Training Target Data:

```
shape: (24000,)
114009      3
73798      0
407200      3
308627      0
114305      2
..
279334      4
219575      0
94202      0
```

```
296535    2
418321    3
Name: Sentiment, Length: 24000, dtype: int64,
```

Test Target Data:

```
shape: (6000,)
```

```
120426    0
```

```
237177    3
```

```
368533    3
```

```
320595    4
```

```
276222    0
```

```
..
```

```
202501    0
```

```
332923    2
```

```
299708    3
```

```
71179     4
```

```
378918    4
```

```
Name: Sentiment, Length: 6000, dtype: int64
```

```
[55]: # Train the Word2Vec model (Skipgram model with vector size 100) on the whole
      ↪ df_under_stem_tokens
word2vec_model = gensim.models.Word2Vec(sentences=df_under['stem_tokens'],
      ↪ vector_size=100, window=5, min_count=1, sg=1)
```

```
[56]: # Generate vectors for the training set using the trained Word2Vec model
x_train_vectors = [get_tweet_vector(tweet, word2vec_model) for tweet in x_train]

# Generate vectors for the test set using the already trained Word2Vec model
x_test_vectors = [get_tweet_vector(tweet, word2vec_model) for tweet in x_test]
```

```
[57]: # Prepare data for SVM model
x_train_array = np.array(x_train_vectors) # Features: Tweet vectors (Training
      ↪ data)
x_test_array = np.array(x_test_vectors) # Features: Tweet vectors (Test data)
y_train_array = np.array(y_train) # Labels: Sentiments (Training data)
y_test_array = np.array(y_test) # Labels: Sentiments (Test data)

print('x_train_array shape: ', x_train_array.shape)
print('x_test_array shape: ', x_test_array.shape)
print('y_train_array shape: ', y_train_array.shape)
print('y_test_array shape: ', y_test_array.shape)
```

```
x_train_array shape: (24000, 100)
```

```
x_test_array shape: (6000, 100)
```

```
y_train_array shape: (24000,)
```

```
y_test_array shape: (6000,)
```

```

[61]: # Initialize the SVM model with a linear kernel
svm_classifier = SVC(kernel='linear', probability=True)

# Measure training time
start_time = time.time()

# Train the SVM model
svm_classifier.fit(x_train_array, y_train_array)

# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")

# Predictions
y_train_pred = svm_classifier.predict(x_train_array)
y_test_pred = svm_classifier.predict(x_test_array)

# Print Classification Report
print("\nClassification Report (Train Set):")
print(classification_report(y_train_array, y_train_pred))

print("\nClassification Report (Test Set):")
print(classification_report(y_test_array, y_test_pred))

# Compute Confusion Matrices
conf_matrix_train = confusion_matrix(y_train_array, y_train_pred)
conf_matrix_test = confusion_matrix(y_test_array, y_test_pred)

# Class labels mapping
classes = [reverse_label_map[i] for i in range(len(label_map))]

# Plot Confusion Matrix for Train & Test
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Train Set Confusion Matrix
sns.heatmap(conf_matrix_train, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes, ax=axes[0])
axes[0].set_title("Confusion Matrix - Train Set")
axes[0].set_xlabel("Predicted Label")
axes[0].set_ylabel("True Label")

# Test Set Confusion Matrix
sns.heatmap(conf_matrix_test, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes, ax=axes[1])
axes[1].set_title("Confusion Matrix - Test Set")
axes[1].set_xlabel("Predicted Label")
axes[1].set_ylabel("True Label")

```

```

plt.show()

# Binarize labels for ROC curve
y_train_bin = label_binarize(y_train_array, classes=[0, 1, 2, 3, 4])
y_test_bin = label_binarize(y_test_array, classes=[0, 1, 2, 3, 4])

# Compute ROC Curve and AUC for Train and Test sets separately
plt.figure(figsize=(12, 6))

# Plot ROC for Train Set
plt.subplot(1, 2, 1)
plt.title('ROC Curve - Train Set')
for i in range(len(classes)):
    fpr_train, tpr_train, _ = roc_curve(y_train_bin[:, i], svm_classifier.
    ↪predict_proba(x_train_array)[:, i])
    roc_auc_train = auc(fpr_train, tpr_train)

    plt.plot(fpr_train, tpr_train, label=f'{classes[i]} (AUC = {roc_auc_train:.
    ↪2f})')

plt.plot([0, 1], [0, 1], color='gray', linestyle='--') # Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")

# Plot ROC for Test Set
plt.subplot(1, 2, 2)
plt.title('ROC Curve - Test Set')
for i in range(len(classes)):
    fpr_test, tpr_test, _ = roc_curve(y_test_bin[:, i], svm_classifier.
    ↪predict_proba(x_test_array)[:, i])
    roc_auc_test = auc(fpr_test, tpr_test)

    plt.plot(fpr_test, tpr_test, label=f'{classes[i]} (AUC = {roc_auc_test:.
    ↪2f})')

plt.plot([0, 1], [0, 1], color='gray', linestyle='--') # Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")

plt.tight_layout()

```

```

plt.show()

# Get the list of words in your model (for example, the vocabulary)
words = list(word2vec_model.wv.index_to_key)

# Now we look at the coefficients from the SVM model
coef = svm_classifier.coef_.flatten() # Coefficients from the SVM model

# Get top N features (words) that have the highest coefficients (importance)
top_indices = np.argsort(coef)[-10:][::-1] # Indices of the highest
↳ coefficients

# Map these indices back to the words in your word2vec model
top_words = [words[i] for i in top_indices]

# Print top 10 words with the highest coefficients
print("Top 10 Words with the Highest Coefficients:")
for word in top_words:
    print(f"{word}: {coef[words.index(word)]:.4f}")

```

Training time: 306.62 seconds

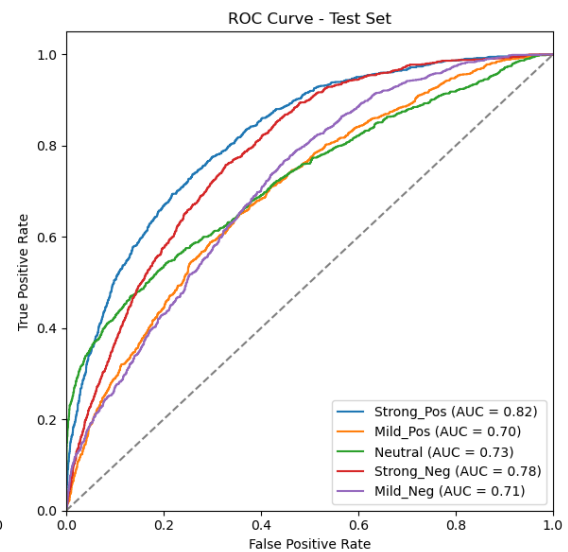
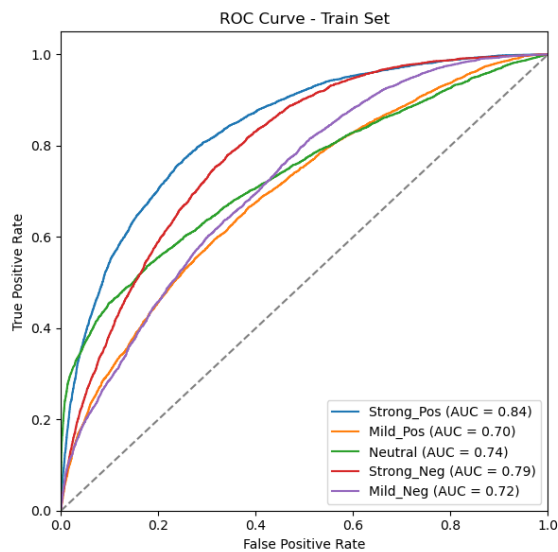
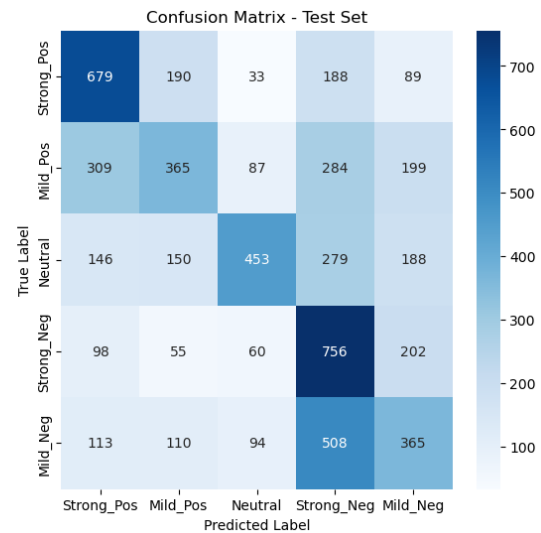
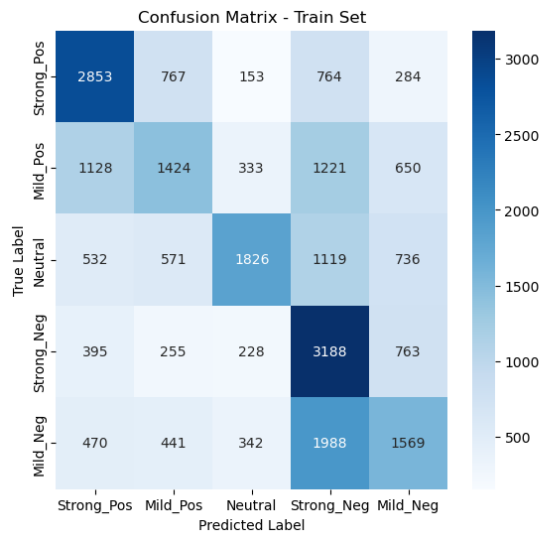
Classification Report (Train Set):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.53      | 0.59   | 0.56     | 4821    |
| 1            | 0.41      | 0.30   | 0.35     | 4756    |
| 2            | 0.63      | 0.38   | 0.48     | 4784    |
| 3            | 0.39      | 0.66   | 0.49     | 4829    |
| 4            | 0.39      | 0.33   | 0.36     | 4810    |
| accuracy     |           |        | 0.45     | 24000   |
| macro avg    | 0.47      | 0.45   | 0.45     | 24000   |
| weighted avg | 0.47      | 0.45   | 0.45     | 24000   |

Classification Report (Test Set):

|          | precision | recall | f1-score | support |
|----------|-----------|--------|----------|---------|
| 0        | 0.50      | 0.58   | 0.54     | 1179    |
| 1        | 0.42      | 0.29   | 0.35     | 1244    |
| 2        | 0.62      | 0.37   | 0.47     | 1216    |
| 3        | 0.38      | 0.65   | 0.47     | 1171    |
| 4        | 0.35      | 0.31   | 0.33     | 1190    |
| accuracy |           |        | 0.44     | 6000    |

|              |      |      |      |      |
|--------------|------|------|------|------|
| macro avg    | 0.45 | 0.44 | 0.43 | 6000 |
| weighted avg | 0.46 | 0.44 | 0.43 | 6000 |



Top 10 Words with the Highest Coefficients:

trudeauumstgo: 5.7603

canadatruck: 4.8610

king: 4.7669

doe: 4.5465

idiot: 4.5317

away: 4.4518



donor: 4.4292  
told: 4.4269  
invok: 4.3874  
demand: 4.3400

```
[62]: lime_explanation_single_record(svm_classifier, x_train_array, x_test_array[0],  
    ↪reverse_label_map, word2vec_model)
```

<IPython.core.display.HTML object>

```
[62]: <lime.explanation.Explanation at 0x1ff52b22a20>
```

grid search cv

model was unable to learn the train data completely. will be doing grid search with hyper parameter to find the best model

```
[65]: import time  
import numpy as np  
import pandas as pd  
from sklearn.svm import SVC  
from sklearn.model_selection import GridSearchCV  
from sklearn.metrics import classification_report, confusion_matrix, roc_curve,  
    ↪auc  
from sklearn.preprocessing import LabelBinarizer  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Assuming x_train_array, y_train_array, x_test_array, y_test_array are already  
    ↪prepared  
  
# Define the parameter grid for GridSearchCV  
param_grid = {  
    'C': [0.1, 1, 10], # Regularization parameter  
    'kernel': ['linear', 'rbf'], # Kernel types  
    'probability': [True], # Enable probability estimates for ROC  
}  
  
# Initialize the SVM classifier  
svm_classifier = SVC()  
  
# Initialize GridSearchCV  
grid_search = GridSearchCV(estimator=svm_classifier, param_grid=param_grid,  
    ↪cv=3, scoring='accuracy', verbose=2, n_jobs=-1)  
  
# Measure training time  
start_time = time.time()
```

```

# Fit the model with GridSearchCV
grid_search.fit(x_train_array, y_train_array)

# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")

# Get the best parameters from GridSearchCV
best_params = grid_search.best_params_
print(f"Best hyperparameters: {best_params}")

# Predictions using the best estimator
svm_best = grid_search.best_estimator_

y_train_pred = svm_best.predict(x_train_array)
y_test_pred = svm_best.predict(x_test_array)

# Print Classification Report
print("\nClassification Report (Train Set):")
print(classification_report(y_train_array, y_train_pred))

print("\nClassification Report (Test Set):")
print(classification_report(y_test_array, y_test_pred))

# Compute Confusion Matrices
conf_matrix_train = confusion_matrix(y_train_array, y_train_pred)
conf_matrix_test = confusion_matrix(y_test_array, y_test_pred)

# Class labels mapping
classes = [reverse_label_map[i] for i in range(len(label_map))]

# Plot Confusion Matrix for Train & Test
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Train Set Confusion Matrix
sns.heatmap(conf_matrix_train, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes, ax=axes[0])
axes[0].set_title("Confusion Matrix - Train Set")
axes[0].set_xlabel("Predicted Label")
axes[0].set_ylabel("True Label")

# Test Set Confusion Matrix
sns.heatmap(conf_matrix_test, annot=True, fmt='d', cmap='Blues',
            xticklabels=classes, yticklabels=classes, ax=axes[1])
axes[1].set_title("Confusion Matrix - Test Set")
axes[1].set_xlabel("Predicted Label")
axes[1].set_ylabel("True Label")

```

```

plt.show()

# Binarize labels for ROC curve
y_train_bin = LabelBinarizer().fit_transform(y_train_array)
y_test_bin = LabelBinarizer().fit_transform(y_test_array)

# Compute ROC Curve and AUC for Train and Test sets separately
plt.figure(figsize=(12, 6))

# Plot ROC for Train Set
plt.subplot(1, 2, 1)
plt.title('ROC Curve - Train Set')
for i in range(len(classes)):
    fpr_train, tpr_train, _ = roc_curve(y_train_bin[:, i], svm_best.
    ↪predict_proba(x_train_array)[:, i])
    roc_auc_train = auc(fpr_train, tpr_train)
    plt.plot(fpr_train, tpr_train, label=f'{classes[i]} (AUC = {roc_auc_train:.
    ↪2f})')

plt.plot([0, 1], [0, 1], color='gray', linestyle='--') # Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")

# Plot ROC for Test Set
plt.subplot(1, 2, 2)
plt.title('ROC Curve - Test Set')
for i in range(len(classes)):
    fpr_test, tpr_test, _ = roc_curve(y_test_bin[:, i], svm_best.
    ↪predict_proba(x_test_array)[:, i])
    roc_auc_test = auc(fpr_test, tpr_test)
    plt.plot(fpr_test, tpr_test, label=f'{classes[i]} (AUC = {roc_auc_test:.
    ↪2f})')

plt.plot([0, 1], [0, 1], color='gray', linestyle='--') # Diagonal line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")

plt.tight_layout()
plt.show()

```

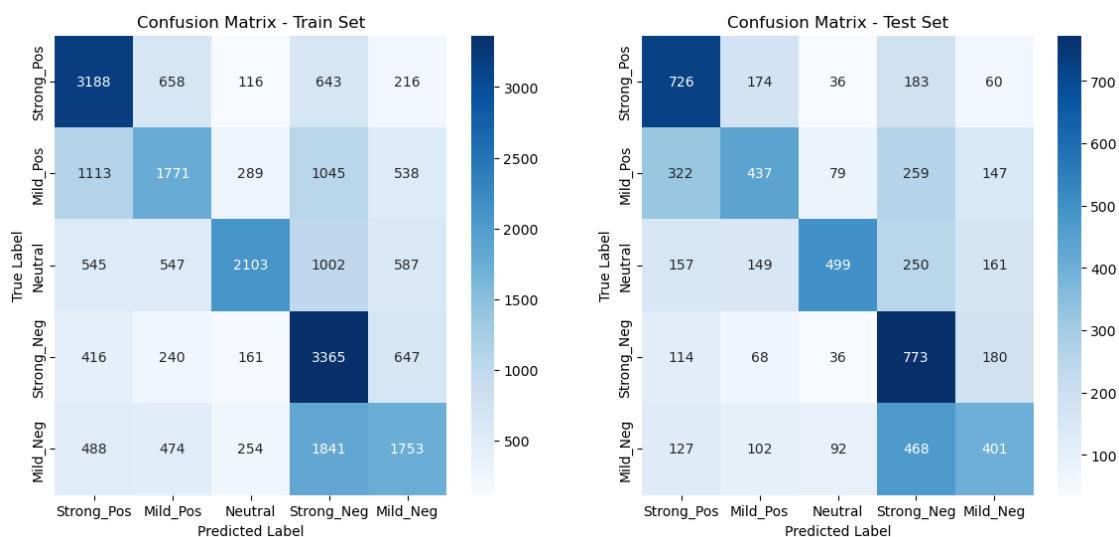
Fitting 3 folds for each of 6 candidates, totalling 18 fits  
 Training time: 848.95 seconds  
 Best hyperparameters: {'C': 10, 'kernel': 'rbf', 'probability': True}

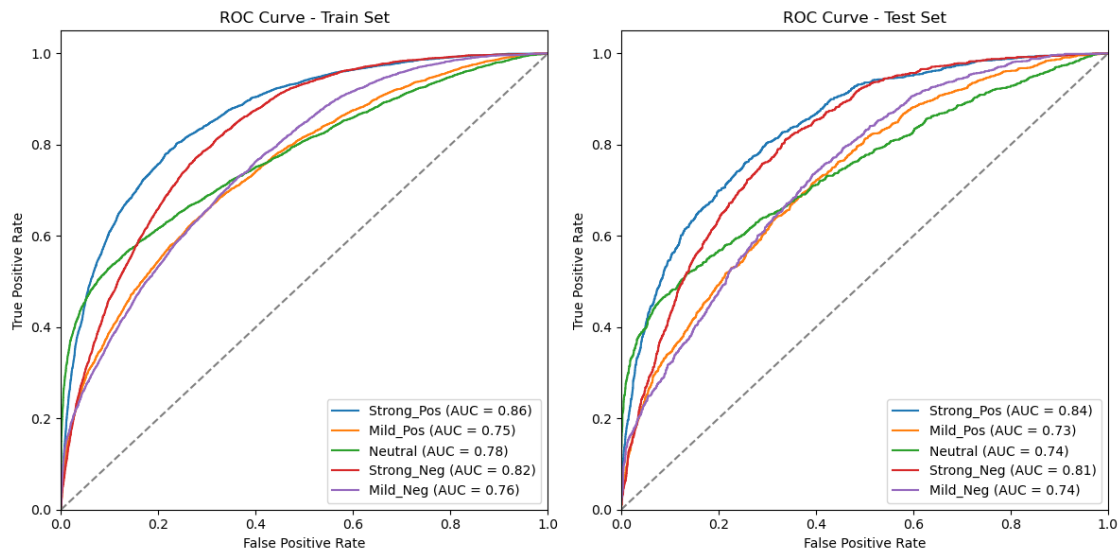
Classification Report (Train Set):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.55      | 0.66   | 0.60     | 4821    |
| 1            | 0.48      | 0.37   | 0.42     | 4756    |
| 2            | 0.72      | 0.44   | 0.55     | 4784    |
| 3            | 0.43      | 0.70   | 0.53     | 4829    |
| 4            | 0.47      | 0.36   | 0.41     | 4810    |
| accuracy     |           |        | 0.51     | 24000   |
| macro avg    | 0.53      | 0.51   | 0.50     | 24000   |
| weighted avg | 0.53      | 0.51   | 0.50     | 24000   |

Classification Report (Test Set):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.50      | 0.62   | 0.55     | 1179    |
| 1            | 0.47      | 0.35   | 0.40     | 1244    |
| 2            | 0.67      | 0.41   | 0.51     | 1216    |
| 3            | 0.40      | 0.66   | 0.50     | 1171    |
| 4            | 0.42      | 0.34   | 0.37     | 1190    |
| accuracy     |           |        | 0.47     | 6000    |
| macro avg    | 0.49      | 0.47   | 0.47     | 6000    |
| weighted avg | 0.49      | 0.47   | 0.47     | 6000    |





```
[67]: lime_explanation_single_record(svm_best, x_train_array, x_test_array[0],
    ↪reverse_label_map, word2vec_model)
```

<IPython.core.display.HTML object>

```
[67]: <lime.explanation.Explanation at 0x1ff6e20b0b0>
```

## 2 Deep Unidirectional RNN Model with an embedding layer to learn embeddings

```
[70]: import numpy as np
import pandas as pd
import tensorflow as tf
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, roc_curve,
    ↪auc
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc, accuracy_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout

# Download stopwords
nltk.download("stopwords")

```

```

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Gaurav\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

[70]: True

[72]: df

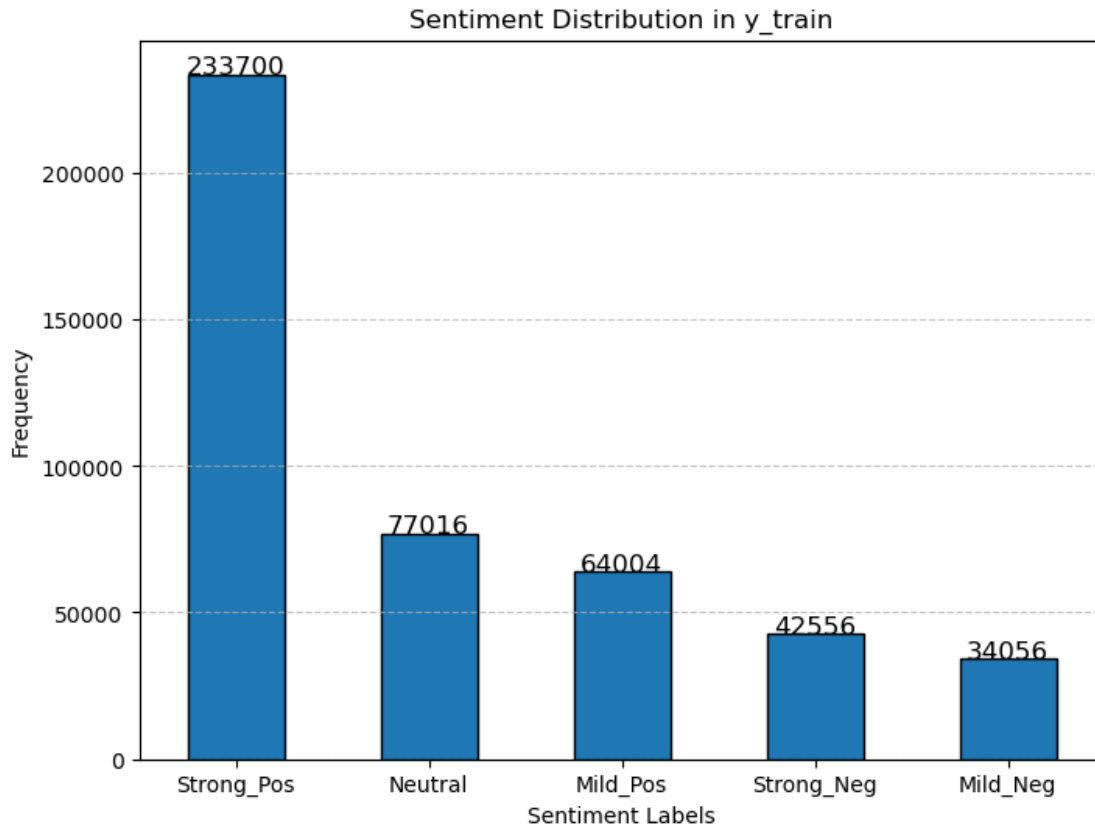
```

[72]:
          Tweet      Sentiment
0  @angelica_toy Happy Anniversary!!!...The Day...  Mild_Pos
1  @McfarlaneGlenda Happy Anniversary!!!...The D...  Mild_Pos
2  @thevivafrei @JustinTrudeau Happy Anniversary!...  Mild_Pos
3  @NChartierET Happy Anniversary!!!...The Day t...  Mild_Pos
4  @tabithapeters05 Happy Anniversary!!!...The D...  Mild_Pos
...
451327  Gaza; Peace n' Freedom - Viva Palestina convoy...  Strong_Pos
451328  Face of Defense: Soldier Finds Freedom in U.S...  Strong_Pos
451329  Face of Defense: Soldier Finds Freedom in U.S...  Strong_Pos
451330  Gaza; Peace n' Freedom - "Israel stops aid con...  Strong_Pos
451331  @convoy 83 yes! get on freedom server!  Strong_Pos

```

[451332 rows x 2 columns]

[74]: bar\_plot(df['Sentiment'])



```
[76]: # Count the number of samples per class
class_counts = df['Sentiment'].value_counts()
print("\nBefore undersampling:\n", class_counts)

# Define the number of samples to take for each class
sample_sizes = {
    'Strong_Pos':15000,
    'Mild_Pos':15000,
    'Neutral': 15000, # Adjust the sample size for Negative
    'Strong_Neg': 15000, # Adjust the sample size for Neutral
    'Mild_Neg': 15000 # Adjust the sample size for Positive
}

# Perform undersampling without replacement
df_rnn = pd.concat([
    df[df['Sentiment'] == label].sample(n=min(sample_sizes[label],
    ↪len(df[df['Sentiment'] == label])),
    random_state=42,
    replace=False)
    for label in sample_sizes.keys()
])
```

```

])

# Print class distribution after undersampling
print("\nAfter undersampling:\n", df_rnn['Sentiment'].value_counts())

```

```

Before undersampling:
Sentiment
Strong_Pos    233700
Neutral        77016
Mild_Pos       64004
Strong_Neg     42556
Mild_Neg       34056
Name: count, dtype: int64

```

```

After undersampling:
Sentiment
Strong_Pos    15000
Mild_Pos       15000
Neutral        15000
Strong_Neg     15000
Mild_Neg       15000
Name: count, dtype: int64

```

```

[78]: # Apply preprocessing
df_rnn["Cleaned_Tweet"] = df_rnn["Tweet"].apply(preprocess_stopwords_stemming)

df_rnn["Sentiment"] = df_rnn["Sentiment"].map(label_map)

# Define input (text) and target (label)
x = df_rnn["Cleaned_Tweet"].values
y = df_rnn["Sentiment"].values

print(x.shape)
print(y.shape)

print(x)
print(y)

```

```

(75000,)
(75000,)
[('viva', 'frei', 'live', 'now', 'live', 'in', 'ottawa', 'freedom',
'convoy', '2022', 'http', 'via', 'youtub')]
('laurajede', 'thi', 'is', 'exactli', 'the', 'convoy', 'type', 'in',
'canada')]
('hollyanndoan', 'q', 'to', 'freeland', 'whi', 'did', 'you', 'chuckl',
'and', 'smile', 'when', 'you', 'announc', 'you', 'were', 'freez', 'the', 'bank',
'account', 'of', 'those', 'who', 'donat', 'to', 'the', 'freedom', 'convoy')]

```



```
...
list(['so', 'the', 'freedomconvoy', 'come', 'to', 'town', 'erin', 'otool',
'get', 'oust', 'and', 'pierr', 'poiliev', 'run', 'a', 'campaign', 'on',
'freedom', 'is', 'it', 'freedom', 'they', 'are', 'chase', 'or', 'to', 'mock',
'and', 'bulli', 'trudeau'])
list(['kaizer', 'chief', 'announc', 'new', 'kit', 'sponsor', 'kappa', 'drop',
'nike', 'full', 'stori', 'http', 'skeemsaam', 'diepciti', 'gomoramzansi',
'uzalo', 'podcastandchil', 'freedom', 'convoy', 'lesetja', 'kganyago',
'panyaza', 'cheat', 'no', 'var', 'dstvprem'])
list(['i', 'guess', 'the', 'attack', 'by', 'trudeauworstpmv', 'doe', 'count',
'trudeaumustgo', 'liberalliar', 'freelandmustgo', 'jagmeetsinghmustgo',
'liberalsmustgo', 'emergencyact', 'freedomconvoy', 'freedom',
'trudeaudestroyingcanada', 'eainquiri', 'ndpcorruptasliber', 'http'])
[0 0 0 ... 4 4 4]
```

chatgpt prompt :- how to implement Deep Unidirectional RNN Model with an embedding layer to learn embeddings

```
[80]: # Tokenization and padding
max_words = 50000 # Vocabulary size
max_len = 50 # Maximum sequence length (increased to 5000)

tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(x)
sequences = tokenizer.texts_to_sequences(x)
x_padded = pad_sequences(sequences, maxlen=max_len, padding="post",
↳truncating="post")
```

```
[81]: # Split data into training and test sets
x_train, x_test, y_train, y_test = train_test_split(x_padded, y, test_size=0.2,
↳random_state=42)
```

```
[82]: print(x_train)
```

```
[[ 311   33  150 ...    0    0    0]
 [1967  412   65 ...    0    0    0]
 [  19   50    2 ...    0    0    0]
 ...
 [  51  516   57 ...    0    0    0]
 [1061  148  287 ...    0    0    0]
 [ 102   18    2 ...    0    0    0]]
```

```
[83]: # Define the model
model = Sequential()

# Embedding Layer: Learns a representation for each word in the input text
model.add(Embedding(input_dim=max_words, output_dim=100, input_length=max_len))
```

```

# Deep Unidirectional RNN layers
model.add(SimpleRNN(64, return_sequences=True)) # First RNN layer
model.add(Dropout(0.5)) # Dropout to prevent overfitting

model.add(SimpleRNN(64)) # Second RNN layer
model.add(Dropout(0.5)) # Dropout to prevent overfitting

# Output Layer: 3 classes for classification
model.add(Dense(5, activation='softmax')) # 5 classes for 5 possible labels
↳ (0, 1, 2, 3, 4)

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
↳ metrics=['accuracy'])

# Print model summary
model.summary()

```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input\_length` is deprecated. Just remove it.

warnings.warn(

Model: "sequential"

| Layer (type)             | Output Shape | Param #     |
|--------------------------|--------------|-------------|
| embedding (Embedding)    | ?            | 0 (unbuilt) |
| simple_rnn (SimpleRNN)   | ?            | 0 (unbuilt) |
| dropout (Dropout)        | ?            | 0           |
| simple_rnn_1 (SimpleRNN) | ?            | 0 (unbuilt) |
| dropout_1 (Dropout)      | ?            | 0           |
| dense (Dense)            | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[89]: start_time = time.time()
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_data=(x_test,
↪y_test))
# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")
```

```
Epoch 1/10
1875/1875          57s 29ms/step -
accuracy: 0.2447 - loss: 1.6300 - val_accuracy: 0.3115 - val_loss: 1.5386
Epoch 2/10
1875/1875          82s 28ms/step -
accuracy: 0.2795 - loss: 1.5651 - val_accuracy: 0.2703 - val_loss: 1.5811
Epoch 3/10
1875/1875          57s 30ms/step -
accuracy: 0.2647 - loss: 1.5812 - val_accuracy: 0.2553 - val_loss: 1.5816
Epoch 4/10
1875/1875          76s 27ms/step -
accuracy: 0.2623 - loss: 1.5783 - val_accuracy: 0.2541 - val_loss: 1.5852
Epoch 5/10
1875/1875          83s 28ms/step -
accuracy: 0.2409 - loss: 1.5948 - val_accuracy: 0.2463 - val_loss: 1.5931
Epoch 6/10
1875/1875          80s 27ms/step -
accuracy: 0.2456 - loss: 1.5964 - val_accuracy: 0.2334 - val_loss: 1.6087
Epoch 7/10
1875/1875          84s 28ms/step -
accuracy: 0.2205 - loss: 1.6102 - val_accuracy: 0.2392 - val_loss: 1.6113
Epoch 8/10
1875/1875          83s 28ms/step -
accuracy: 0.2334 - loss: 1.6024 - val_accuracy: 0.2535 - val_loss: 1.5890
Epoch 9/10
1875/1875          79s 26ms/step -
accuracy: 0.2510 - loss: 1.5869 - val_accuracy: 0.2497 - val_loss: 1.5918
Epoch 10/10
1875/1875          50s 27ms/step -
accuracy: 0.2650 - loss: 1.5719 - val_accuracy: 0.2407 - val_loss: 1.5943
Training time: 762.22 seconds
```

```
[91]: # Function to plot confusion matrix with reverse label map
def plot_confusion_matrix(y_true, y_pred, title, reverse_label_map):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 5))

    # Extracting the class labels from the reverse_label_map dynamically
```

```

class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.
↪keys())]

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_labels, yticklabels=class_labels)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(title)
plt.show()

```

```
[93]: print(y_train.shape)
```

```
(60000,)
```

chatgpt prompt :- Lime for Deep Unidirectional RNN Model with an embedding layer

```
[96]: import numpy as np
from lime.lime_text import LimeTextExplainer

def lime_explanation_rnn(model, sample_text_sequence, reverse_label_map,
↪max_len):
    """
    Generates a LIME explanation for a single RNN input.

    Parameters:
    - model: Trained RNN model with an embedding layer
    - sample_text_sequence: A single processed test instance (already tokenized,
↪padded)
    - reverse_label_map: Dictionary mapping class indices to labels
    - max_len: Maximum sequence length (same as used in Embedding layer)
    """

    # Define class names from reverse_label_map
    class_names = [reverse_label_map[i] for i in range(len(reverse_label_map))]

    # Initialize LIME explainer
    explainer = LimeTextExplainer(class_names=class_names)

    # Define the prediction function for LIME
    def predict_fn(text_instances):
        """
        Converts LIME text instances (words) into padded sequences & predicts,
↪probabilities.
        """
        # Convert words into numerical sequences
        sequences = [np.array(list(map(int, text.split())))) for text in
↪text_instances]

```

```

    # Pad sequences to max_len
    sequences_padded = np.array([np.pad(seq, (0, max_len - len(seq)),
↪ 'constant') for seq in sequences])

    # Get model predictions
    return model.predict(sequences_padded)

# Generate LIME explanation
exp = explainer.explain_instance(
    " ".join(map(str, sample_text_sequence)), # Convert array to
↪ space-separated string
    predict_fn,
    num_features=10,
    top_labels=5
)

# Show explanation
exp.show_in_notebook()

return exp

```

```

[98]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↪ classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
↪ round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

```

```

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
print(classification_report(y_train_true, y_train_pred_classes,
    ↪target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
    ↪target_names=class_labels))

# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix -
    ↪Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix -
    ↪Test Data", reverse_label_map)

```

1875/1875                      14s 7ms/step  
469/469                        4s 8ms/step

(60000, 5)  
(15000, 5)  
(60000,)  
(15000,)

Train Accuracy: 0.2676  
Test Accuracy: 0.2407

#### Classification Report - Train Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.32      | 0.50   | 0.39     | 12003   |
| Mild_Pos     | 0.24      | 0.04   | 0.06     | 12005   |
| Neutral      | 0.00      | 0.00   | 0.00     | 11917   |
| Strong_Neg   | 0.24      | 0.26   | 0.25     | 12004   |
| Mild_Neg     | 0.24      | 0.54   | 0.33     | 12071   |
| accuracy     |           |        | 0.27     | 60000   |
| macro avg    | 0.21      | 0.27   | 0.21     | 60000   |
| weighted avg | 0.21      | 0.27   | 0.21     | 60000   |

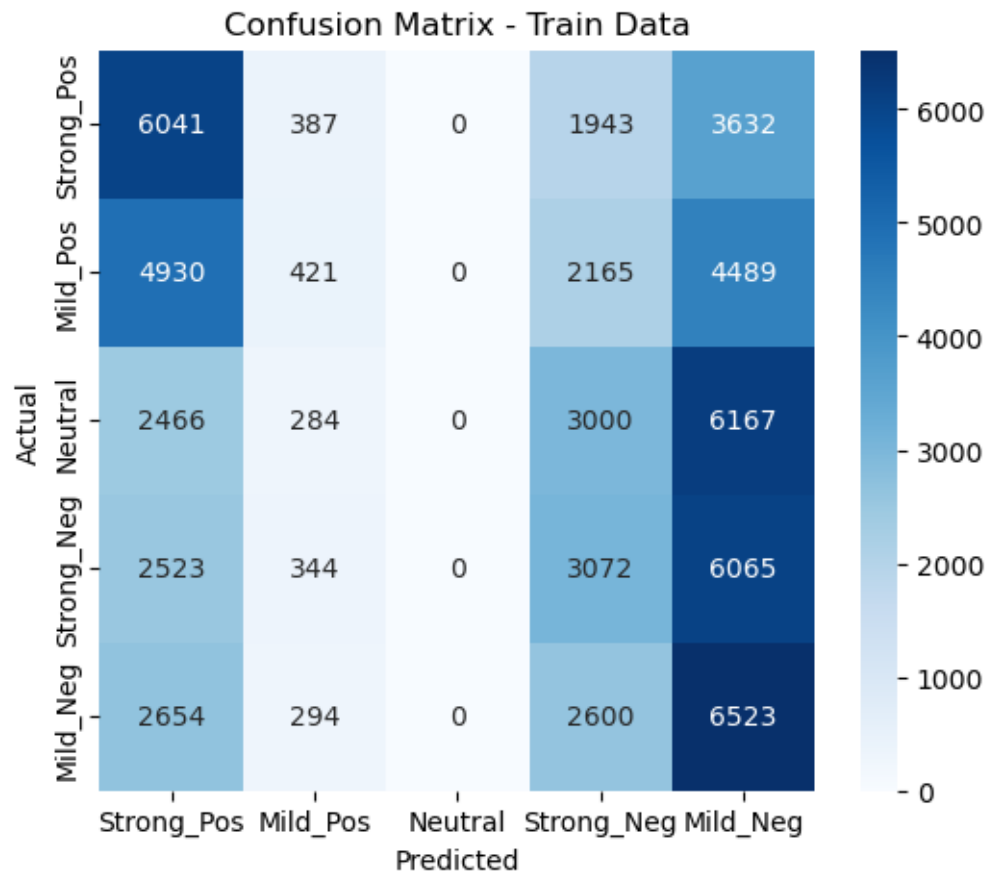
# Classification Report - Test Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.29      | 0.42   | 0.35     | 2997    |
| Mild_Pos     | 0.21      | 0.03   | 0.05     | 2995    |
| Neutral      | 0.00      | 0.00   | 0.00     | 3083    |
| Strong_Neg   | 0.23      | 0.23   | 0.23     | 2996    |
| Mild_Neg     | 0.22      | 0.53   | 0.31     | 2929    |
| accuracy     |           |        | 0.24     | 15000   |
| macro avg    | 0.19      | 0.24   | 0.19     | 15000   |
| weighted avg | 0.19      | 0.24   | 0.19     | 15000   |

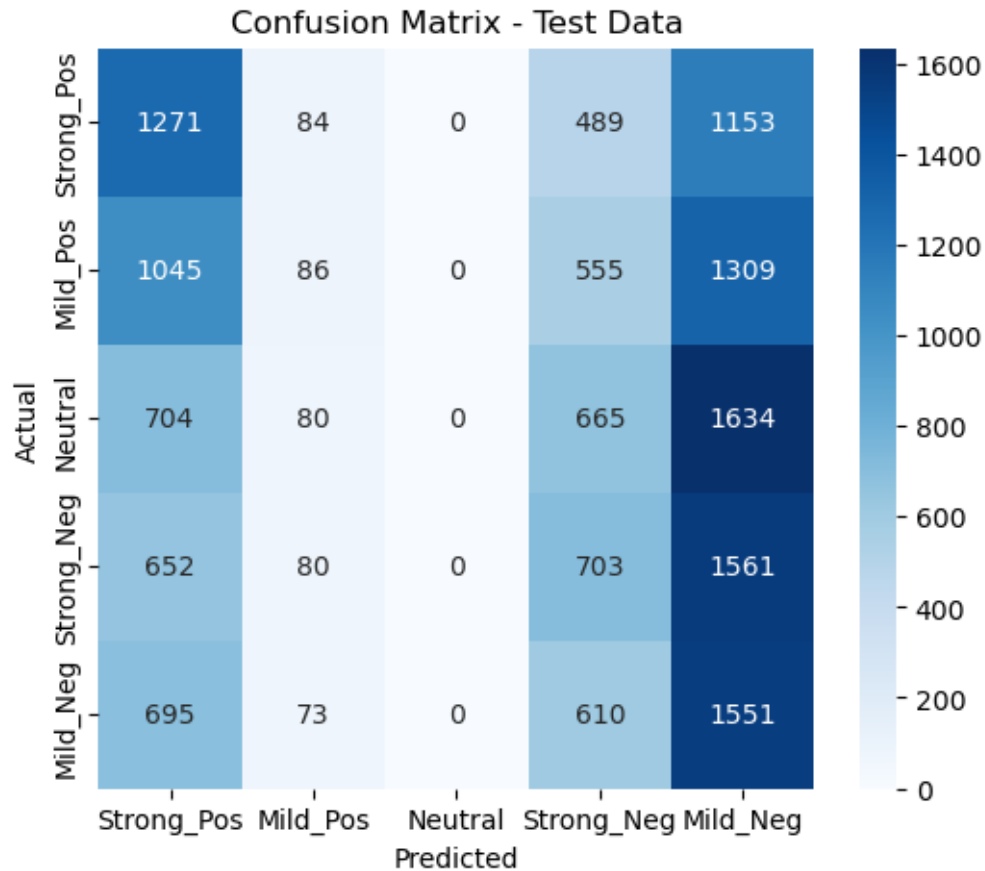
```

C:\Users\Gaurav\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gaurav\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gaurav\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gaurav\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gaurav\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gaurav\anaconda3\Lib\site-
packages\sklearn\metrics\_classification.py:1531: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```







chatgpt prompts :- get the top 10 features with heighest weights

```
[101]: import numpy as np

# Get the weights from the Embedding layer
embedding_weights = model.layers[0].get_weights()[0] # Embedding layer is the
↳ first layer (index 0)

# Get the word index mapping from the tokenizer
word_index = tokenizer.word_index

# Calculate the magnitude (norm) of each word vector in the embedding layer
norms = np.linalg.norm(embedding_weights, axis=1)

# Get the indices of the top 10 largest norms
top_10_indices = np.argsort(norms)[::-1][:10]

# Get the corresponding words from the word index
```

```

top_10_words = [word for word, index in word_index.items() if index in
    ↪top_10_indices]

# Print the top 10 words with the highest weights
print("Top 10 Features with the Highest Weights:")
for i, word in enumerate(top_10_words, 1):
    print(f"{i}. {word} (Weight: {norms[word_index[word] - 1]:.4f})")

```

Top 10 Features with the Highest Weights:

1. thank (Weight: 0.9253)
2. accord (Weight: 0.7350)
3. ce (Weight: 0.4380)
4. success (Weight: 1.1525)
5. cupe (Weight: 0.9899)
6. ear (Weight: 1.2406)
7. grip (Weight: 0.8045)
8. whoever (Weight: 0.7774)
9. unitedwestand (Weight: 0.4460)
10. protesti (Weight: 0.5422)

```

[103]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
    ↪y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
    ↪y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

```

```

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

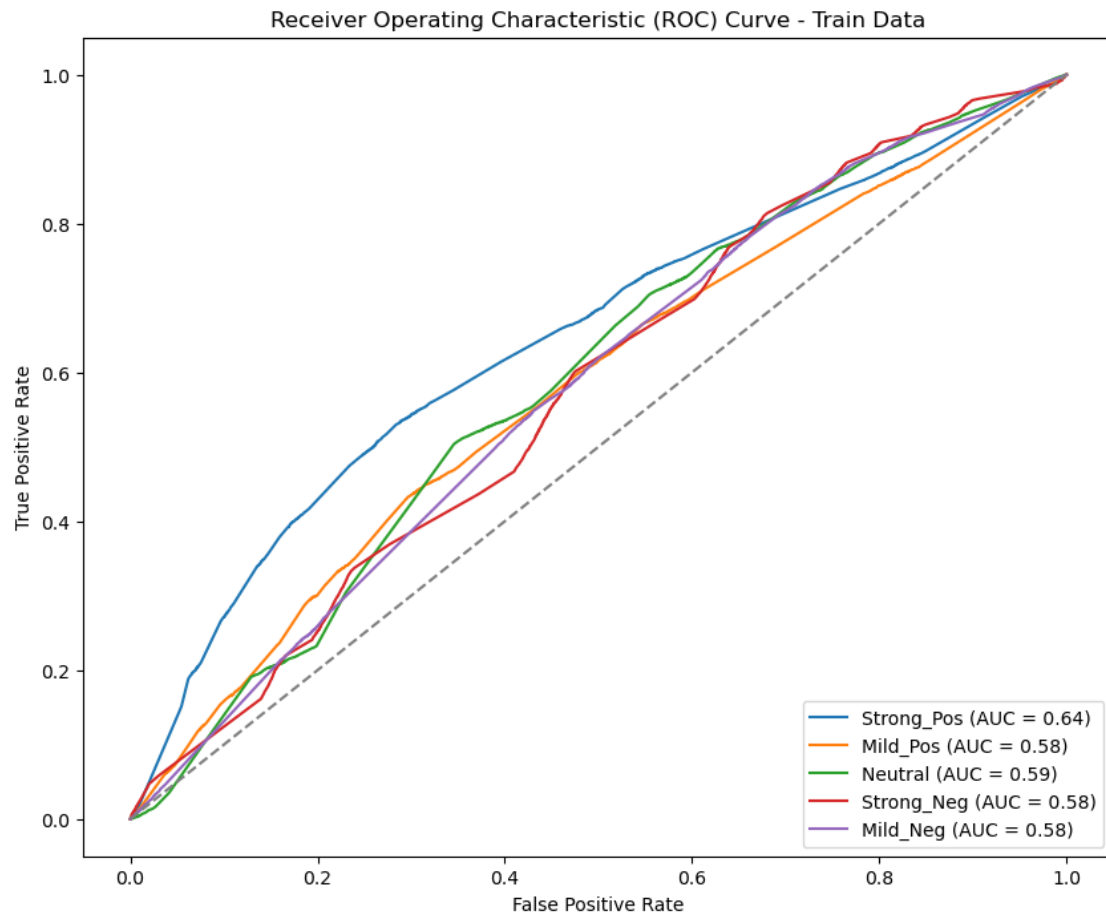
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

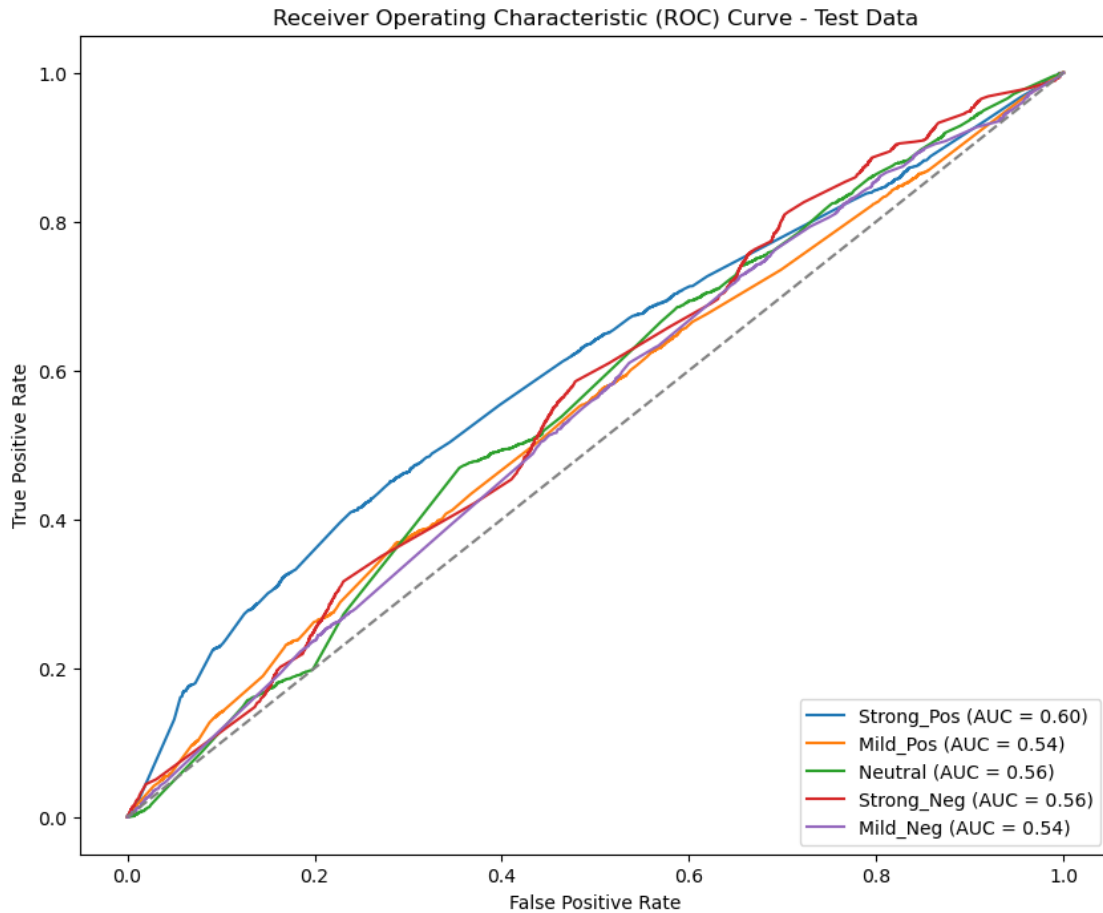
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_test[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```





Lime

```
[106]: lime_explanation_rnn(model, x_test[0], reverse_label_map, 50)
```

157/157

2s 11ms/step

<IPython.core.display.HTML object>

```
[106]: <lime.explanation.Explanation at 0x1ff7df417f0>
```

adding rnn layers to improve accuracy and balancing all weights

chatgpt prompt :- add more layers for unidirectional rnn model and balance the class weights & increase the epoch size and batch size

```
[110]: from keras.models import Sequential
from keras.layers import Embedding, SimpleRNN, Dropout, Dense
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
```

```

# Calculate class weights to address class imbalance
class_weights = compute_class_weight('balanced', classes=np.unique(y_train),
    ↪y=y_train)
class_weights = dict(enumerate(class_weights))

# Define the model
model = Sequential()

# Embedding Layer: Learns a representation for each word in the input text
model.add(Embedding(input_dim=max_words, output_dim=100, input_length=max_len))

# First Unidirectional SimpleRNN Layer
model.add(SimpleRNN(128, return_sequences=True)) # Return sequences for the
    ↪next RNN layer
model.add(Dropout(0.5)) # Dropout to prevent overfitting

# Second Unidirectional SimpleRNN Layer
model.add(SimpleRNN(128, return_sequences=True)) # Return sequences for the
    ↪next RNN layer
model.add(Dropout(0.5)) # Dropout to prevent overfitting

# Third Unidirectional SimpleRNN Layer
model.add(SimpleRNN(128)) # No need to return sequences here, as it's the
    ↪final layer
model.add(Dropout(0.5)) # Dropout to prevent overfitting

# Output Layer: 5 classes for classification
model.add(Dense(5, activation='softmax')) # 5 classes for 5 possible labels
    ↪(0, 1, 2, 3, 4)

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
    ↪metrics=['accuracy'])

# Print model summary
model.summary()

# Train the model with class weights
start_time = time.time()
model.fit(x_train, y_train, epochs=10, batch_size=150, validation_data=(x_test,
    ↪y_test), class_weight=class_weights)

# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")

```

C:\Users\Gaurav\anaconda3\Lib\site-

```
packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential\_1"

| Layer (type)                               | Output Shape | Param #     |
|--------------------------------------------|--------------|-------------|
| embedding_1 ( <a href="#">Embedding</a> )  | ?            | 0 (unbuilt) |
| simple_rnn_2 ( <a href="#">SimpleRNN</a> ) | ?            | 0 (unbuilt) |
| dropout_2 ( <a href="#">Dropout</a> )      | ?            | 0           |
| simple_rnn_3 ( <a href="#">SimpleRNN</a> ) | ?            | 0 (unbuilt) |
| dropout_3 ( <a href="#">Dropout</a> )      | ?            | 0           |
| simple_rnn_4 ( <a href="#">SimpleRNN</a> ) | ?            | 0 (unbuilt) |
| dropout_4 ( <a href="#">Dropout</a> )      | ?            | 0           |
| dense_1 ( <a href="#">Dense</a> )          | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

400/400 51s 117ms/step -

accuracy: 0.2303 - loss: 1.8153 - val\_accuracy: 0.2937 - val\_loss: 1.5542

Epoch 2/10

400/400 81s 114ms/step -

accuracy: 0.3239 - loss: 1.5164 - val\_accuracy: 0.4595 - val\_loss: 1.3057

Epoch 3/10

400/400 83s 117ms/step -

accuracy: 0.5142 - loss: 1.1958 - val\_accuracy: 0.5039 - val\_loss: 1.3056

Epoch 4/10

400/400 82s 118ms/step -

accuracy: 0.5940 - loss: 1.0583 - val\_accuracy: 0.5149 - val\_loss: 1.2323

Epoch 5/10

400/400 83s 121ms/step -

```

accuracy: 0.6353 - loss: 0.9744 - val_accuracy: 0.5443 - val_loss: 1.1997
Epoch 6/10
400/400          80s 115ms/step -
accuracy: 0.6648 - loss: 0.9217 - val_accuracy: 0.5525 - val_loss: 1.2200
Epoch 7/10
400/400          83s 116ms/step -
accuracy: 0.6837 - loss: 0.8846 - val_accuracy: 0.5413 - val_loss: 1.2491
Epoch 8/10
400/400          83s 118ms/step -
accuracy: 0.7073 - loss: 0.8369 - val_accuracy: 0.5351 - val_loss: 1.3794
Epoch 9/10
400/400          80s 114ms/step -
accuracy: 0.7343 - loss: 0.7761 - val_accuracy: 0.5425 - val_loss: 1.2796
Epoch 10/10
400/400          84s 119ms/step -
accuracy: 0.7526 - loss: 0.7306 - val_accuracy: 0.5341 - val_loss: 1.2526
Training time: 823.99 seconds

```

```

[111]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↳classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
↳round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

```



```

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
print(classification_report(y_train_true, y_train_pred_classes,
    ↳target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
    ↳target_names=class_labels))

# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix -
    ↳Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix -
    ↳Test Data", reverse_label_map)

```

1875/1875                      28s 15ms/step

469/469                        8s 16ms/step

(60000, 5)

(15000, 5)

(60000,)

(15000,)

Train Accuracy: 0.7712

Test Accuracy: 0.5341

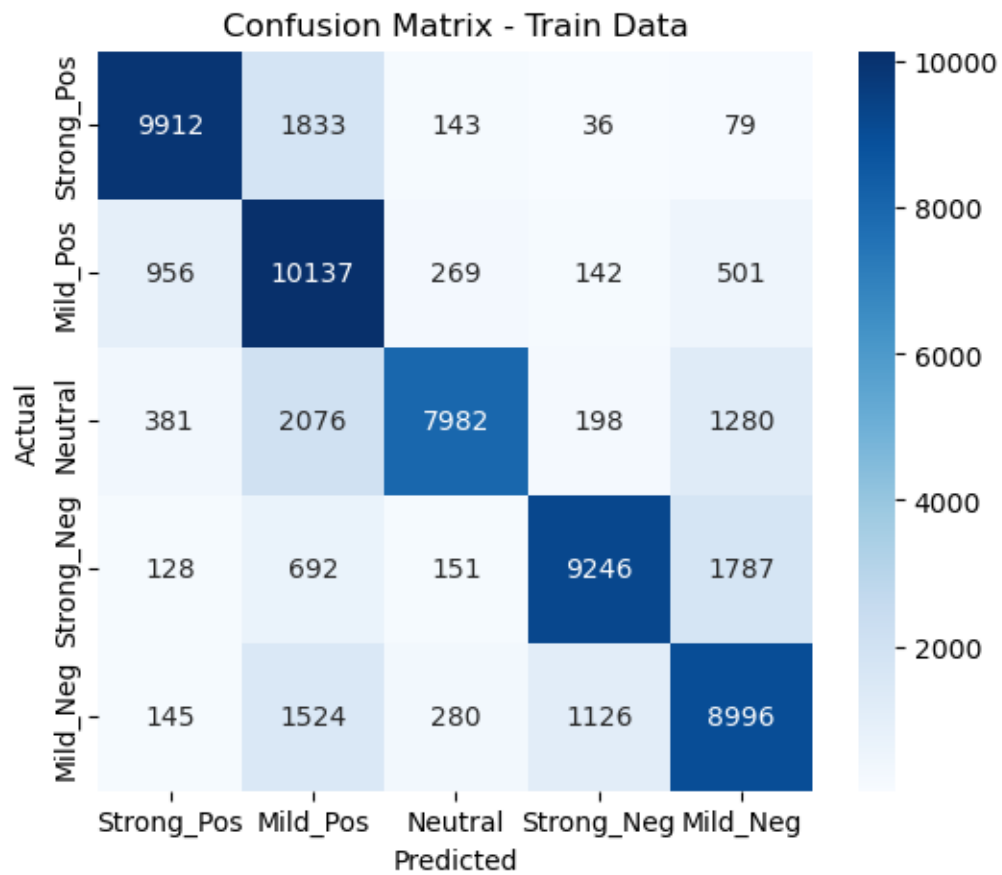
Classification Report - Train Data

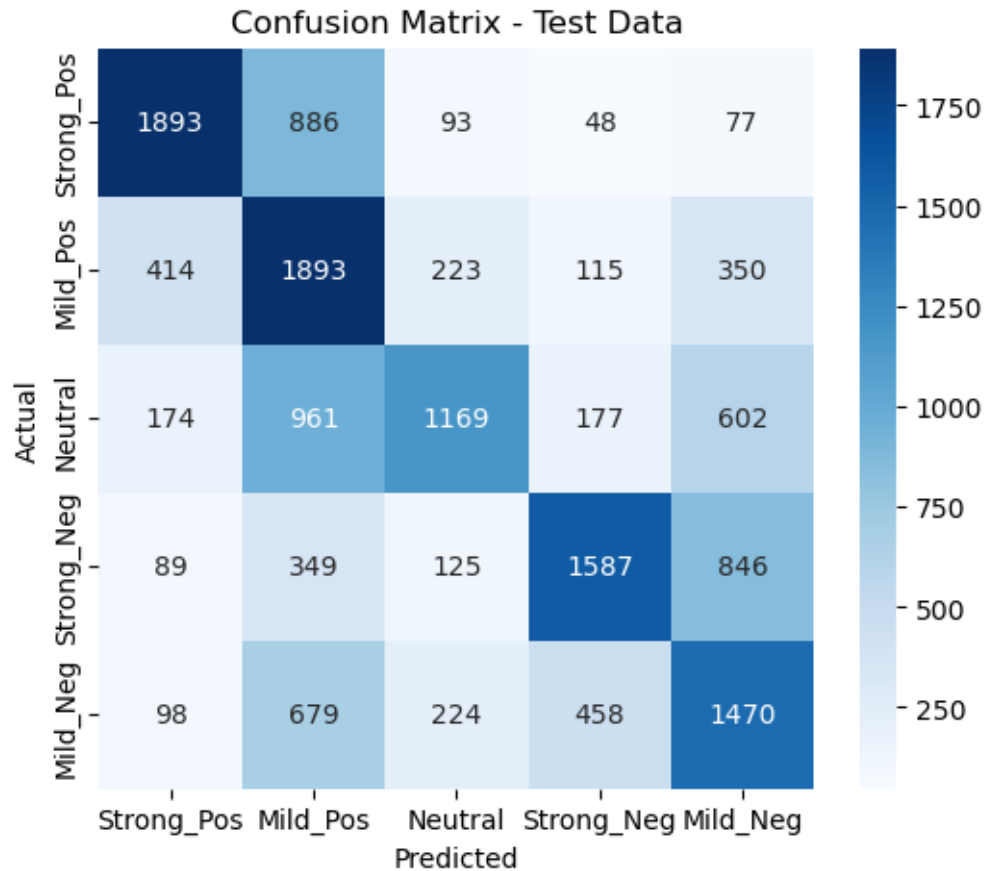
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.86      | 0.83   | 0.84     | 12003   |
| Mild_Pos     | 0.62      | 0.84   | 0.72     | 12005   |
| Neutral      | 0.90      | 0.67   | 0.77     | 11917   |
| Strong_Neg   | 0.86      | 0.77   | 0.81     | 12004   |
| Mild_Neg     | 0.71      | 0.75   | 0.73     | 12071   |
| accuracy     |           |        | 0.77     | 60000   |
| macro avg    | 0.79      | 0.77   | 0.77     | 60000   |
| weighted avg | 0.79      | 0.77   | 0.77     | 60000   |

Classification Report - Test Data

|  | precision | recall | f1-score | support |
|--|-----------|--------|----------|---------|
|--|-----------|--------|----------|---------|

|              |      |      |      |       |
|--------------|------|------|------|-------|
| Strong_Pos   | 0.71 | 0.63 | 0.67 | 2997  |
| Mild_Pos     | 0.40 | 0.63 | 0.49 | 2995  |
| Neutral      | 0.64 | 0.38 | 0.48 | 3083  |
| Strong_Neg   | 0.67 | 0.53 | 0.59 | 2996  |
| Mild_Neg     | 0.44 | 0.50 | 0.47 | 2929  |
| accuracy     |      |      | 0.53 | 15000 |
| macro avg    | 0.57 | 0.53 | 0.54 | 15000 |
| weighted avg | 0.57 | 0.53 | 0.54 | 15000 |





```
[112]: import numpy as np

# Get the weights from the Embedding layer
embedding_weights = model.layers[0].get_weights()[0] # Embedding layer is the
↳ first layer (index 0)

# Get the word index mapping from the tokenizer
word_index = tokenizer.word_index

# Calculate the magnitude (norm) of each word vector in the embedding layer
norms = np.linalg.norm(embedding_weights, axis=1)

# Get the indices of the top 10 largest norms
top_10_indices = np.argsort(norms)[::-1][:10]

# Get the corresponding words from the word index
top_10_words = [word for word, index in word_index.items() if index in
↳ top_10_indices]
```

```

# Print the top 10 words with the highest weights
print("Top 10 Features with the Highest Weights:")
for i, word in enumerate(top_10_words, 1):
    print(f"{i}. {word} (Weight: {norms[word_index[word] - 1]:.4f})")

```

Top 10 Features with the Highest Weights:

```

1. kill (Weight: 0.6513)
2. hell (Weight: 1.2383)
3. crisi (Weight: 0.6268)
4. murder (Weight: 0.5340)
5. dead (Weight: 0.4674)
6. suicid (Weight: 0.9388)
7. perfectli (Weight: 0.5793)
8. poverti (Weight: 0.6381)
9. doom (Weight: 0.4587)
10. drape (Weight: 0.5140)

```

```

[113]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust
↳based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust
↳based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
↳y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
↳y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))

```

```

for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

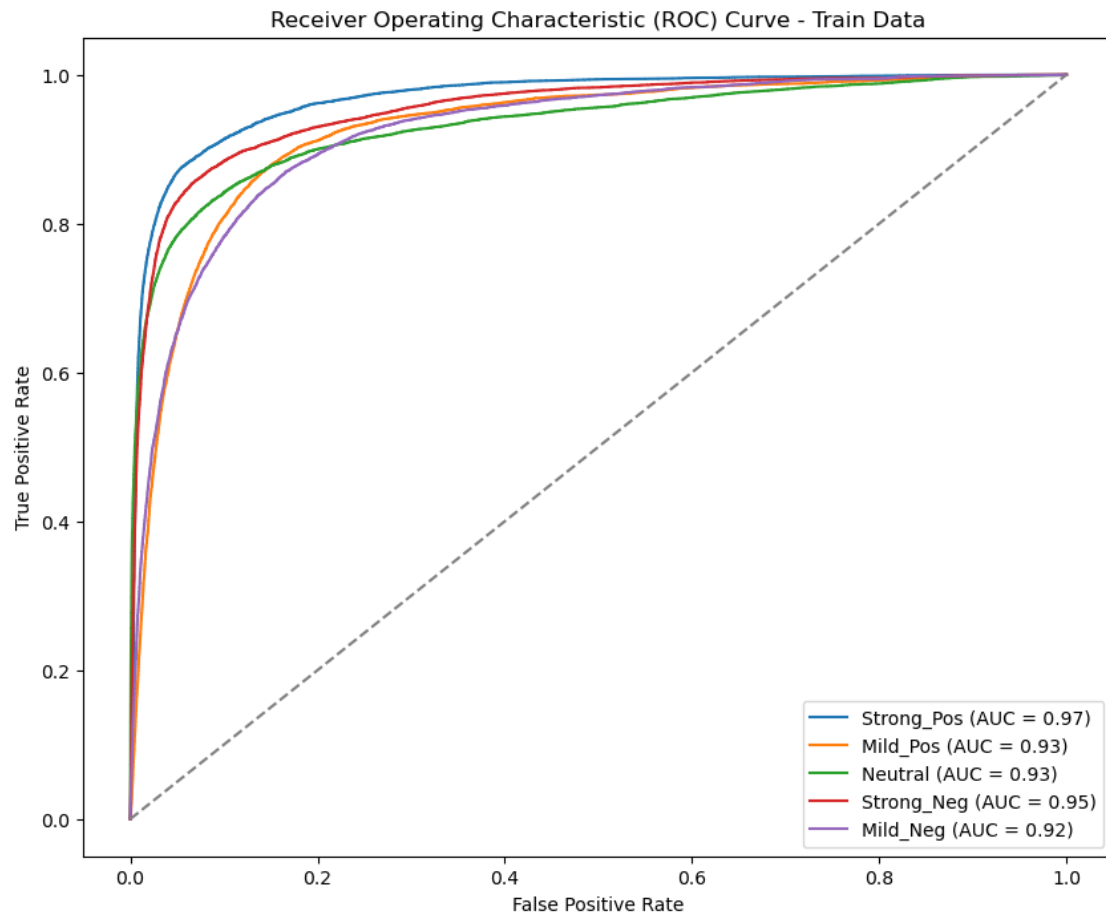
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

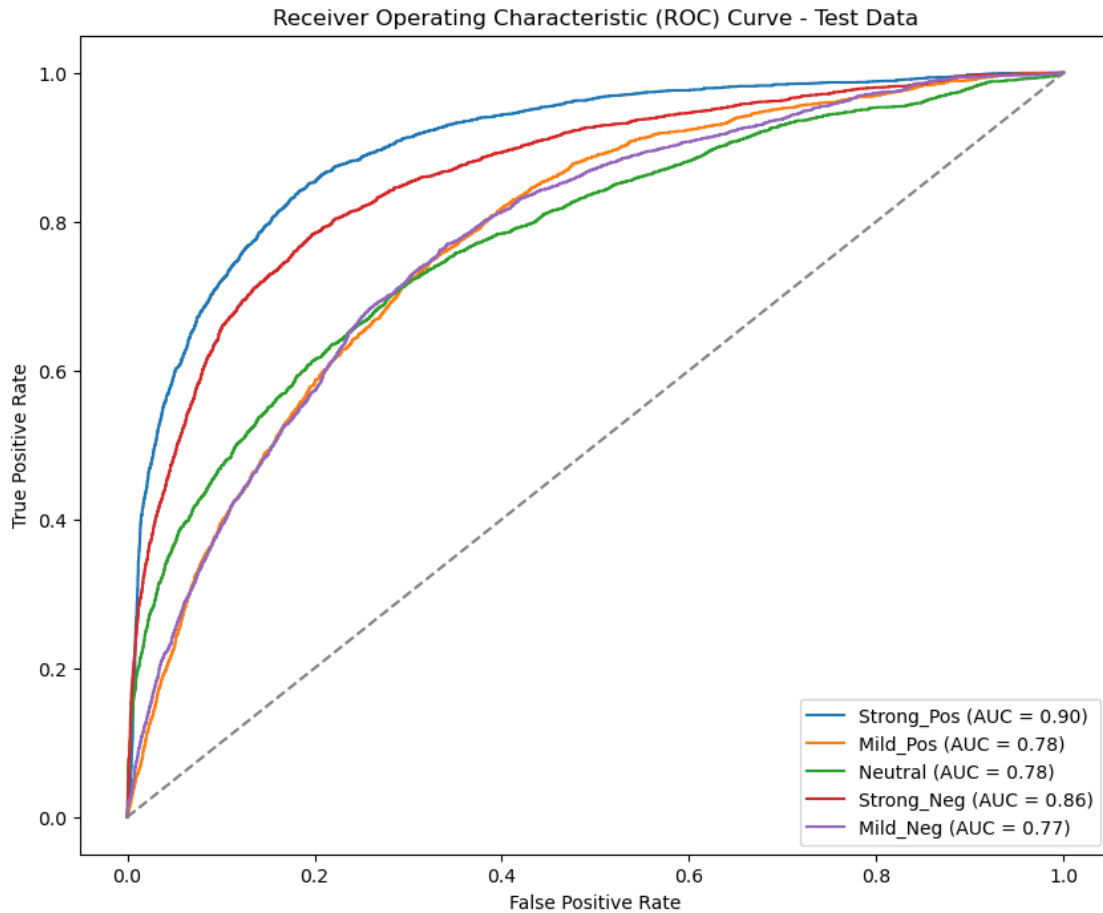
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_test[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```





we can train the model with data that has test data characteristics or increase the rnn layers or do hyperparameter tuning to increase the accuracy

```
[119]: print(x_test[0])
```

```
[18030 30909 30910 18031 18032 18033 18034 18035 30911 0 0 0
      0 0 0 0 0 0 0 0 0 0 0 0
      0 0 0 0 0 0 0 0 0 0 0 0
      0 0 0 0 0 0 0 0 0 0 0 0
      0 0]
```

```
[121]: print(reverse_label_map[y_test[0]])
```

Mild\_Pos

Lime

```
[124]: lime_explanation_rnn(model, x_test[0], reverse_label_map, 50)
```

157/157

3s 16ms/step

<IPython.core.display.HTML object>

[124]: <lime.explanation.Explanation at 0x1ff977a8d10>

### 3 Single Layer Bidirectional GRU Model using word2vec (CBOW or Skipgram and any vector size from 64 to 300)

```
[127]: import numpy as np
import nltk
from nltk.stem import PorterStemmer
from keras.models import Sequential
from keras.layers import GRU, Bidirectional, Dense, Dropout
from keras.utils import to_categorical
from sklearn.model_selection import train_test_split
from gensim.models import Word2Vec
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
import time
```

[129]: df

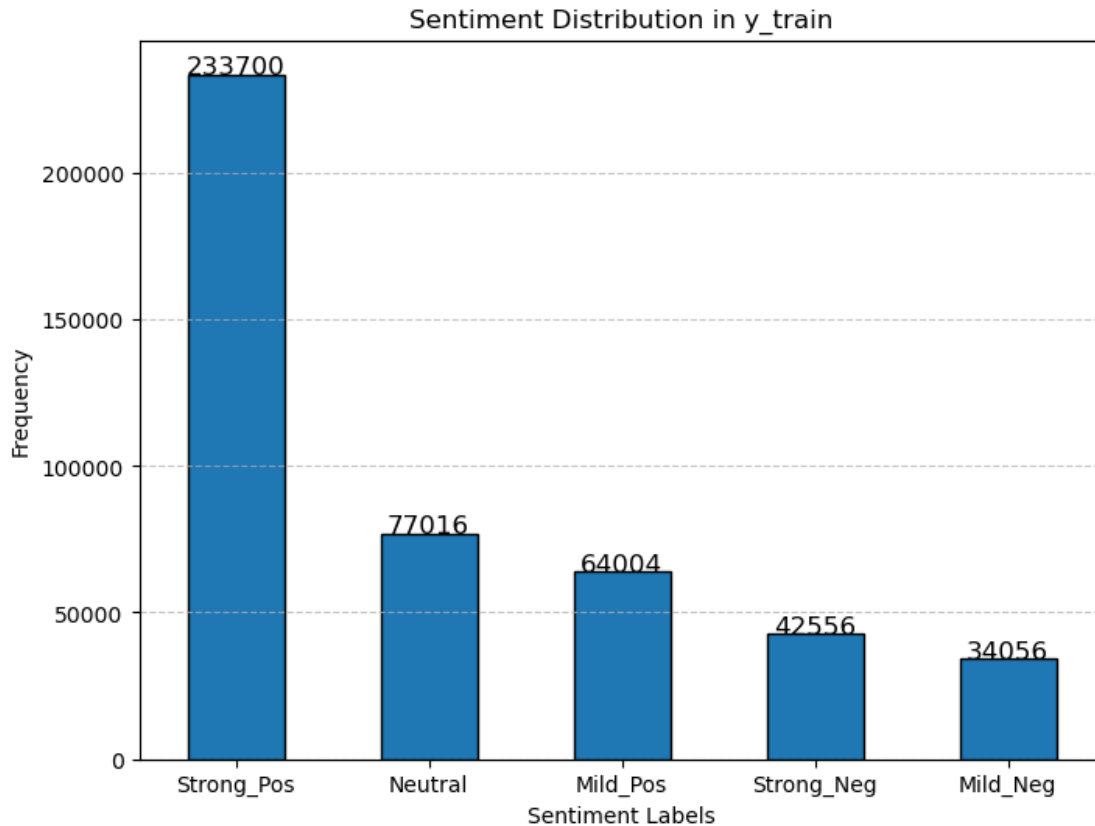
```
[129]:
```

|        |                                                   | Tweet                                | Sentiment  |
|--------|---------------------------------------------------|--------------------------------------|------------|
| 0      | @angelica_toy                                     | Happy Anniversary!!!!...The Day...   | Mild_Pos   |
| 1      | @McfarlaneGlenda                                  | Happy Anniversary!!!!...The D...     | Mild_Pos   |
| 2      | @thevivafrei                                      | @JustinTrudeau Happy Anniversary!... | Mild_Pos   |
| 3      | @NChartierET                                      | Happy Anniversary!!!!...The Day t... | Mild_Pos   |
| 4      | @tabithapeters05                                  | Happy Anniversary!!!!...The D...     | Mild_Pos   |
| ...    |                                                   | ...                                  | ...        |
| 451327 | Gaza; Peace n' Freedom - Viva Palestina convoy... |                                      | Strong_Pos |
| 451328 | Face of Defense: Soldier Finds Freedom in U.S...  |                                      | Strong_Pos |
| 451329 | Face of Defense: Soldier Finds Freedom in U.S...  |                                      | Strong_Pos |
| 451330 | Gaza; Peace n' Freedom - "Israel stops aid con... |                                      | Strong_Pos |
| 451331 | @convoy 83 yes! get on freedom server!            |                                      | Strong_Pos |

[451332 rows x 2 columns]

```
[131]: bar_plot(df['Sentiment'])
```





```
[133]: # Count the number of samples per class
class_counts = df['Sentiment'].value_counts()
print("\nBefore undersampling:\n", class_counts)

# Define the number of samples to take for each class
sample_sizes = {
    'Strong_Pos': 30000,
    'Mild_Pos': 30000,
    'Neutral': 30000, # Adjust the sample size for Negative
    'Strong_Neg': 30000, # Adjust the sample size for Neutral
    'Mild_Neg': 30000 # Adjust the sample size for Positive
}

# Perform undersampling without replacement
df_gru = pd.concat([
    df[df['Sentiment'] == label].sample(n=min(sample_sizes[label],
    ↪len(df[df['Sentiment'] == label])),
    random_state=42,
    replace=False)
    for label in sample_sizes.keys()])
```

```
])
```

```
# Print class distribution after undersampling  
print("\nAfter undersampling:\n", df_gru['Sentiment'].value_counts())
```

Before undersampling:

| Sentiment  |        |
|------------|--------|
| Strong_Pos | 233700 |
| Neutral    | 77016  |
| Mild_Pos   | 64004  |
| Strong_Neg | 42556  |
| Mild_Neg   | 34056  |

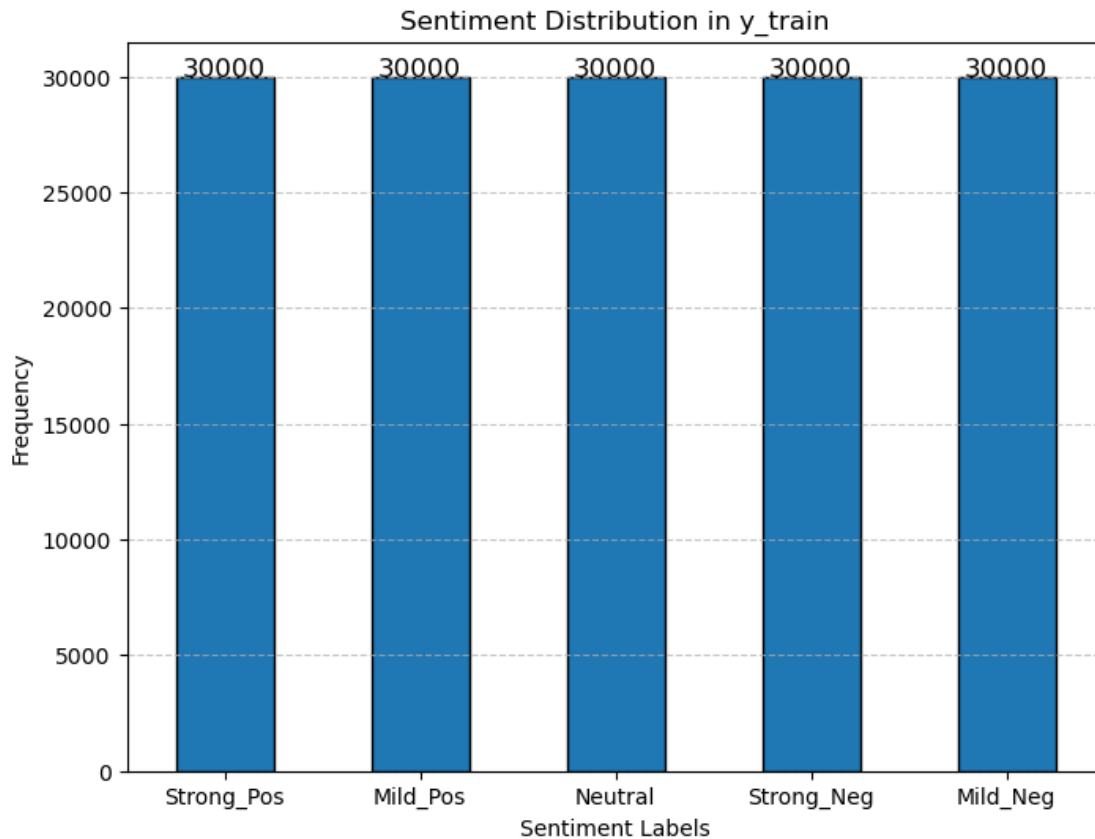
Name: count, dtype: int64

After undersampling:

| Sentiment  |       |
|------------|-------|
| Strong_Pos | 30000 |
| Mild_Pos   | 30000 |
| Neutral    | 30000 |
| Strong_Neg | 30000 |
| Mild_Neg   | 30000 |

Name: count, dtype: int64

```
[134]: bar_plot(df_gru['Sentiment'])
```



```
[137]: df_gru['Sentiment'] = df_gru['Sentiment'].map(label_map)
y = df_gru['Sentiment']
```

```
[139]: # Apply preprocessing to the text data
df_gru['stem_tokens'] = df_gru['Tweet'].apply(preprocess_stopwords_stemming)
```

```
[140]: # Train a Word2Vec model (CBOW model)
word2vec_model = Word2Vec(sentences = df_gru['stem_tokens'], vector_size=200,
    ↪window=5, min_count=1, sg=0)
```

chatgpt prompt :- implement word2vector for the tweet data

```
[142]: import numpy as np

# Function to convert a tweet (list of words) into a vector
def get_tweet_vector(words, model, vector_size=200):
    vec = np.zeros(vector_size)
    count = 0
    for word in words: # No need to split, 'words' is already a list
        if word in model.wv:
```

```

        vec += model.wv[word]
        count += 1
    if count > 0:
        vec /= count  # Average word vectors
    return vec

```

```

[179]: x = np.array([get_tweet_vector(tweet, word2vec_model) for tweet in
↳df_gru['stem_tokens']])

```

```

[181]: # Split into train and test sets (80% train, 20% test)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
↳random_state=42)

```

```

[183]: from lime.lime_tabular import LimeTabularExplainer
import numpy as np

def explain_single_instance_with_lime(model, x_test_instance, num_features=10):
    """
    Explains a single instance from x_test using LIME.

    Parameters:
    - model: The trained Bidirectional GRU model
    - x_test_instance: A single test instance (e.g., x_test[0]) to explain
    - num_features: Number of features to display in the explanation

    Returns:
    - Explanation object containing the explanation of the prediction
    """

    # Flatten the instance for LIME
    x_test_instance_flattened = x_test_instance.reshape(1, -1)  # Flatten the
↳instance (timesteps * features)

    # Function to predict class probabilities (same as model.predict)
    def predict_fn(x):
        # Reshape the input back to (batch_size, timesteps, features) for the
↳GRU model
        x_resaped = x.reshape(x.shape[0], x_test_instance.shape[0],
↳x_test_instance.shape[1])
        return model.predict(x_resaped)

    # Initialize LIME's Tabular Explainer
    explainer = LimeTabularExplainer(
        training_data=x_train.reshape(x_train.shape[0], -1),  # Flatten
↳training data to match model input shape
        mode="classification",  # Classification task

```

```

        class_names=list(reverse_label_map.values()), # Using
↪reverse_label_map for class names
        feature_names=[f"Feature {i}" for i in range(x_test_instance_flattened.
↪shape[1])], # Flattened feature names
        discretize_continuous=True # Handle continuous features
    )

    # Explain the instance
    explanation = explainer.explain_instance(
        x_test_instance_flattened[0], # Flattened instance
        predict_fn, # The prediction function for the model
        num_features=num_features, # Limit to the top N features
        top_labels=5
    )

    # Display the explanation (in notebook, or in a web-based view)
    explanation.show_in_notebook()

    return explanation

```

chatgpt prompt :- Single Layer Bidirectional GRU Model using word2vec (CBOW or Skipgram and any vector size from 64 to 300)

```

[186]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GRU, Bidirectional
import time

# Reshape input to 3D for GRU
x_train = x_train.reshape((x_train.shape[0], 1, x_train.shape[1]))
x_test = x_test.reshape((x_test.shape[0], 1, x_test.shape[1]))

# Define the Bidirectional GRU model
model = Sequential()
model.add(Bidirectional(GRU(128, dropout=0.2, recurrent_dropout=0.2,
↪input_shape=(1, x_train.shape[2]))))

# Output layer
model.add(Dense(5, activation='softmax'))

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
↪metrics=['accuracy'])

# Track training time
start_time = time.time()

# Train the model

```

```

history = model.fit(x_train, y_train, epochs=10, batch_size=125,
                    validation_data=(x_test, y_test))

# Calculate training time
end_time = time.time()
training_time = end_time - start_time
print(f"Training time: {training_time:.2f} seconds")

# Evaluate on test data
loss, accuracy = model.evaluate(x_test, y_test)
print(f'Test Loss: {loss}')
print(f'Test Accuracy: {accuracy}')

```

Epoch 1/10

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:200:  
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When  
using Sequential models, prefer using an `Input(shape)` object as the first  
layer in the model instead.

```
super().__init__(**kwargs)
```

```
960/960          11s 6ms/step -
accuracy: 0.3869 - loss: 1.4037 - val_accuracy: 0.4485 - val_loss: 1.3007
```

Epoch 2/10

```
960/960          5s 6ms/step -
accuracy: 0.4438 - loss: 1.3111 - val_accuracy: 0.4633 - val_loss: 1.2628
```

Epoch 3/10

```
960/960          6s 6ms/step -
accuracy: 0.4605 - loss: 1.2810 - val_accuracy: 0.4777 - val_loss: 1.2432
```

Epoch 4/10

```
960/960          5s 6ms/step -
accuracy: 0.4699 - loss: 1.2630 - val_accuracy: 0.4878 - val_loss: 1.2313
```

Epoch 5/10

```
960/960          5s 6ms/step -
accuracy: 0.4768 - loss: 1.2478 - val_accuracy: 0.4999 - val_loss: 1.2082
```

Epoch 6/10

```
960/960          6s 6ms/step -
accuracy: 0.4826 - loss: 1.2311 - val_accuracy: 0.5013 - val_loss: 1.2016
```

Epoch 7/10

```
960/960          5s 6ms/step -
accuracy: 0.4868 - loss: 1.2242 - val_accuracy: 0.5054 - val_loss: 1.1897
```

Epoch 8/10

```
960/960          6s 6ms/step -
accuracy: 0.4904 - loss: 1.2129 - val_accuracy: 0.5086 - val_loss: 1.1881
```

Epoch 9/10

```
960/960          9s 5ms/step -
accuracy: 0.4967 - loss: 1.2049 - val_accuracy: 0.5142 - val_loss: 1.1780
```

Epoch 10/10

```

960/960          5s 5ms/step -
accuracy: 0.5036 - loss: 1.1949 - val_accuracy: 0.5141 - val_loss: 1.1742
Training time: 64.09 seconds
938/938          2s 2ms/step -
accuracy: 0.5110 - loss: 1.1839
Test Loss: 1.174174189567566
Test Accuracy: 0.5141000151634216

```

```

[188]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↳classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
↳round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")

```

```

print(classification_report(y_train_true, y_train_pred_classes,
    ↳target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
    ↳target_names=class_labels))

# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix -
    ↳Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix -
    ↳Test Data", reverse_label_map)

```

```

3750/3750          7s 2ms/step
938/938           2s 3ms/step
(120000, 5)
(30000, 5)
(120000,)
(30000,)
Train Accuracy: 0.5274
Test Accuracy: 0.5141

```

#### Classification Report - Train Data

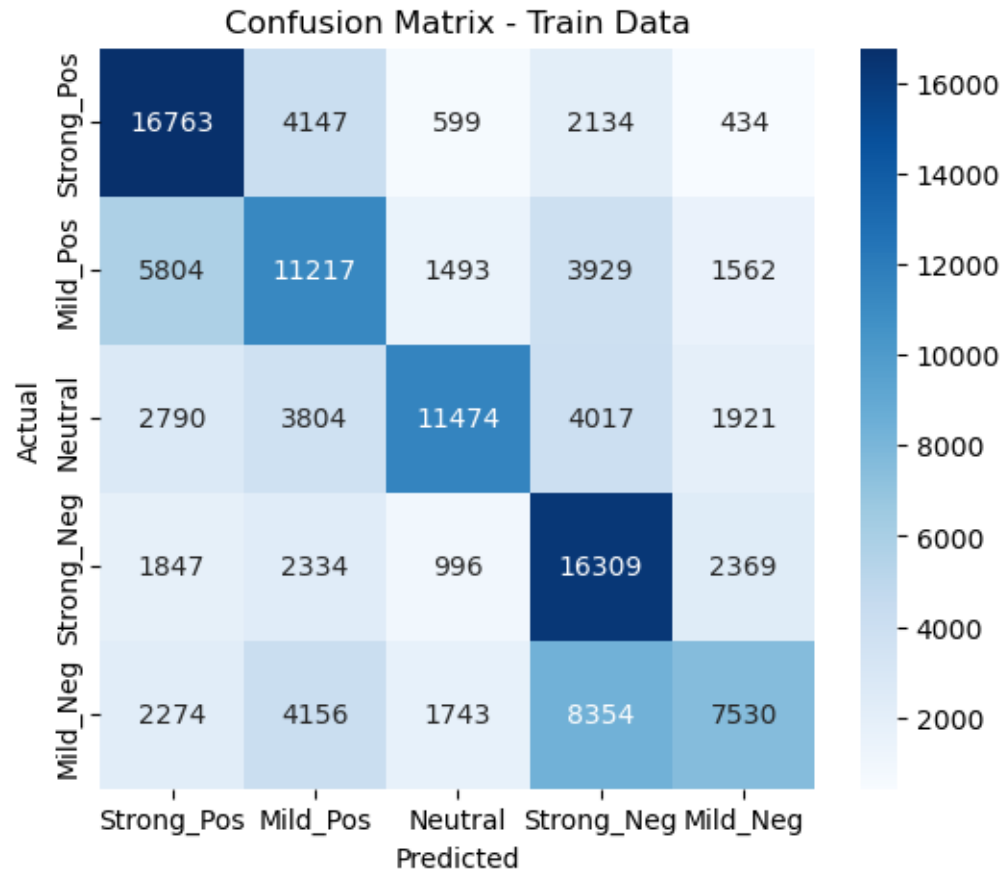
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.57      | 0.70   | 0.63     | 24077   |
| Mild_Pos     | 0.44      | 0.47   | 0.45     | 24005   |
| Neutral      | 0.70      | 0.48   | 0.57     | 24006   |
| Strong_Neg   | 0.47      | 0.68   | 0.56     | 23855   |
| Mild_Neg     | 0.55      | 0.31   | 0.40     | 24057   |
| accuracy     |           |        | 0.53     | 120000  |
| macro avg    | 0.54      | 0.53   | 0.52     | 120000  |
| weighted avg | 0.54      | 0.53   | 0.52     | 120000  |

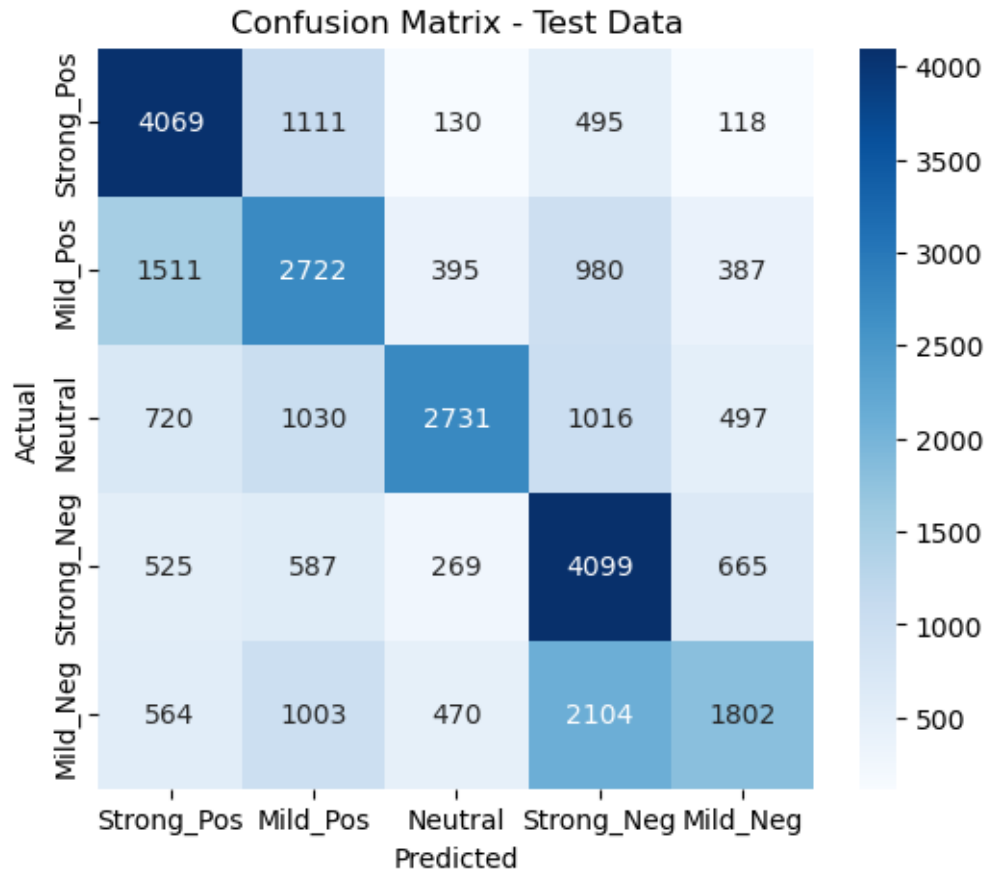
#### Classification Report - Test Data

|            | precision | recall | f1-score | support |
|------------|-----------|--------|----------|---------|
| Strong_Pos | 0.55      | 0.69   | 0.61     | 5923    |
| Mild_Pos   | 0.42      | 0.45   | 0.44     | 5995    |
| Neutral    | 0.68      | 0.46   | 0.55     | 5994    |
| Strong_Neg | 0.47      | 0.67   | 0.55     | 6145    |
| Mild_Neg   | 0.52      | 0.30   | 0.38     | 5943    |
| accuracy   |           |        | 0.51     | 30000   |
| macro avg  | 0.53      | 0.51   | 0.51     | 30000   |



weighted avg      0.53      0.51      0.51      30000





chatgpt prompt :- get top 10 features with heights weights for word2vector

```
[190]: # Get the word vectors from the Word2Vec model
word_vectors = word2vec_model.wv

# Calculate the magnitude (norm) of each word vector
word_norms = {word: np.linalg.norm(word_vectors[word]) for word in word_vectors.
               ↪index_to_key}

# Sort the words based on their norm (magnitude) in descending order and get
               ↪the top 10
top_10_words = sorted(word_norms.items(), key=lambda x: x[1], reverse=True)[:10]

# Print the top 10 words with the highest weights
print("Top 10 Features with the Highest Weights:")
for i, (word, norm) in enumerate(top_10_words, 1):
    print(f"{i}. {word} (Weight: {norm:.4f})")
```

Top 10 Features with the Highest Weights:  
 1. de (Weight: 23.8090)

2. emerg (Weight: 22.8618)
3. via (Weight: 21.2433)
4. le (Weight: 20.6590)
5. up (Weight: 20.5395)
6. down (Weight: 20.3334)
7. ha (Weight: 20.2168)
8. polic (Weight: 20.0853)
9. an (Weight: 20.0648)
10. arrest (Weight: 20.0154)

```
[193]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust
↳based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust
↳based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
↳y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
↳y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =
↳{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
```

```

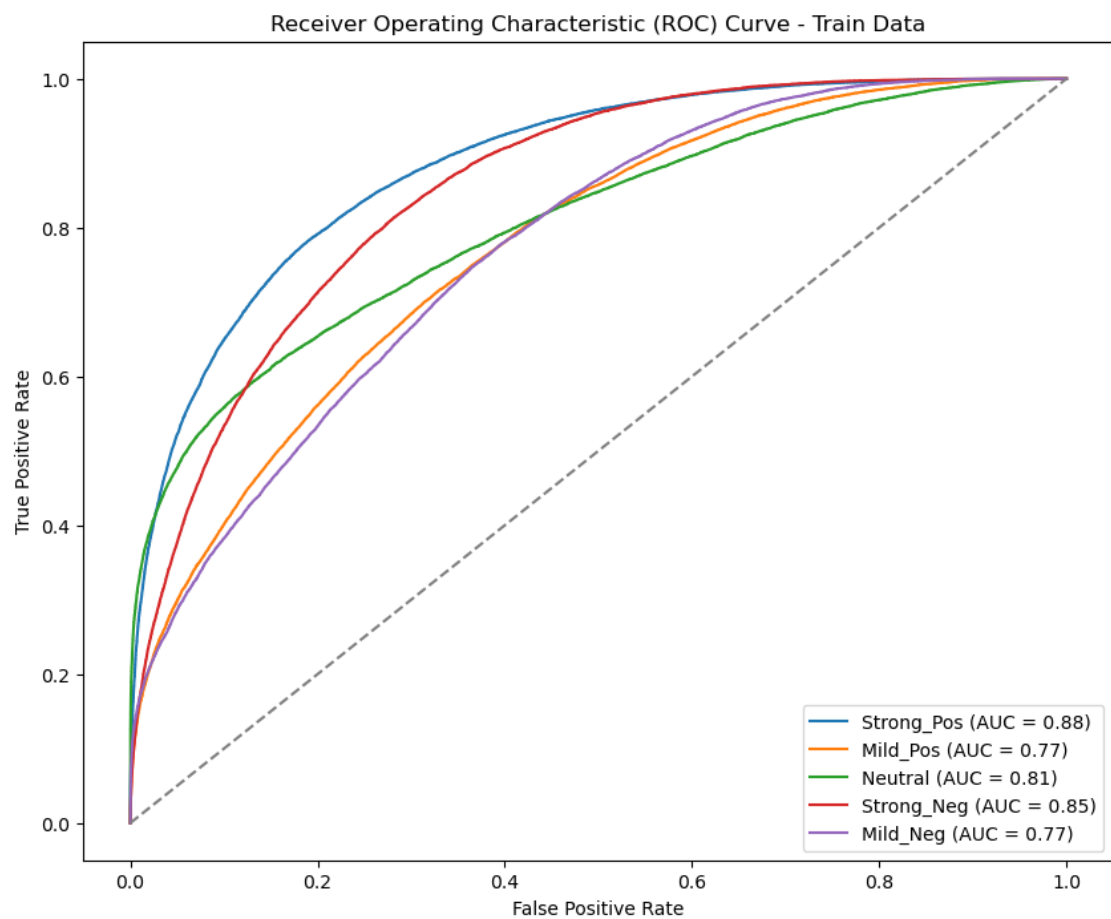
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

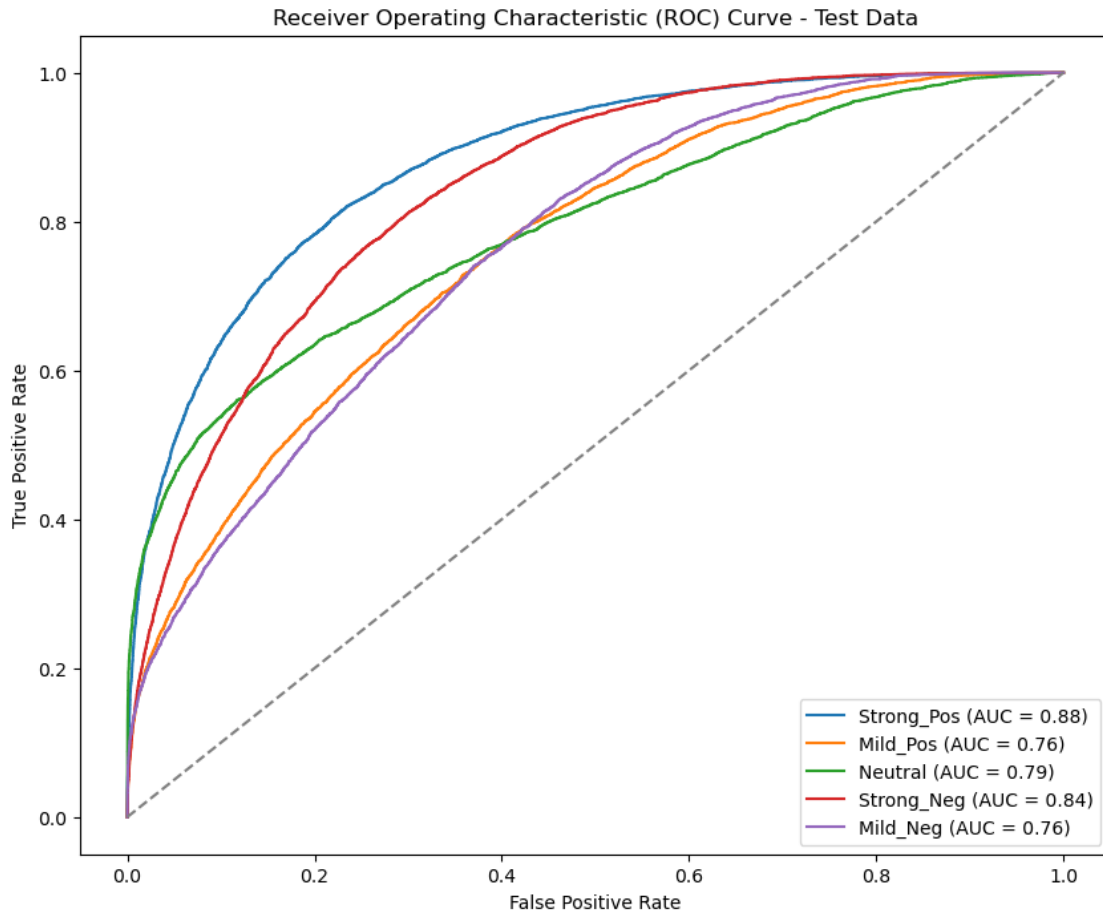
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC = {
        roc_auc_test[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```





model was unable to learn the train data completely, so let's add some more layers or hyper parameter tuning the model might improve performance of the model or i can train the model with new more mild\_pos, neutral, mild\_pos data

```
[196]: # Example usage for a single test instance (e.g., x_test[0])
        explanation = explain_single_instance_with_lime(model, x_test[0],
        ↪ num_features=10)
```

157/157                      0s 2ms/step

<IPython.core.display.HTML object>

chatgpt prompt :- add some more layers and balance the class weighs for the give code

```
[199]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, GRU, Bidirectional, Dropout
        from sklearn.utils.class_weight import compute_class_weight
        import numpy as np
        import time
```

```

# Compute class weights to balance the dataset
class_weights = compute_class_weight(class_weight='balanced', classes=np.
    ↪unique(y_train), y=y_train)
class_weights = dict(enumerate(class_weights))
print("Class Weights:", class_weights)

# Define the improved Bidirectional GRU model
model = Sequential()

# First Bidirectional GRU layer
model.add(Bidirectional(GRU(128, return_sequences=True, dropout=0.3, ↪
    ↪recurrent_dropout=0.3), input_shape=(1, x_train.shape[2])))

# Second Bidirectional GRU layer
model.add(Bidirectional(GRU(64, return_sequences=True, dropout=0.3, ↪
    ↪recurrent_dropout=0.3)))

# Third Bidirectional GRU layer
model.add(Bidirectional(GRU(32, dropout=0.3, recurrent_dropout=0.3)))

# Fully connected layers
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.3)) # Prevents overfitting
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))

# Output layer
model.add(Dense(5, activation='softmax'))

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', ↪
    ↪metrics=['accuracy'])

# Track training time
start_time = time.time()

# Train the model with class weights
history = model.fit(x_train, y_train, epochs=10, batch_size=125, ↪
    ↪validation_data=(x_test, y_test), class_weight=class_weights)

# Calculate training time
end_time = time.time()
training_time = end_time - start_time
print(f"Training time: {training_time:.2f} seconds")

# Evaluate on test data
loss, accuracy = model.evaluate(x_test, y_test)

```

```
print(f'Test Loss: {loss}')
```

```
print(f'Test Accuracy: {accuracy}')
```

Class Weights: {0: 0.9968019271503925, 1: 0.9997917100604041, 2:  
0.9997500624843789, 3: 1.0060783902745756, 4: 0.9976306272602569}

C:\Users\Gaurav\anaconda3\Lib\site-  
packages\keras\src\layers\rnn\bidirectional.py:107: UserWarning: Do not pass an  
`input\_shape`/`input\_dim` argument to a layer. When using Sequential models,  
prefer using an `Input(shape)` object as the first layer in the model instead.  
super().\_\_init\_\_(\*\*kwargs)

Epoch 1/10

960/960 25s 11ms/step -

accuracy: 0.3482 - loss: 1.4639 - val\_accuracy: 0.4446 - val\_loss: 1.3129

Epoch 2/10

960/960 19s 9ms/step -

accuracy: 0.4207 - loss: 1.3595 - val\_accuracy: 0.4549 - val\_loss: 1.2838

Epoch 3/10

960/960 9s 9ms/step -

accuracy: 0.4378 - loss: 1.3324 - val\_accuracy: 0.4670 - val\_loss: 1.2673

Epoch 4/10

960/960 10s 9ms/step -

accuracy: 0.4467 - loss: 1.3113 - val\_accuracy: 0.4778 - val\_loss: 1.2474

Epoch 5/10

960/960 9s 10ms/step -

accuracy: 0.4543 - loss: 1.2970 - val\_accuracy: 0.4822 - val\_loss: 1.2301

Epoch 6/10

960/960 10s 9ms/step -

accuracy: 0.4587 - loss: 1.2859 - val\_accuracy: 0.4901 - val\_loss: 1.2200

Epoch 7/10

960/960 10s 9ms/step -

accuracy: 0.4635 - loss: 1.2768 - val\_accuracy: 0.4963 - val\_loss: 1.2117

Epoch 8/10

960/960 10s 9ms/step -

accuracy: 0.4695 - loss: 1.2667 - val\_accuracy: 0.4957 - val\_loss: 1.2077

Epoch 9/10

960/960 10s 9ms/step -

accuracy: 0.4734 - loss: 1.2606 - val\_accuracy: 0.5002 - val\_loss: 1.2029

Epoch 10/10

960/960 10s 9ms/step -

accuracy: 0.4796 - loss: 1.2499 - val\_accuracy: 0.4970 - val\_loss: 1.1999

Training time: 124.88 seconds

938/938 7s 4ms/step -

accuracy: 0.4932 - loss: 1.2068

Test Loss: 1.1999059915542603

Test Accuracy: 0.4969666600227356



```

[201]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↳ classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
↳ round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
print(classification_report(y_train_true, y_train_pred_classes,
↳ target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
↳ target_names=class_labels))

# Plot confusion matrix for train and test data

```

```

plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix -␣
↳Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix -␣
↳Test Data", reverse_label_map)

```

```

3750/3750          12s 3ms/step
938/938            5s 5ms/step
(120000, 5)
(30000, 5)
(120000,)
(30000,)
Train Accuracy: 0.5050
Test Accuracy: 0.4970

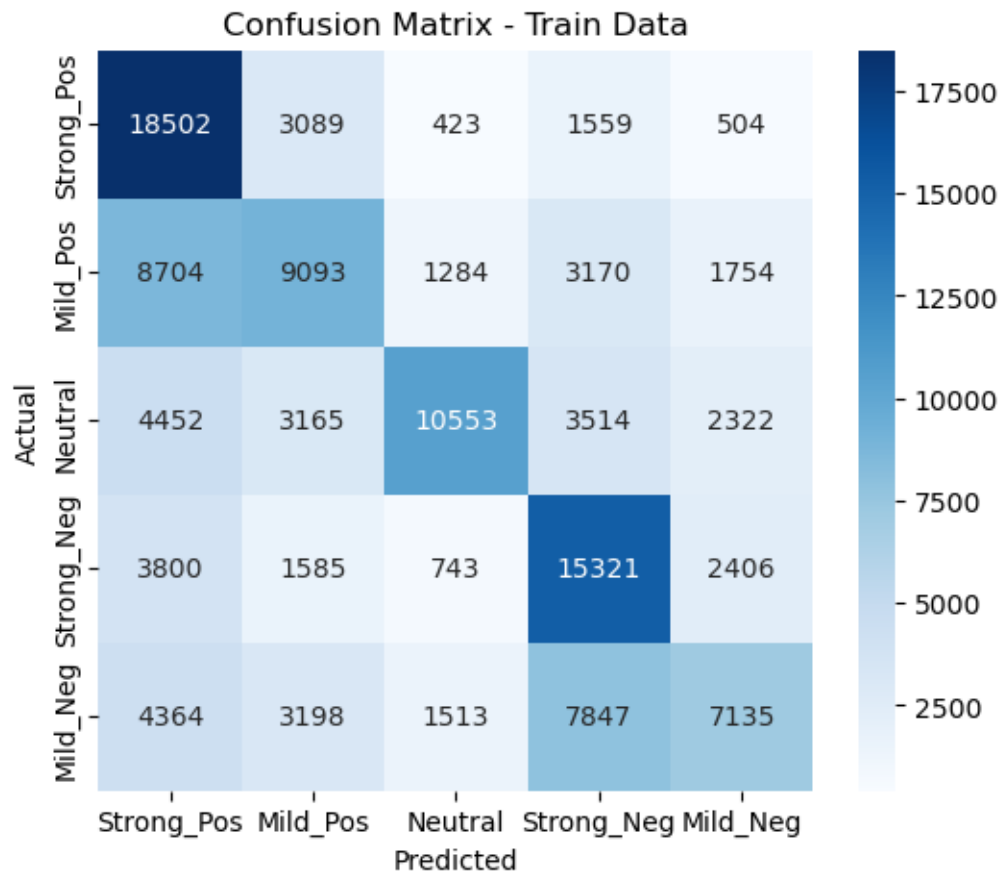
```

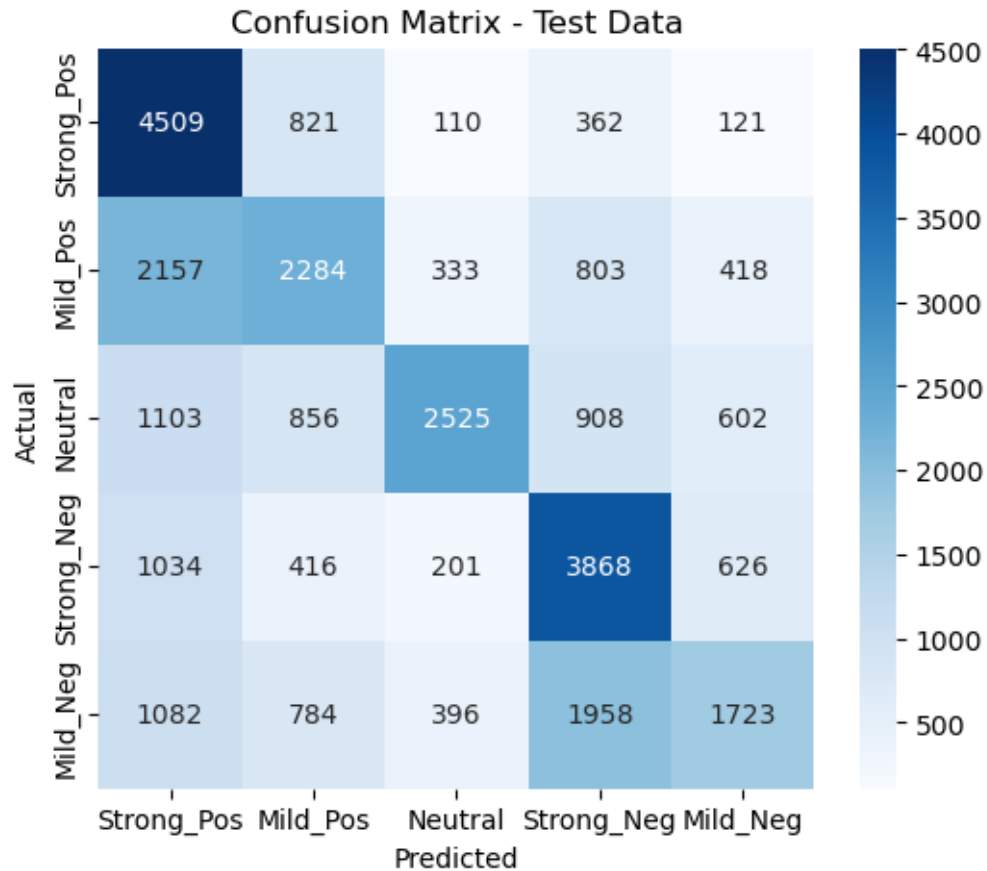
#### Classification Report - Train Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.46      | 0.77   | 0.58     | 24077   |
| Mild_Pos     | 0.45      | 0.38   | 0.41     | 24005   |
| Neutral      | 0.73      | 0.44   | 0.55     | 24006   |
| Strong_Neg   | 0.49      | 0.64   | 0.55     | 23855   |
| Mild_Neg     | 0.51      | 0.30   | 0.37     | 24057   |
| accuracy     |           |        | 0.51     | 120000  |
| macro avg    | 0.53      | 0.51   | 0.49     | 120000  |
| weighted avg | 0.53      | 0.51   | 0.49     | 120000  |

#### Classification Report - Test Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.46      | 0.76   | 0.57     | 5923    |
| Mild_Pos     | 0.44      | 0.38   | 0.41     | 5995    |
| Neutral      | 0.71      | 0.42   | 0.53     | 5994    |
| Strong_Neg   | 0.49      | 0.63   | 0.55     | 6145    |
| Mild_Neg     | 0.49      | 0.29   | 0.37     | 5943    |
| accuracy     |           |        | 0.50     | 30000   |
| macro avg    | 0.52      | 0.50   | 0.48     | 30000   |
| weighted avg | 0.52      | 0.50   | 0.49     | 30000   |





```
[205]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust_
↳based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust_
↳based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
```

```

for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
    ↪y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
    ↪y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

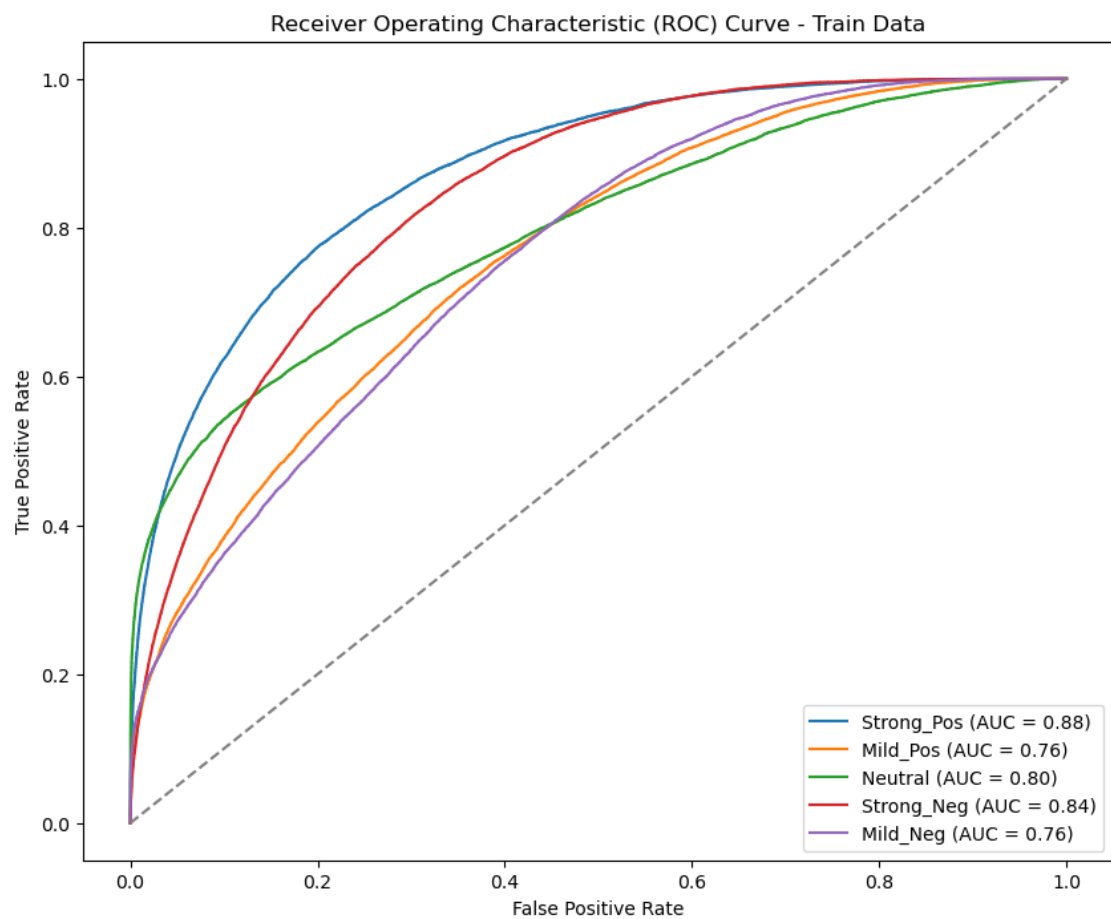
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

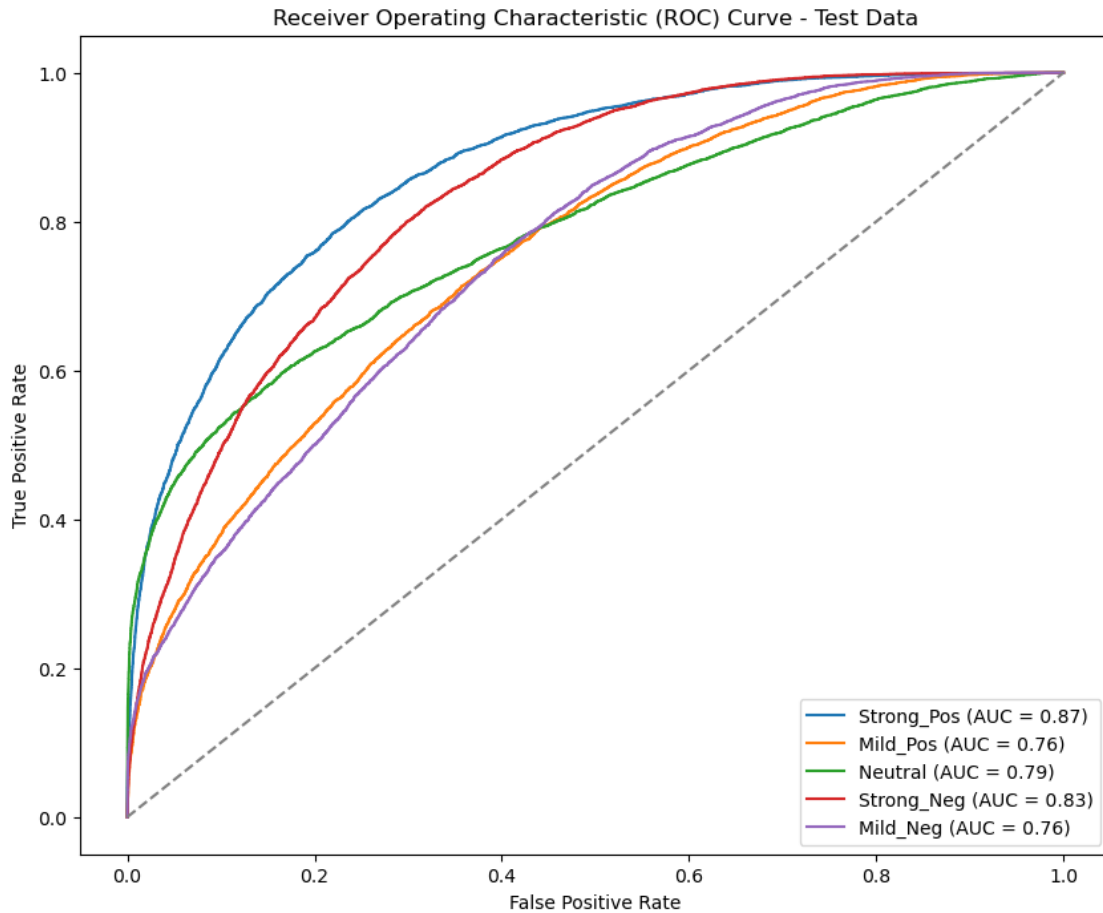
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =
    ↪{roc_auc_test[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```





Lime

```
[208]: print(x_test[0])
```

```
[[ 1.25719554 -0.41164173  0.03937192  0.23420211  0.03625457 -0.14867249
 -1.45006443 -0.25666924  0.56612408  0.20245122  0.76199453 -0.30004725
 -0.38850009  0.06613257 -0.23839876  0.15767671  0.89830472 -0.8875954
 -0.14502146 -0.00765527 -0.17310824  0.66012943  0.71927057  0.409258
 -0.39004092 -0.78437164  1.10897271  0.84250841 -0.1165294  0.50530086
  0.24146133 -1.05567074 -0.24565794  0.33809038 -0.42335865 -0.34497834
 -0.21693602 -0.15194975 -0.18484646  0.59684756  0.20117821  0.60836417
  0.08033145  1.35694003 -0.31652682 -0.0305607 -1.21567632 -0.19018959
 -0.43891595  0.42219875 -0.83723509  0.47607623 -0.93780022  0.73912474
 -0.3932573 -0.46987313  0.56138489  0.01763635  0.03753074 -0.78604676
  0.37642879  0.8034537 -0.51319243  0.48182605  0.07896908  0.354418
 -0.38544097 -0.82890988  0.30292447 -1.52768361  0.46811597 -0.23306829
 -1.23371508 -0.05723122  1.00253495 -0.34547522  0.2321527  0.56978237
  0.54074966  0.73504078 -0.07265866  0.27444491  0.34645748 -0.43317542
  0.10456906  0.91499885 -0.75317572  0.0406496 -0.03662473 -0.9650607]
```

```

-0.13182894  0.10350033 -0.26571854 -0.43130256 -0.95193259 -0.3626343
 0.52236487  0.93243518  0.29178191  0.26999248  0.71456206  0.22824424
-0.08480711 -0.08738589  0.02858498  0.81402026  0.51542559  0.5922568
-0.44467668  0.57153693 -0.64496371  0.3101186  0.16019839  0.19380308
 0.08731755 -0.19855758 -1.09488241  0.97574055  0.66457069 -0.43051034
 0.39753752 -0.55977816  0.25494936 -0.12943631  0.07021938 -0.02185813
 1.3203159  -0.23486705 -0.49688144 -0.21579228 -0.32213317 -0.90648305
 0.59127907 -0.07144193  0.87026942 -0.11791942 -0.4282763  0.0854995
 0.21665386  0.32113303  0.02681166 -0.28346243 -0.90709991  0.14012232
 0.89535011 -0.06876119 -0.10184863 -0.3755895  -0.32003578 -0.16370472
-0.68488514  0.75579827 -0.06589071 -0.89349902 -0.03985184  0.31147948
-0.39270057  0.27620398  0.98481834  0.16054667 -0.33326953  0.15297233
-0.1575307   0.6019106  -0.59308638 -0.03252503  0.15595327  1.00950026
 0.1870098   0.5841145  -0.16105072 -0.44218352  0.47439712  0.92699034
-1.06236315 -0.38556874  0.71922455 -0.83765952  0.56063879  0.03173749
 0.25542969  0.18823607  0.58141056  0.09892227  0.38893772 -0.37725187
-0.05513628 -1.0471114  0.46515306  0.14130786 -0.40385513 -0.2505211
 0.26540788  0.24994981 -0.14008164 -0.01760084 -0.04671536 -0.18285797
-0.05098898  0.31103884]]

```

```

[210]: # Example usage for a single test instance (e.g., x_test[0])
        explanation = explain_single_instance_with_lime(model, x_test[0],
        ↪num_features=10)

```

157/157                      0s 3ms/step

<IPython.core.display.HTML object>

after adding anothe layer the performace was not increasing, it is same as model 1

chatgpt prompt :- add the some more layeras , learning rate scheduler and early stopping for the above code to imporve the accuracy

```

[214]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, GRU, Bidirectional, Dropout
        from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
        from sklearn.utils.class_weight import compute_class_weight
        from tensorflow.keras.optimizers import Adam
        import numpy as np
        import time

        # Compute class weights to balance the dataset
        class_weights = compute_class_weight(class_weight='balanced', classes=np.
        ↪unique(y_train), y=y_train)
        class_weights = dict(enumerate(class_weights))

        # Define the Bidirectional GRU model
        model = Sequential()

```



```

# First Bidirectional GRU layer with more units and increased dropout
model.add(Bidirectional(GRU(256, return_sequences=True, dropout=0.4,
    ↳recurrent_dropout=0.4), input_shape=(x_train.shape[1], x_train.shape[2])))

# Second Bidirectional GRU layer with more units and increased dropout
model.add(Bidirectional(GRU(128, return_sequences=True, dropout=0.4,
    ↳recurrent_dropout=0.4)))

# Third Bidirectional GRU layer with more units and increased dropout
model.add(Bidirectional(GRU(64, dropout=0.4, recurrent_dropout=0.4)))

# Fully connected layers with increased dropout
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5)) # Increased dropout
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.5)) # Increased dropout

# Output layer
model.add(Dense(5, activation='softmax'))

# Compile the model with a lower learning rate
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
    ↳metrics=['accuracy'])

# Implement EarlyStopping
early_stopping = EarlyStopping(monitor='val_loss', patience=5,
    ↳restore_best_weights=True)

# Implement Learning Rate Scheduler
lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3,
    ↳min_lr=1e-6)

# Track training time
start_time = time.time()

# Train the model with class weights
history = model.fit(x_train, y_train, epochs=10, batch_size=125,
    ↳validation_data=(x_test, y_test), class_weight=class_weights,
    ↳callbacks=[early_stopping, lr_scheduler])

# Calculate training time
end_time = time.time()
training_time = end_time - start_time
print(f"Training time: {training_time:.2f} seconds")

# Evaluate on test data

```

```
loss, accuracy = model.evaluate(x_test, y_test)
print(f'Test Loss: {loss}')
print(f'Test Accuracy: {accuracy}')
```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\layers\rnn\bidirectional.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.  
super().\_\_init\_\_(\*\*kwargs)

Epoch 1/10

960/960 35s 22ms/step -  
accuracy: 0.3064 - loss: 1.5133 - val\_accuracy: 0.4262 - val\_loss: 1.3404 -  
learning\_rate: 0.0010

Epoch 2/10

960/960 20s 21ms/step -  
accuracy: 0.4008 - loss: 1.4047 - val\_accuracy: 0.4472 - val\_loss: 1.3077 -  
learning\_rate: 0.0010

Epoch 3/10

960/960 8s 8ms/step -  
accuracy: 0.4148 - loss: 1.3771 - val\_accuracy: 0.4612 - val\_loss: 1.2854 -  
learning\_rate: 0.0010

Epoch 4/10

960/960 7s 8ms/step -  
accuracy: 0.4260 - loss: 1.3578 - val\_accuracy: 0.4645 - val\_loss: 1.2712 -  
learning\_rate: 0.0010

Epoch 5/10

960/960 7s 8ms/step -  
accuracy: 0.4306 - loss: 1.3521 - val\_accuracy: 0.4711 - val\_loss: 1.2565 -  
learning\_rate: 0.0010

Epoch 6/10

960/960 7s 7ms/step -  
accuracy: 0.4377 - loss: 1.3379 - val\_accuracy: 0.4705 - val\_loss: 1.2685 -  
learning\_rate: 0.0010

Epoch 7/10

960/960 7s 7ms/step -  
accuracy: 0.4436 - loss: 1.3291 - val\_accuracy: 0.4763 - val\_loss: 1.2442 -  
learning\_rate: 0.0010

Epoch 8/10

960/960 7s 7ms/step -  
accuracy: 0.4478 - loss: 1.3213 - val\_accuracy: 0.4847 - val\_loss: 1.2340 -  
learning\_rate: 0.0010

Epoch 9/10

960/960 7s 8ms/step -  
accuracy: 0.4488 - loss: 1.3160 - val\_accuracy: 0.4868 - val\_loss: 1.2238 -  
learning\_rate: 0.0010

Epoch 10/10

960/960 7s 7ms/step -

accuracy: 0.4525 - loss: 1.3065 - val\_accuracy: 0.4897 - val\_loss: 1.2279 -  
learning\_rate: 0.0010  
Training time: 112.74 seconds  
938/938 2s 1ms/step -  
accuracy: 0.4866 - loss: 1.2289  
Test Loss: 1.22378408908844  
Test Accuracy: 0.4868333339691162

```
[218]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↳classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
↳round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
```

```

print(classification_report(y_train_true, y_train_pred_classes,
    ↳target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
    ↳target_names=class_labels))

# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix -
    ↳Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix -
    ↳Test Data", reverse_label_map)

```

```

3750/3750          4s 1ms/step
938/938           1s 1ms/step
(120000, 5)
(30000, 5)
(120000,)
(30000,)
Train Accuracy: 0.4899
Test Accuracy: 0.4868

```

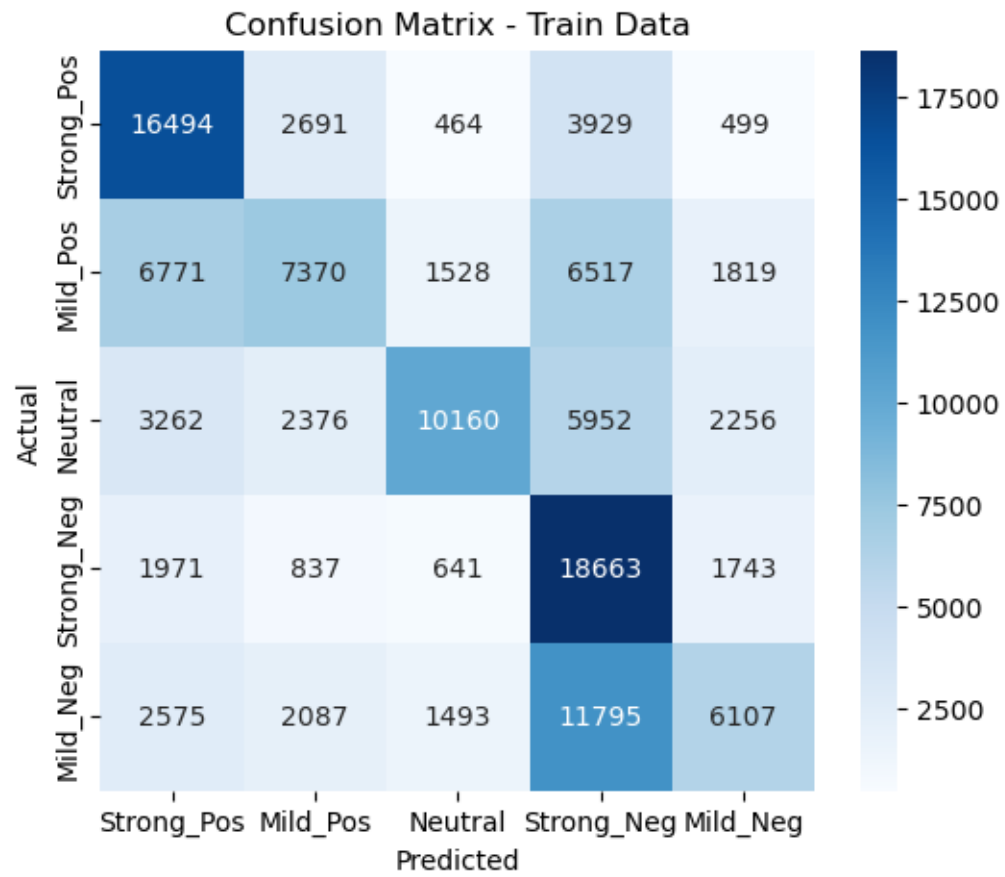
#### Classification Report - Train Data

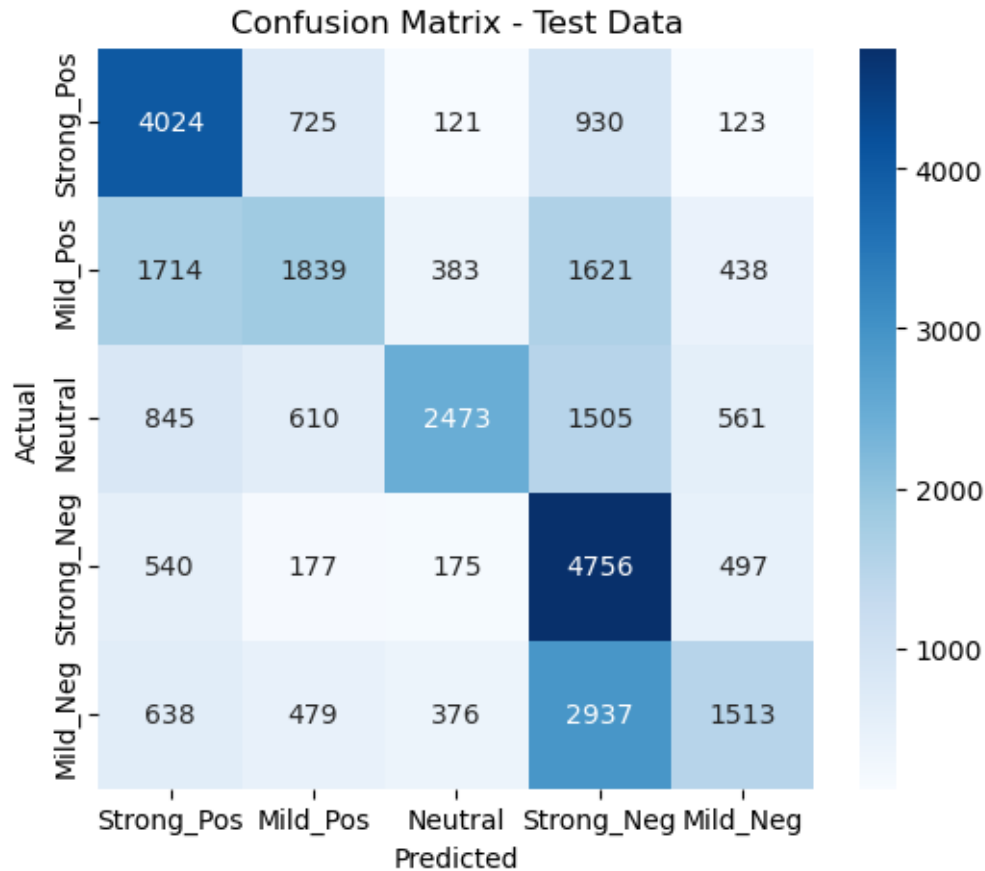
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.53      | 0.69   | 0.60     | 24077   |
| Mild_Pos     | 0.48      | 0.31   | 0.37     | 24005   |
| Neutral      | 0.71      | 0.42   | 0.53     | 24006   |
| Strong_Neg   | 0.40      | 0.78   | 0.53     | 23855   |
| Mild_Neg     | 0.49      | 0.25   | 0.33     | 24057   |
| accuracy     |           |        | 0.49     | 120000  |
| macro avg    | 0.52      | 0.49   | 0.47     | 120000  |
| weighted avg | 0.52      | 0.49   | 0.47     | 120000  |

#### Classification Report - Test Data

|            | precision | recall | f1-score | support |
|------------|-----------|--------|----------|---------|
| Strong_Pos | 0.52      | 0.68   | 0.59     | 5923    |
| Mild_Pos   | 0.48      | 0.31   | 0.37     | 5995    |
| Neutral    | 0.70      | 0.41   | 0.52     | 5994    |
| Strong_Neg | 0.40      | 0.77   | 0.53     | 6145    |
| Mild_Neg   | 0.48      | 0.25   | 0.33     | 5943    |
| accuracy   |           |        | 0.49     | 30000   |
| macro avg  | 0.52      | 0.49   | 0.47     | 30000   |

weighted avg      0.52      0.49      0.47      30000





```
[220]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust_
↳based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust_
↳based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
```

```

for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
    ↪y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
    ↪y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

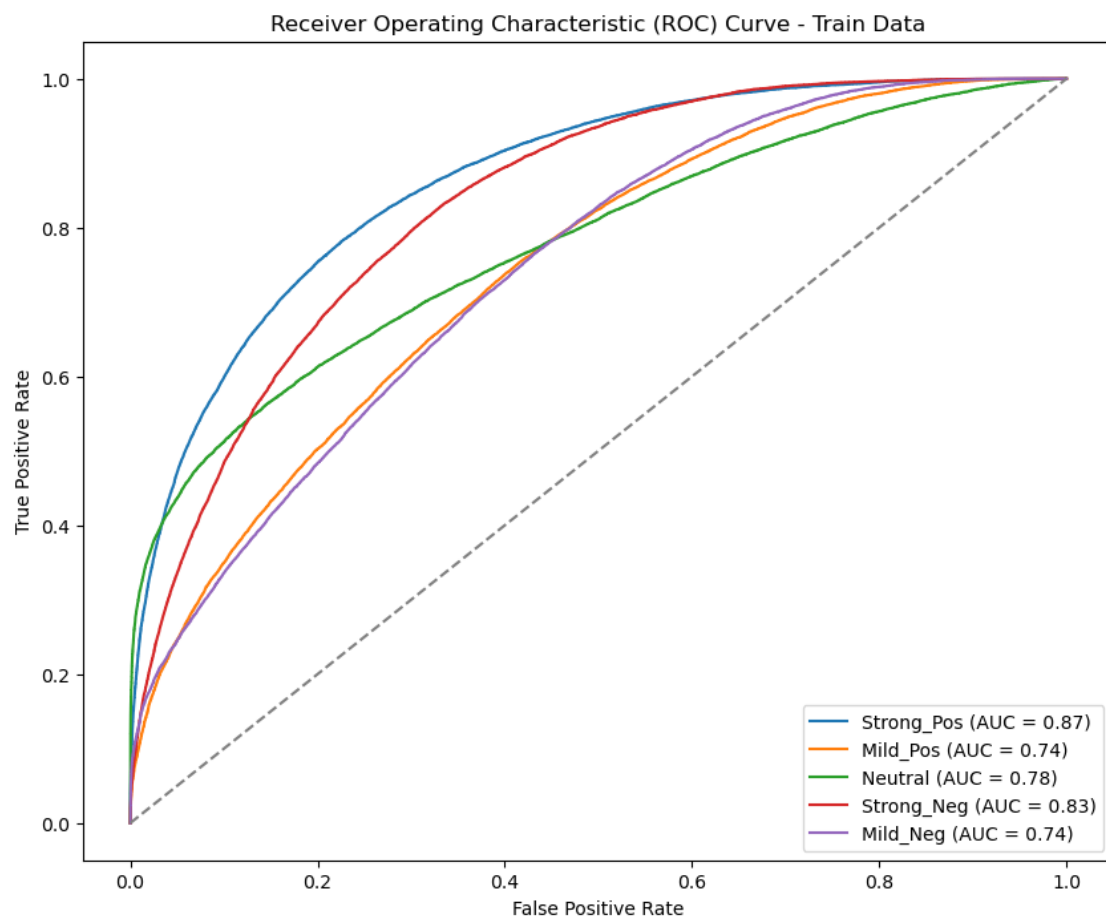
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =
    ↪{roc_auc_test[i]:.2f})')

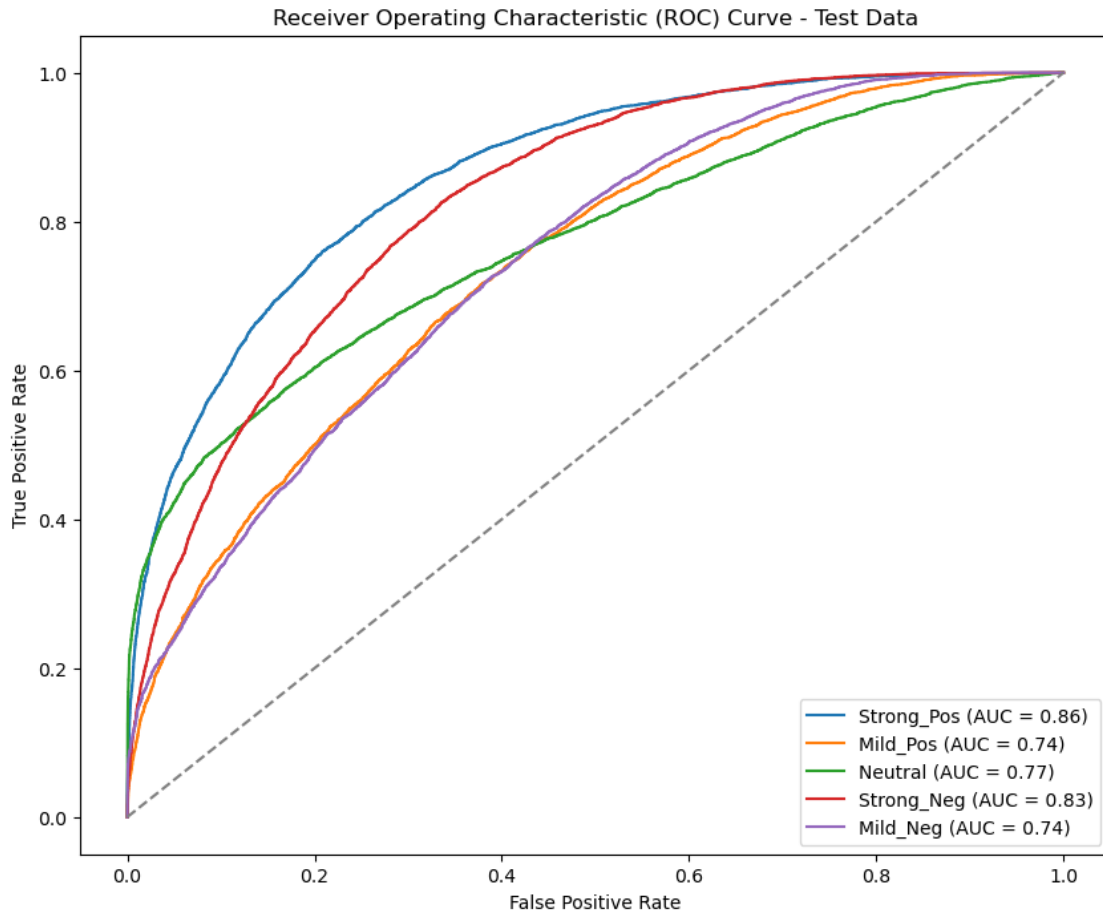
# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```







```
[222]: # Example usage for a single test instance (e.g., x_test[0])
explanation = explain_single_instance_with_lime(model, x_test[0],
↪ num_features=10)
```

157/157                      0s 2ms/step

<IPython.core.display.HTML object>

To improve accuracy further, you can try optimizing hyperparameters, increasing model complexity, exploring advanced architectures, use some pretrained models

## 4 Deep Unidirectional LSTM Model using an embedding layer to learn embeddings

```
[225]: df
```

```
[225]:
```

|   | Tweet                                            | Sentiment |
|---|--------------------------------------------------|-----------|
| 0 | @_angelica_toy Happy Anniversary!!!...The Day... | Mild_Pos  |

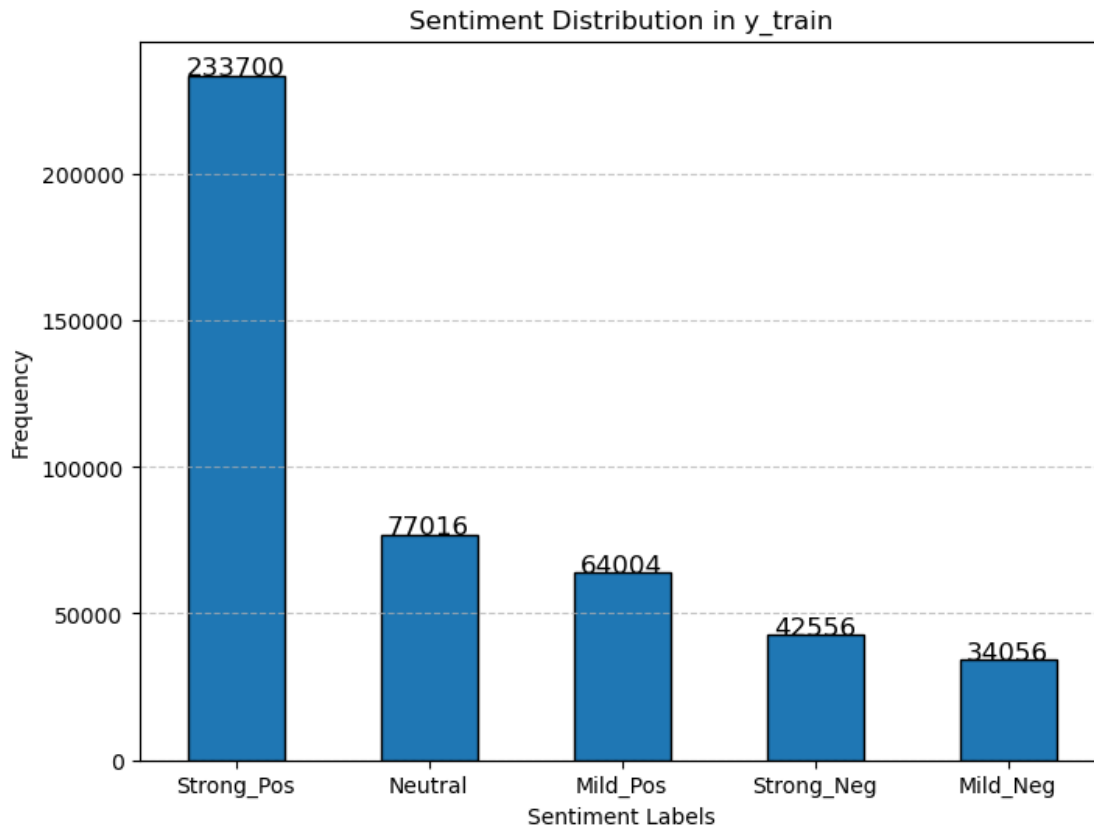
```

1      @McfarlaneGlenda Happy Anniversary!!!!...The D... Mild_Pos
2      @thevivafrei @JustinTrudeau Happy Anniversary!... Mild_Pos
3      @NChartierET Happy Anniversary!!!!...The Day t... Mild_Pos
4      @tabithapeters05 Happy Anniversary!!!!...The D... Mild_Pos
...
451327 Gaza; Peace n' Freedom - Viva Palestina convoy... Strong_Pos
451328 Face of Defense: Soldier Finds Freedom in U.S... Strong_Pos
451329 Face of Defense: Soldier Finds Freedom in U.S... Strong_Pos
451330 Gaza; Peace n' Freedom - "Israel stops aid con... Strong_Pos
451331      @convoy 83 yes! get on freedom server! Strong_Pos

```

[451332 rows x 2 columns]

```
[228]: bar_plot(df['Sentiment'])
```



```

[230]: # Count the number of samples per class
class_counts = df['Sentiment'].value_counts()
print("\nBefore undersampling:\n", class_counts)

# Define the number of samples to take for each class

```

```

sample_sizes = {
    'Strong_Pos': 34000,
    'Mild_Pos': 34000,
    'Neutral': 34000, # Adjust the sample size for Negative
    'Strong_Neg': 34000, # Adjust the sample size for Neutral
    'Mild_Neg': 34000 # Adjust the sample size for Positive
}

# Perform undersampling without replacement
df_lstm = pd.concat([
    df[df['Sentiment'] == label].sample(n=min(sample_sizes[label],
    ↪len(df[df['Sentiment'] == label])),
                                random_state=42,
                                replace=False)
    for label in sample_sizes.keys()
])

# Print class distribution after undersampling
print("\nAfter undersampling:\n", df_lstm['Sentiment'].value_counts())

```

Before undersampling:

| Sentiment  |        |
|------------|--------|
| Strong_Pos | 233700 |
| Neutral    | 77016  |
| Mild_Pos   | 64004  |
| Strong_Neg | 42556  |
| Mild_Neg   | 34056  |

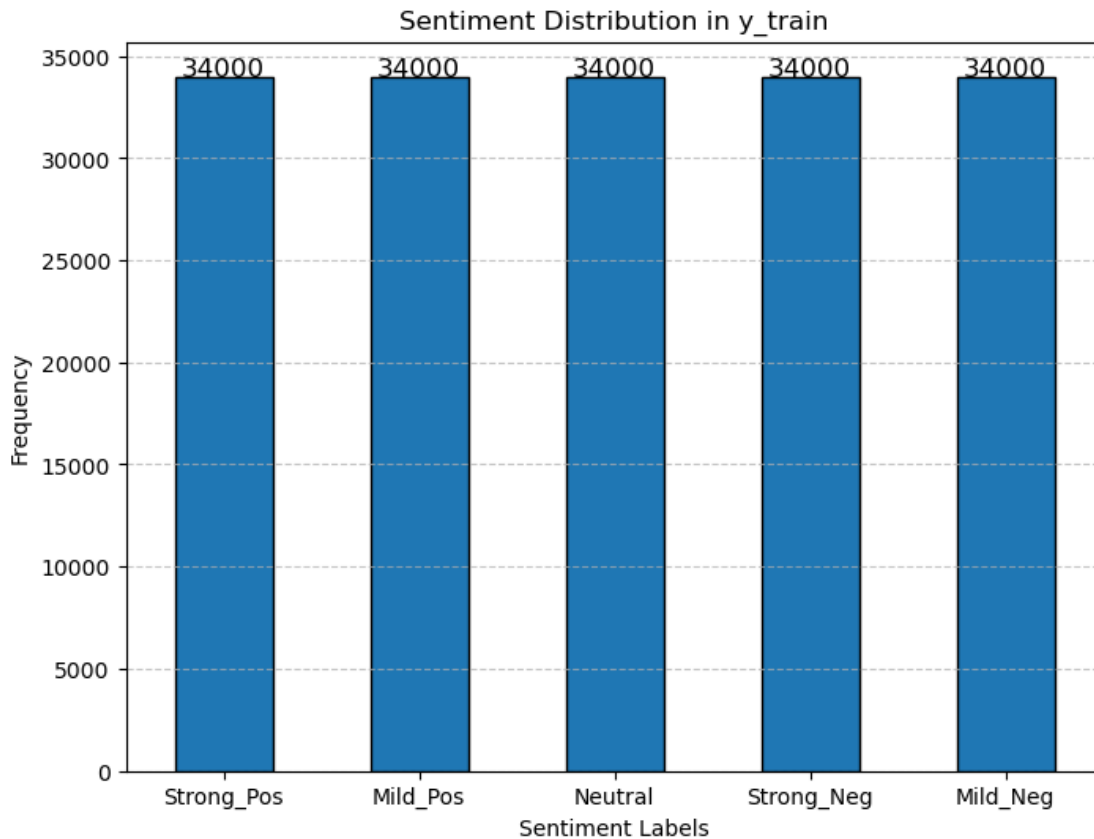
Name: count, dtype: int64

After undersampling:

| Sentiment  |       |
|------------|-------|
| Strong_Pos | 34000 |
| Mild_Pos   | 34000 |
| Neutral    | 34000 |
| Strong_Neg | 34000 |
| Mild_Neg   | 34000 |

Name: count, dtype: int64

```
[232]: bar_plot(df_lstm['Sentiment'])
```



```
[234]: # Apply preprocessing
df_lstm["Cleaned_Tweet"] = df_lstm["Tweet"].apply(preprocess_stopwords_stemming)

df_lstm["Sentiment"] = df_lstm["Sentiment"].map(label_map)

# Define input (text) and target (label)
x = df_lstm["Cleaned_Tweet"].values
y = df_lstm["Sentiment"].values

print(x.shape)
print(y.shape)

print(x)
print(y)
```

```
(170000,)
(170000,)
[list(['viva', 'frei', 'live', 'now', 'live', 'in', 'ottawa', 'freedom',
'convoy', '2022', 'http', 'via', 'youtub'])
  list(['laurajede', 'thi', 'is', 'exactli', 'the', 'convoy', 'type', 'in',
'canada'])]
```

```
list(['hollyanndoan', 'q', 'to', 'freeland', 'whi', 'did', 'you', 'chuckl',
'and', 'smile', 'when', 'you', 'announc', 'you', 'were', 'freez', 'the', 'bank',
'account', 'of', 'those', 'who', 'donat', 'to', 'the', 'freedom', 'convoy'])
...
list(['forget', 'isi', 'thi', 'is', 'the', 'true', 'face', 'of', 'terror',
'right', 'here', 'honkhonk', 'freedomconvoy', 'http'])
list(['protest', 'caus', 'traffic', 'jam', 'near', 'border', 'in', 'michigan',
'http', 'foxnew'])
list(['hey', 'not', 'everi', 'protestor', 'is', 'a', 'racist', 'homophob',
'misogynist', 'nazi', 'clown', 'you', 'know', 'do', 'paint', 'the', 'entir',
'crowd', 'becaus', 'to', 'the', '20', 'who', 'are', 'like', 'that',
'freedomconvoy'])]
[0 0 0 ... 4 4 4]
```

```
[235]: # Tokenization and padding
max_words = 50000 # Vocabulary size
max_len = 50 # Maximum sequence length (increased to 5000)

tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(x)
sequences = tokenizer.texts_to_sequences(x)
x_padded = pad_sequences(sequences, maxlen=max_len, padding="post",
↳truncating="post")
```

```
[238]: # Split data into training and test sets
x_train, x_test, y_train, y_test = train_test_split(x_padded, y, test_size=0.2,
↳random_state=42)
```

```
[240]: print(x_train)
```

```
[[3309 7040 2755 ...    0    0    0]
 [  57   20   26 ...    0    0    0]
 [6315    2  130 ...    0    0    0]
 ...
 [ 121   43    9 ...    0    0    0]
 [   2  209    7 ...    0    0    0]
 [  20   58  141 ...   14   31  128]]
```

```
[242]: import numpy as np
from lime.lime_text import LimeTextExplainer

def lime_explanation_lstm(model, sample_text_sequence, reverse_label_map,
↳max_len):
    """
    Generates a LIME explanation for a single RNN input.

    Parameters:
```

```

- model: Trained RNN model with an embedding layer
- sample_text_sequence: A single processed test instance (already tokenized,
↳ & padded)
- reverse_label_map: Dictionary mapping class indices to labels
- max_len: Maximum sequence length (same as used in Embedding layer)
"""

# Define class names from reverse_label_map
class_names = [reverse_label_map[i] for i in range(len(reverse_label_map))]

# Initialize LIME explainer
explainer = LimeTextExplainer(class_names=class_names)

# Define the prediction function for LIME
def predict_fn(text_instances):
    """
    Converts LIME text instances (words) into padded sequences & predicts,
↳ probabilities.
    """
    # Convert words into numerical sequences
    sequences = [np.array(list(map(int, text.split())))] for text in
↳ text_instances]

    # Pad sequences to max_len
    sequences_padded = np.array([np.pad(seq, (0, max_len - len(seq)),
↳ 'constant') for seq in sequences])

    # Get model predictions
    return model.predict(sequences_padded)

# Generate LIME explanation
exp = explainer.explain_instance(
    " ".join(map(str, sample_text_sequence)), # Convert array to
↳ space-separated string
    predict_fn,
    num_features=10,
    top_labels=5
)

# Show explanation
exp.show_in_notebook()

return exp

```

```
[244]: print(y_train)
```

```
[2 2 2 ... 3 4 3]
```

chatgpt prompt :- how to implement Deep Unidirectional LSTM Model using an embedding layer to learn embeddings

```
[247]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

# Define Deep Unidirectional LSTM Model
model = Sequential([
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len), # Learn embeddings
    LSTM(128, return_sequences=True), # First LSTM layer
    LSTM(128), # Second LSTM layer (deep LSTM)
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(5, activation='softmax') # Output layer (5 classes)
])

# Compile the Model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# Model Summary
model.summary()
```

```
C:\Users\Gaurav\anaconda3\Lib\site-
packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential_7"
```

| Layer (type)            | Output Shape | Param #     |
|-------------------------|--------------|-------------|
| embedding_2 (Embedding) | ?            | 0 (unbuilt) |
| lstm (LSTM)             | ?            | 0 (unbuilt) |
| lstm_1 (LSTM)           | ?            | 0 (unbuilt) |
| dropout_11 (Dropout)    | ?            | 0           |
| dense_13 (Dense)        | ?            | 0 (unbuilt) |
| dropout_12 (Dropout)    | ?            | 0           |

dense\_14 (Dense) ? 0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[249]: start_time = time.time()
model.fit(x_train, y_train, epochs=10, batch_size=125, validation_data=(x_test,
    ↪ y_test))
# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")
```

```
Epoch 1/10
1088/1088          74s 67ms/step -
accuracy: 0.4405 - loss: 1.2931 - val_accuracy: 0.6440 - val_loss: 0.9250
Epoch 2/10
1088/1088          71s 65ms/step -
accuracy: 0.6821 - loss: 0.8366 - val_accuracy: 0.6919 - val_loss: 0.8018
Epoch 3/10
1088/1088          68s 63ms/step -
accuracy: 0.7377 - loss: 0.7031 - val_accuracy: 0.6999 - val_loss: 0.8086
Epoch 4/10
1088/1088          66s 60ms/step -
accuracy: 0.7732 - loss: 0.6190 - val_accuracy: 0.7044 - val_loss: 0.7826
Epoch 5/10
1088/1088          65s 60ms/step -
accuracy: 0.8035 - loss: 0.5480 - val_accuracy: 0.6980 - val_loss: 0.8239
Epoch 6/10
1088/1088          65s 59ms/step -
accuracy: 0.8332 - loss: 0.4756 - val_accuracy: 0.7034 - val_loss: 0.8666
Epoch 7/10
1088/1088          65s 60ms/step -
accuracy: 0.8528 - loss: 0.4245 - val_accuracy: 0.6997 - val_loss: 0.9430
Epoch 8/10
1088/1088          65s 60ms/step -
accuracy: 0.8695 - loss: 0.3797 - val_accuracy: 0.6972 - val_loss: 0.9453
Epoch 9/10
1088/1088          65s 60ms/step -
accuracy: 0.8893 - loss: 0.3310 - val_accuracy: 0.6970 - val_loss: 1.0607
Epoch 10/10
1088/1088          65s 60ms/step -
```



accuracy: 0.8986 - loss: 0.2983 - val\_accuracy: 0.6996 - val\_loss: 1.1839  
Training time: 667.96 seconds

```
[251]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↳classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
↳round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
print(classification_report(y_train_true, y_train_pred_classes,
↳target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
↳target_names=class_labels))
```

```
# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix - ␣
↳Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix - ␣
↳Test Data", reverse_label_map)
```

4250/4250                      29s 7ms/step

1063/1063                     8s 7ms/step

(136000, 5)

(34000, 5)

(136000,)

(34000,)

Train Accuracy: 0.9206

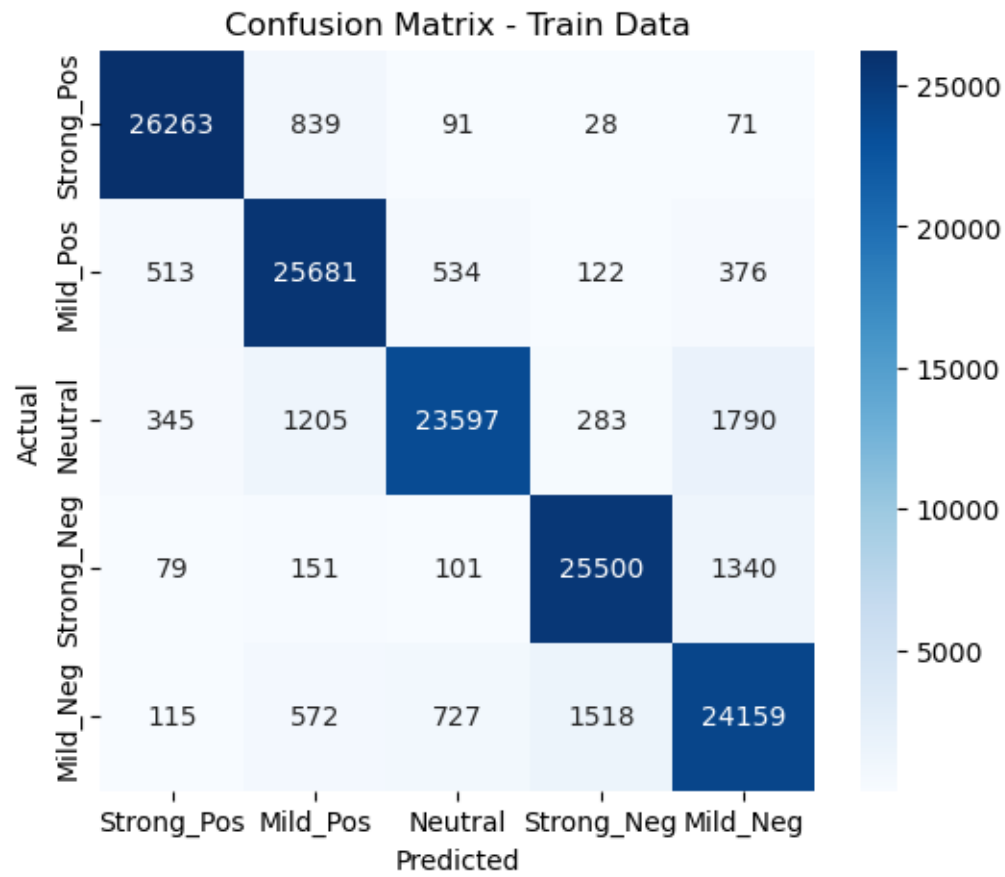
Test Accuracy: 0.6996

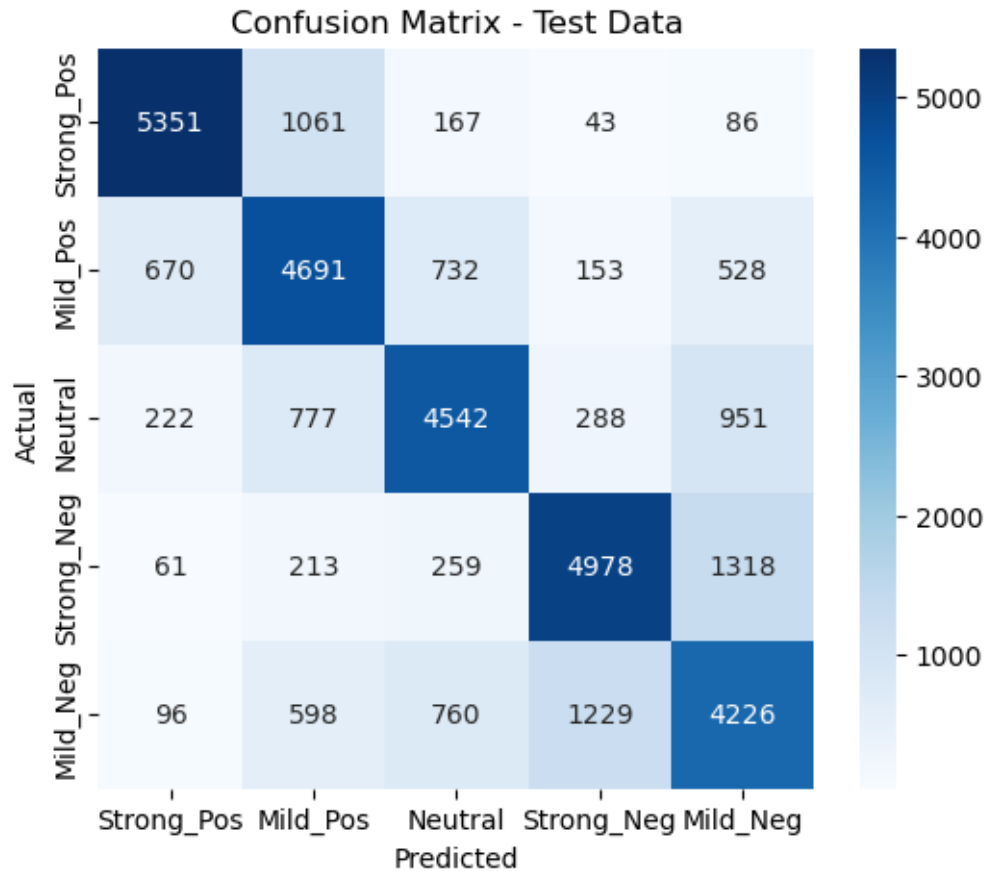
#### Classification Report - Train Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.96      | 0.96   | 0.96     | 27292   |
| Mild_Pos     | 0.90      | 0.94   | 0.92     | 27226   |
| Neutral      | 0.94      | 0.87   | 0.90     | 27220   |
| Strong_Neg   | 0.93      | 0.94   | 0.93     | 27171   |
| Mild_Neg     | 0.87      | 0.89   | 0.88     | 27091   |
| accuracy     |           |        | 0.92     | 136000  |
| macro avg    | 0.92      | 0.92   | 0.92     | 136000  |
| weighted avg | 0.92      | 0.92   | 0.92     | 136000  |

#### Classification Report - Test Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.84      | 0.80   | 0.82     | 6708    |
| Mild_Pos     | 0.64      | 0.69   | 0.66     | 6774    |
| Neutral      | 0.70      | 0.67   | 0.69     | 6780    |
| Strong_Neg   | 0.74      | 0.73   | 0.74     | 6829    |
| Mild_Neg     | 0.59      | 0.61   | 0.60     | 6909    |
| accuracy     |           |        | 0.70     | 34000   |
| macro avg    | 0.70      | 0.70   | 0.70     | 34000   |
| weighted avg | 0.70      | 0.70   | 0.70     | 34000   |





```
[253]: import numpy as np

# Get the weights from the Embedding layer
embedding_weights = model.layers[0].get_weights()[0] # Embedding layer is the
↳ first layer (index 0)

# Get the word index mapping from the tokenizer
word_index = tokenizer.word_index

# Calculate the magnitude (norm) of each word vector in the embedding layer
norms = np.linalg.norm(embedding_weights, axis=1)

# Get the indices of the top 10 largest norms
top_10_indices = np.argsort(norms)[::-1][:10]

# Get the corresponding words from the word index
top_10_words = [word for word, index in word_index.items() if index in
↳ top_10_indices]
```

```
# Print the top 10 words with the highest weights
print("Top 10 Features with the Highest Weights:")
for i, word in enumerate(top_10_words, 1):
    print(f"{i}. {word} (Weight: {norms[word_index[word] - 1]:.4f})")
```

Top 10 Features with the Highest Weights:

```
1. kill (Weight: 1.0762)
2. murder (Weight: 1.2360)
3. rape (Weight: 1.3482)
4. cancer (Weight: 0.7672)
5. disast (Weight: 1.0264)
6. perfectli (Weight: 1.0543)
7. sockboymustgo (Weight: 1.1573)
8. brilliant (Weight: 1.2566)
9. raci (Weight: 1.2647)
10. terrori (Weight: 1.6988)
```

```
[255]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪ based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪ based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
    ↪ y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
    ↪ y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
```

```

for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

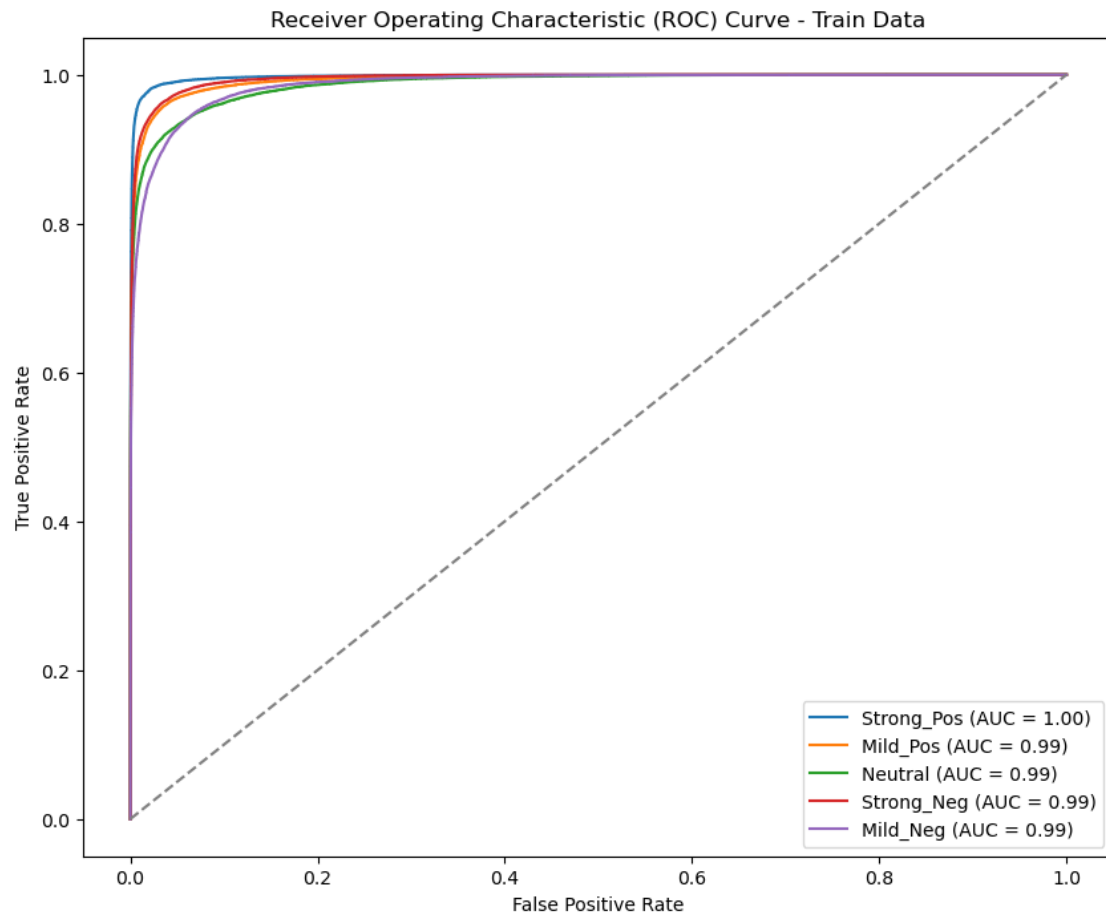
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

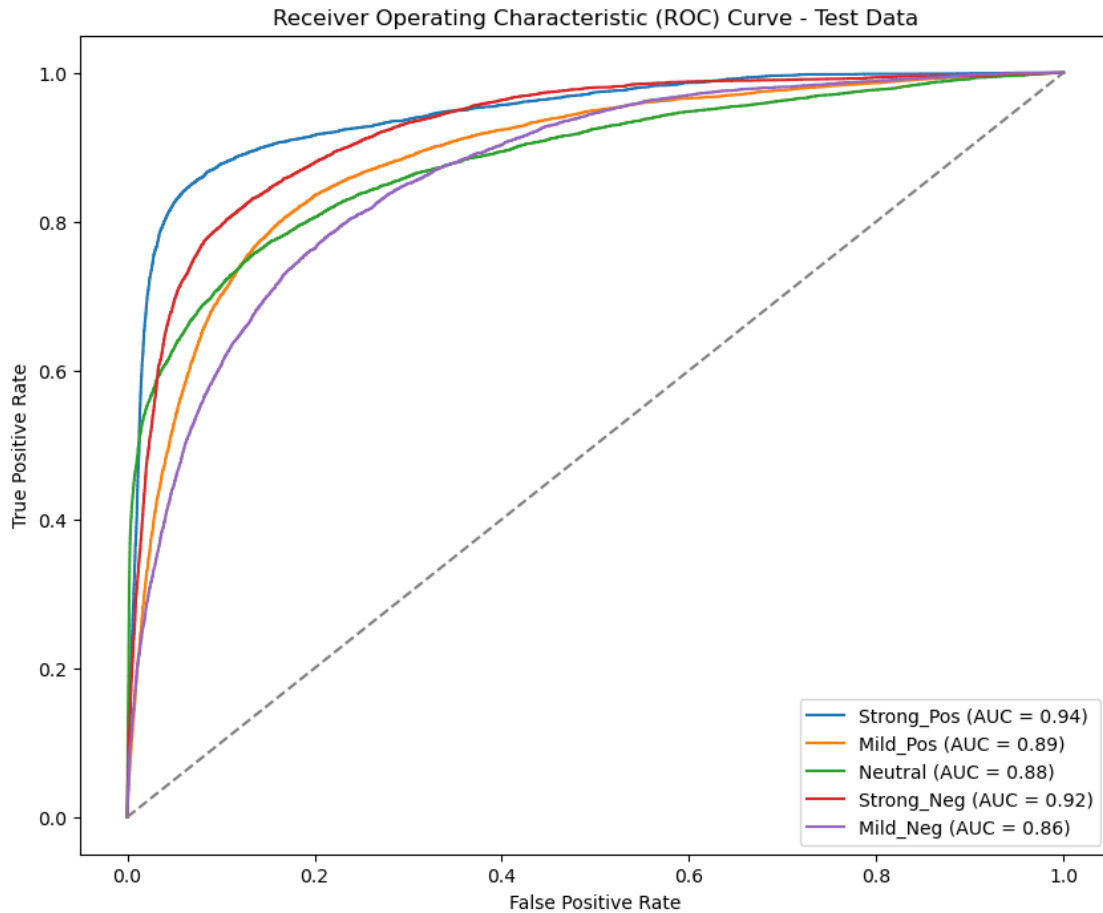
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_test[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```





```
[257]: lime_explanation_lstm(model, x_test[0], reverse_label_map, 50)
```

```
157/157          1s 8ms/step
<IPython.core.display.HTML object>
```

```
[257]: <lime.explanation.Explanation at 0x1ff84c0af60>
```

chatgpt prompt :- add more lstm layers, batch noarmalization to improve the accuracy of test dataset

```
[260]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout,
↳BatchNormalization
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Define the model
```



```

model = Sequential([
    # Embedding layer
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len),

    # First LSTM layer with L2 regularization and batch normalization
    LSTM(128, return_sequences=True, kernel_regularizer=regularizers.l2(0.01)),
    BatchNormalization(),

    # Second LSTM layer with L2 regularization
    LSTM(128, kernel_regularizer=regularizers.l2(0.01)),
    BatchNormalization(),

    # Dropout to prevent overfitting
    Dropout(0.4),

    # Dense layer with L2 regularization
    Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
    Dropout(0.4),

    # Output layer with 5 classes and softmax activation
    Dense(5, activation='softmax')
])

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
    metrics=['accuracy'])

# Callbacks: Early stopping and learning rate reduction
early_stopping = EarlyStopping(monitor='val_loss', patience=3,
    restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2)

# Model summary
model.summary()

```

```

C:\Users\Gaurav\anaconda3\Lib\site-
packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(

```

Model: "sequential\_8"

| Layer (type)                              | Output Shape | Param #     |
|-------------------------------------------|--------------|-------------|
| embedding_3 ( <a href="#">Embedding</a> ) | ?            | 0 (unbuilt) |

|                                               |   |             |
|-----------------------------------------------|---|-------------|
| lstm_2 (LSTM)                                 | ? | 0 (unbuilt) |
| batch_normalization<br>(BatchNormalization)   | ? | 0 (unbuilt) |
| lstm_3 (LSTM)                                 | ? | 0 (unbuilt) |
| batch_normalization_1<br>(BatchNormalization) | ? | 0 (unbuilt) |
| dropout_13 (Dropout)                          | ? | 0           |
| dense_15 (Dense)                              | ? | 0 (unbuilt) |
| dropout_14 (Dropout)                          | ? | 0           |
| dense_16 (Dense)                              | ? | 0 (unbuilt) |

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[262]: start_time = time.time()
model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=125,
    validation_data=(x_test, y_test),
    callbacks=[early_stopping, reduce_lr]
)
# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")
```

Epoch 1/10

1088/1088 70s 63ms/step -

accuracy: 0.3905 - loss: 2.7317 - val\_accuracy: 0.2111 - val\_loss: 3.4339 -

learning\_rate: 0.0010

Epoch 2/10

1088/1088 68s 62ms/step -

accuracy: 0.6251 - loss: 1.0111 - val\_accuracy: 0.3570 - val\_loss: 2.3095 -

learning\_rate: 0.0010

```

Epoch 3/10
1088/1088          68s 62ms/step -
accuracy: 0.6675 - loss: 0.8956 - val_accuracy: 0.6308 - val_loss: 0.9768 -
learning_rate: 0.0010
Epoch 4/10
1088/1088          68s 63ms/step -
accuracy: 0.6948 - loss: 0.8298 - val_accuracy: 0.6263 - val_loss: 0.9814 -
learning_rate: 0.0010
Epoch 5/10
1088/1088          68s 63ms/step -
accuracy: 0.7178 - loss: 0.7799 - val_accuracy: 0.6326 - val_loss: 0.9925 -
learning_rate: 0.0010
Epoch 6/10
1088/1088          68s 63ms/step -
accuracy: 0.7491 - loss: 0.6962 - val_accuracy: 0.6221 - val_loss: 1.0961 -
learning_rate: 5.0000e-04
Training time: 410.77 seconds

```

```

[264]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
→classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
    →round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

```

```

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
print(classification_report(y_train_true, y_train_pred_classes,
    ↳target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
    ↳target_names=class_labels))

# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix -
    ↳Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix -
    ↳Test Data", reverse_label_map)

```

```

4250/4250          31s 7ms/step
1063/1063          8s 7ms/step
(136000, 5)
(34000, 5)
(136000,)
(34000,)
Train Accuracy: 0.7011
Test Accuracy: 0.6308

```

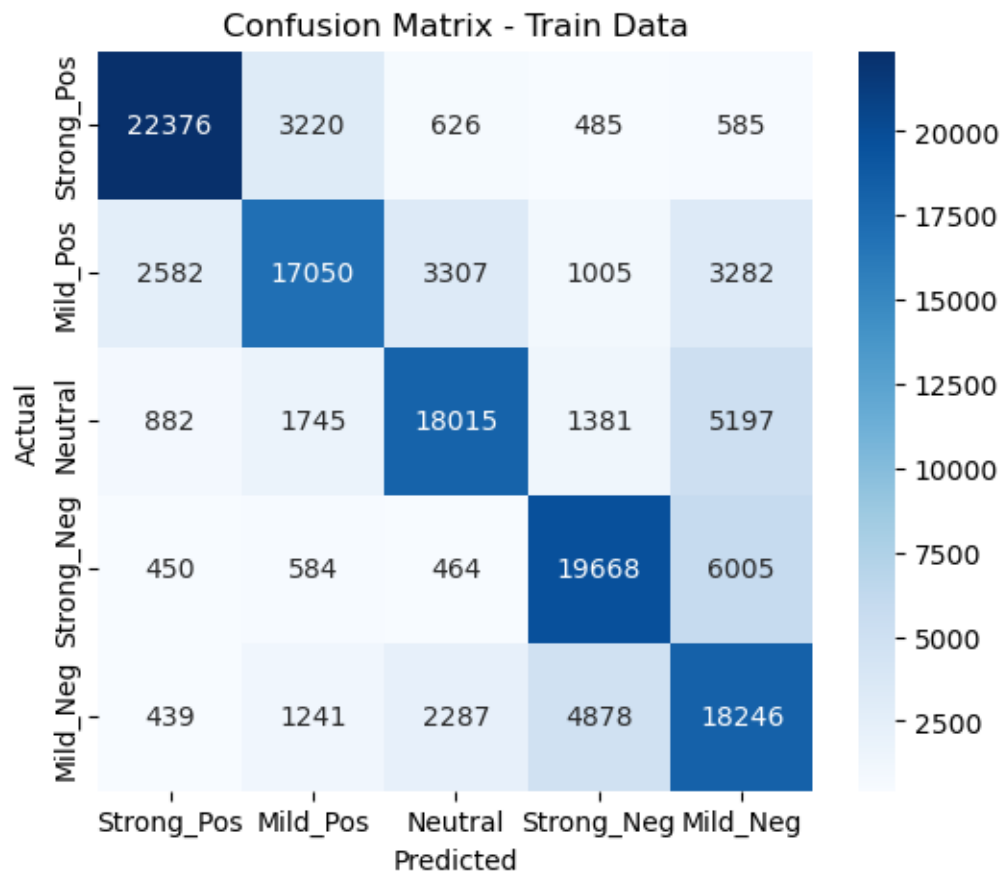
#### Classification Report - Train Data

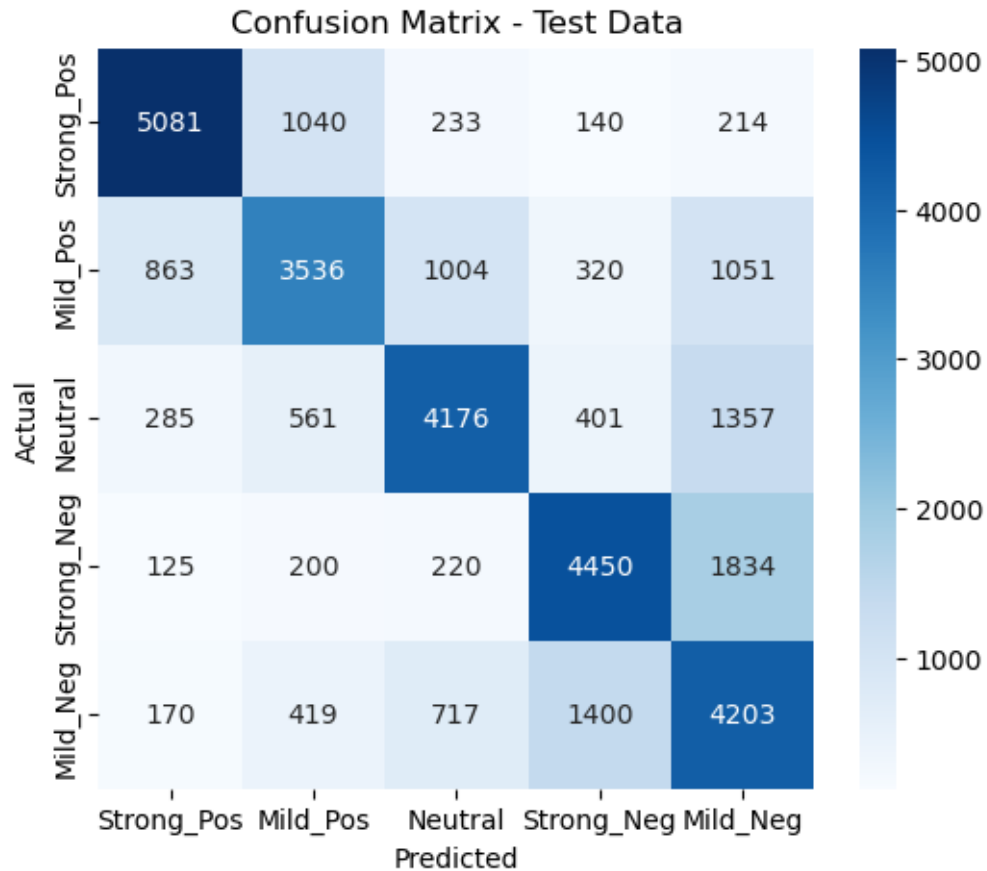
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.84      | 0.82   | 0.83     | 27292   |
| Mild_Pos     | 0.72      | 0.63   | 0.67     | 27226   |
| Neutral      | 0.73      | 0.66   | 0.69     | 27220   |
| Strong_Neg   | 0.72      | 0.72   | 0.72     | 27171   |
| Mild_Neg     | 0.55      | 0.67   | 0.60     | 27091   |
| accuracy     |           |        | 0.70     | 136000  |
| macro avg    | 0.71      | 0.70   | 0.70     | 136000  |
| weighted avg | 0.71      | 0.70   | 0.70     | 136000  |

#### Classification Report - Test Data

|  | precision | recall | f1-score | support |
|--|-----------|--------|----------|---------|
|--|-----------|--------|----------|---------|

|              |      |      |      |       |
|--------------|------|------|------|-------|
| Strong_Pos   | 0.78 | 0.76 | 0.77 | 6708  |
| Mild_Pos     | 0.61 | 0.52 | 0.56 | 6774  |
| Neutral      | 0.66 | 0.62 | 0.64 | 6780  |
| Strong_Neg   | 0.66 | 0.65 | 0.66 | 6829  |
| Mild_Neg     | 0.49 | 0.61 | 0.54 | 6909  |
| accuracy     |      |      | 0.63 | 34000 |
| macro avg    | 0.64 | 0.63 | 0.63 | 34000 |
| weighted avg | 0.64 | 0.63 | 0.63 | 34000 |





```
[266]: import numpy as np

# Get the weights from the Embedding layer
embedding_weights = model.layers[0].get_weights()[0] # Embedding layer is the
↳ first layer (index 0)

# Get the word index mapping from the tokenizer
word_index = tokenizer.word_index

# Calculate the magnitude (norm) of each word vector in the embedding layer
norms = np.linalg.norm(embedding_weights, axis=1)

# Get the indices of the top 10 largest norms
top_10_indices = np.argsort(norms)[::-1][:10]

# Get the corresponding words from the word index
top_10_words = [word for word, index in word_index.items() if index in
↳ top_10_indices]
```

```
# Print the top 10 words with the highest weights
print("Top 10 Features with the Highest Weights:")
for i, word in enumerate(top_10_words, 1):
    print(f"{i}. {word} (Weight: {norms[word_index[word] - 1]:.4f})")
```

Top 10 Features with the Highest Weights:

```
1. terror (Weight: 0.4469)
2. kill (Weight: 0.5133)
3. hell (Weight: 0.5304)
4. crisi (Weight: 0.4862)
5. murder (Weight: 0.4011)
6. wtf (Weight: 0.5413)
7. disast (Weight: 0.7200)
8. perfectli (Weight: 0.7437)
9. fucker (Weight: 1.3432)
10. kube (Weight: 1.2309)
```

```
[268]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪ based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪ based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
    ↪ y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
    ↪ y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
```

```

for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

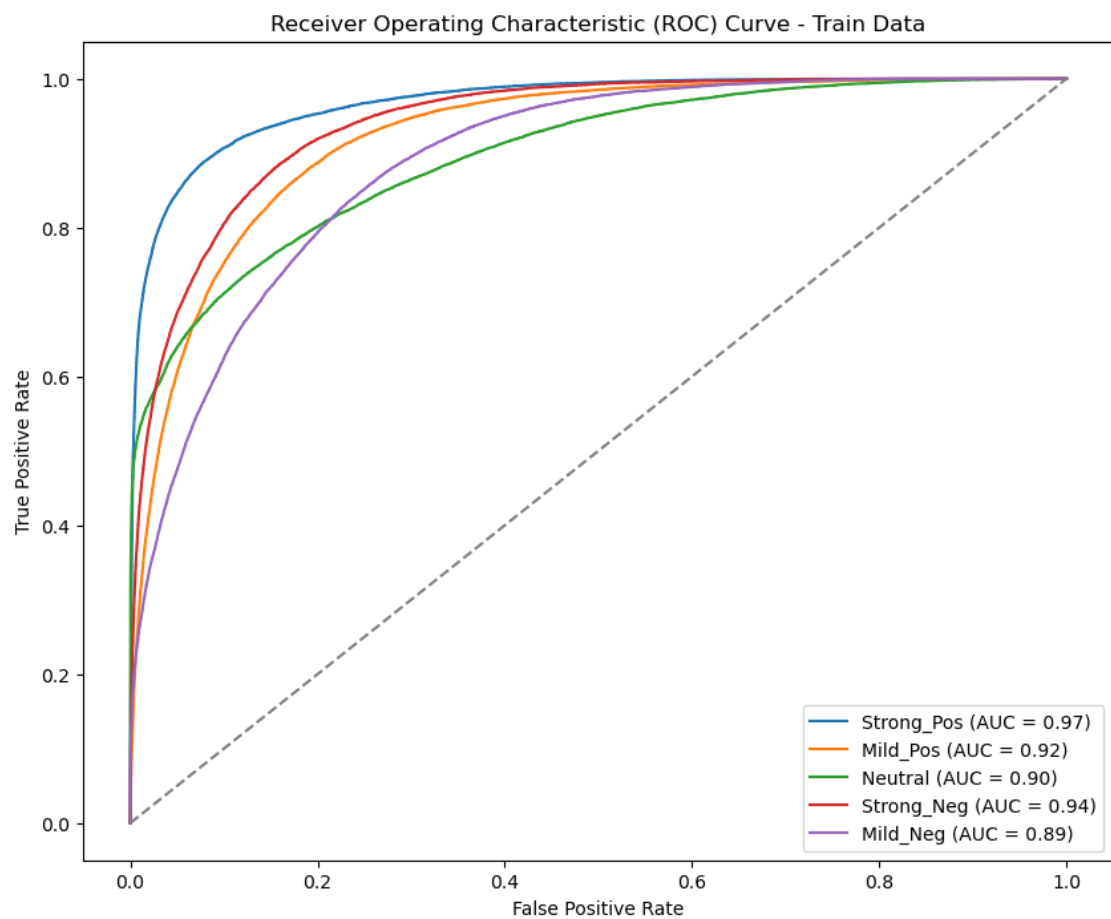
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_test[i]:.2f})')

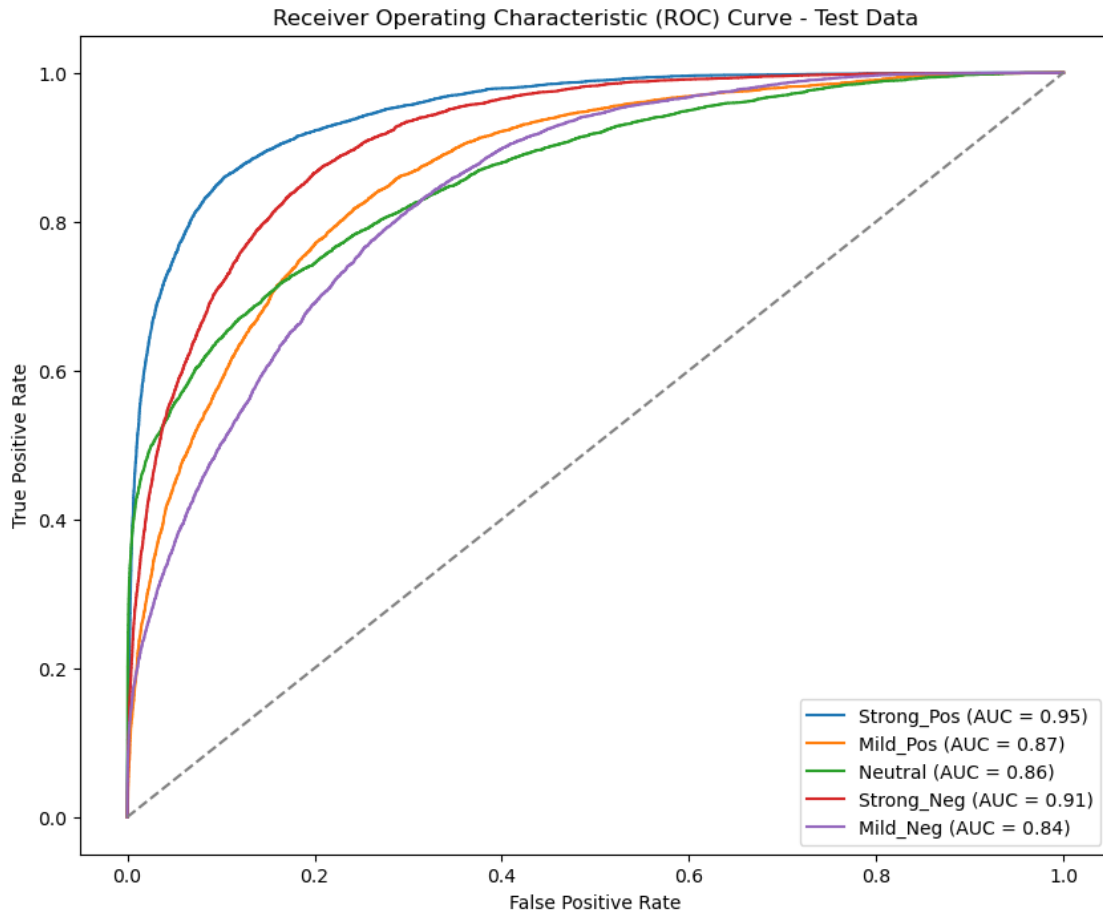
# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```







```
[270]: lime_explanation_lstm(model, x_test[0], reverse_label_map, 50)
```

```
157/157          1s 8ms/step
<IPython.core.display.HTML object>
```

```
[270]: <lime.explanation.Explanation at 0x1ff831c2ed0>
```

## 5 Deep Bidirectional LSTM Model using an embedding layer to learn embeddings

chatgpt prompt :- Deep Bidirectional LSTM Model using an embedding layer to learn embeddings for twitter data with 5 labels

```
[274]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout
```

```

# Define a simple Bidirectional LSTM Model
model = Sequential([
    # Embedding layer
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len),

    # Bidirectional LSTM layer
    Bidirectional(LSTM(128)),

    # Dropout layer to prevent overfitting
    Dropout(0.3),

    # Fully connected layer
    Dense(64, activation='relu'),

    # Output layer with 5 classes
    Dense(5, activation='softmax')
])

# Compile the Model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
              metrics=['accuracy'])

# Model Summary
model.summary()

```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input\_length` is deprecated. Just remove it.  
 warnings.warn(  
Model: "sequential\_9"

| Layer (type)                                       | Output Shape | Param #     |
|----------------------------------------------------|--------------|-------------|
| embedding_4 ( <a href="#">Embedding</a> )          | ?            | 0 (unbuilt) |
| bidirectional_11 ( <a href="#">Bidirectional</a> ) | ?            | 0 (unbuilt) |
| dropout_15 ( <a href="#">Dropout</a> )             | ?            | 0           |
| dense_17 ( <a href="#">Dense</a> )                 | ?            | 0 (unbuilt) |
| dense_18 ( <a href="#">Dense</a> )                 | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[276]: start_time = time.time()
model.fit(x_train, y_train, epochs=5, batch_size=125, validation_data=(x_test,
↪y_test))
# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")
```

```
Epoch 1/5
1088/1088          50s 45ms/step -
accuracy: 0.5000 - loss: 1.1844 - val_accuracy: 0.6624 - val_loss: 0.8483
Epoch 2/5
1088/1088          48s 44ms/step -
accuracy: 0.7062 - loss: 0.7538 - val_accuracy: 0.6971 - val_loss: 0.7835
Epoch 3/5
1088/1088          49s 45ms/step -
accuracy: 0.7610 - loss: 0.6252 - val_accuracy: 0.7059 - val_loss: 0.7697
Epoch 4/5
1088/1088          48s 45ms/step -
accuracy: 0.8021 - loss: 0.5251 - val_accuracy: 0.7069 - val_loss: 0.8054
Epoch 5/5
1088/1088          49s 45ms/step -
accuracy: 0.8334 - loss: 0.4504 - val_accuracy: 0.7016 - val_loss: 0.8472
Training time: 244.01 seconds
```

```
[278]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↪classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
```

```

y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
    ↪round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
print(classification_report(y_train_true, y_train_pred_classes,
    ↪target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
    ↪target_names=class_labels))

# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix -
    ↪Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix -
    ↪Test Data", reverse_label_map)

```

4250/4250                      18s 4ms/step

1063/1063                     5s 4ms/step

(136000, 5)

(34000, 5)

(136000,)

(34000,)

Train Accuracy: 0.8695

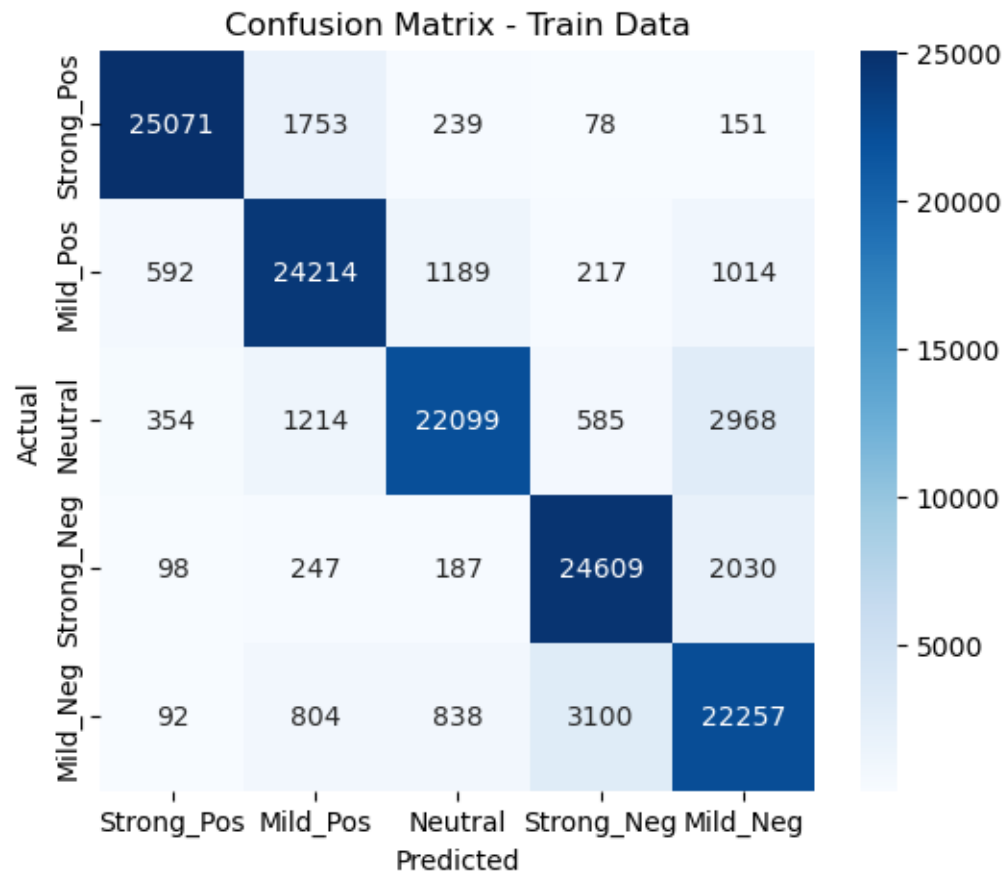
Test Accuracy: 0.7016

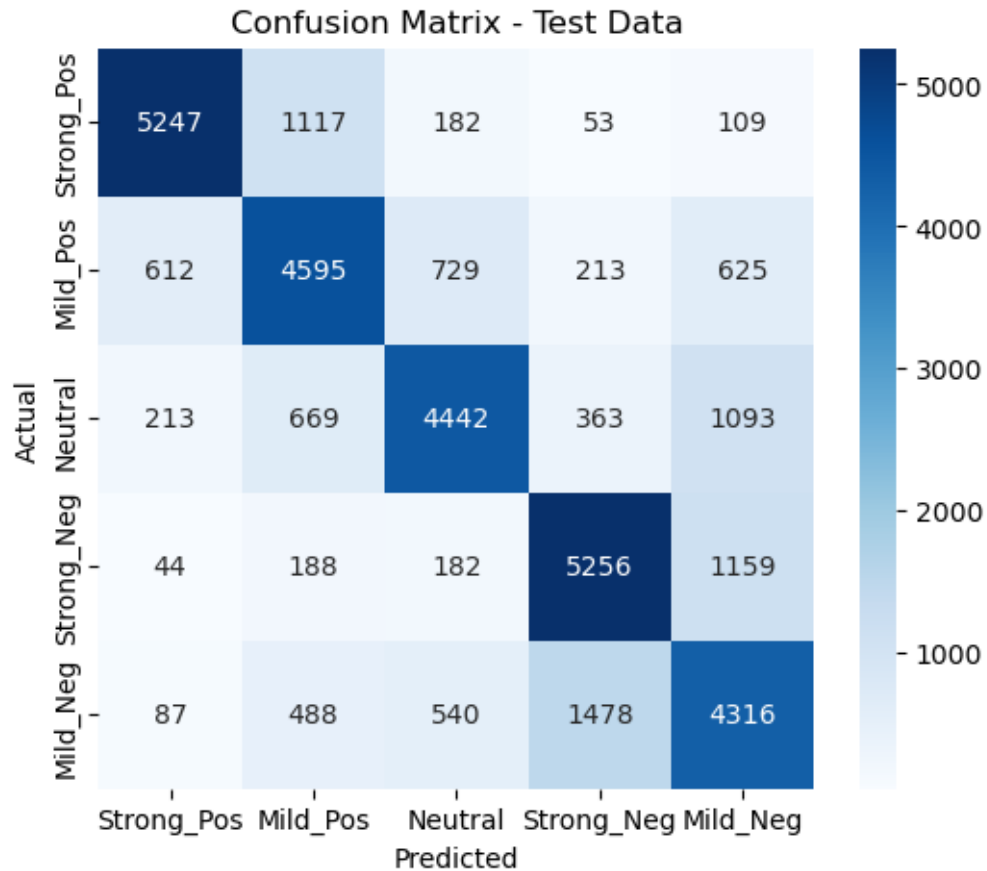
Classification Report - Train Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.96      | 0.92   | 0.94     | 27292   |
| Mild_Pos     | 0.86      | 0.89   | 0.87     | 27226   |
| Neutral      | 0.90      | 0.81   | 0.85     | 27220   |
| Strong_Neg   | 0.86      | 0.91   | 0.88     | 27171   |
| Mild_Neg     | 0.78      | 0.82   | 0.80     | 27091   |
| accuracy     |           |        | 0.87     | 136000  |
| macro avg    | 0.87      | 0.87   | 0.87     | 136000  |
| weighted avg | 0.87      | 0.87   | 0.87     | 136000  |

#### Classification Report - Test Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.85      | 0.78   | 0.81     | 6708    |
| Mild_Pos     | 0.65      | 0.68   | 0.66     | 6774    |
| Neutral      | 0.73      | 0.66   | 0.69     | 6780    |
| Strong_Neg   | 0.71      | 0.77   | 0.74     | 6829    |
| Mild_Neg     | 0.59      | 0.62   | 0.61     | 6909    |
| accuracy     |           |        | 0.70     | 34000   |
| macro avg    | 0.71      | 0.70   | 0.70     | 34000   |
| weighted avg | 0.71      | 0.70   | 0.70     | 34000   |





```
[280]: import numpy as np

# Get the weights from the Embedding layer
embedding_weights = model.layers[0].get_weights()[0] # Embedding layer is the
↳ first layer (index 0)

# Get the word index mapping from the tokenizer
word_index = tokenizer.word_index

# Calculate the magnitude (norm) of each word vector in the embedding layer
norms = np.linalg.norm(embedding_weights, axis=1)

# Get the indices of the top 10 largest norms
top_10_indices = np.argsort(norms)[::-1][:10]

# Get the corresponding words from the word index
top_10_words = [word for word, index in word_index.items() if index in
↳ top_10_indices]
```



```
# Print the top 10 words with the highest weights
print("Top 10 Features with the Highest Weights:")
for i, word in enumerate(top_10_words, 1):
    print(f"{i}. {word} (Weight: {norms[word_index[word] - 1]:.4f})")
```

Top 10 Features with the Highest Weights:

1. <OOV> (Weight: 1.1970)
2. love (Weight: 0.9508)
3. kill (Weight: 0.7155)
4. great (Weight: 0.6606)
5. murder (Weight: 0.7797)
6. best (Weight: 0.8872)
7. suicid (Weight: 0.7675)
8. perfectli (Weight: 0.9451)
9. fucker (Weight: 1.0868)
10. brilliant (Weight: 1.0136)

```
[282]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust
↳ based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust
↳ based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
↳ y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
↳ y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
```

```

for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

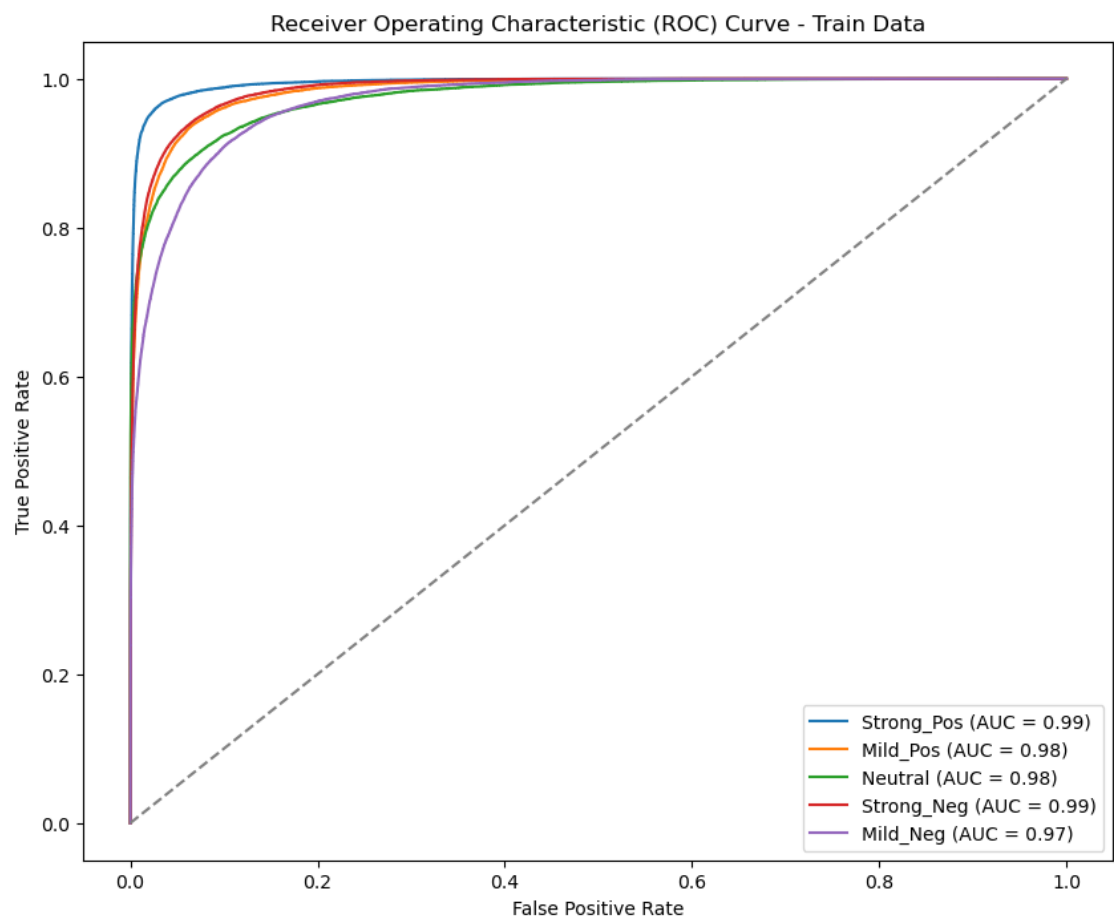
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

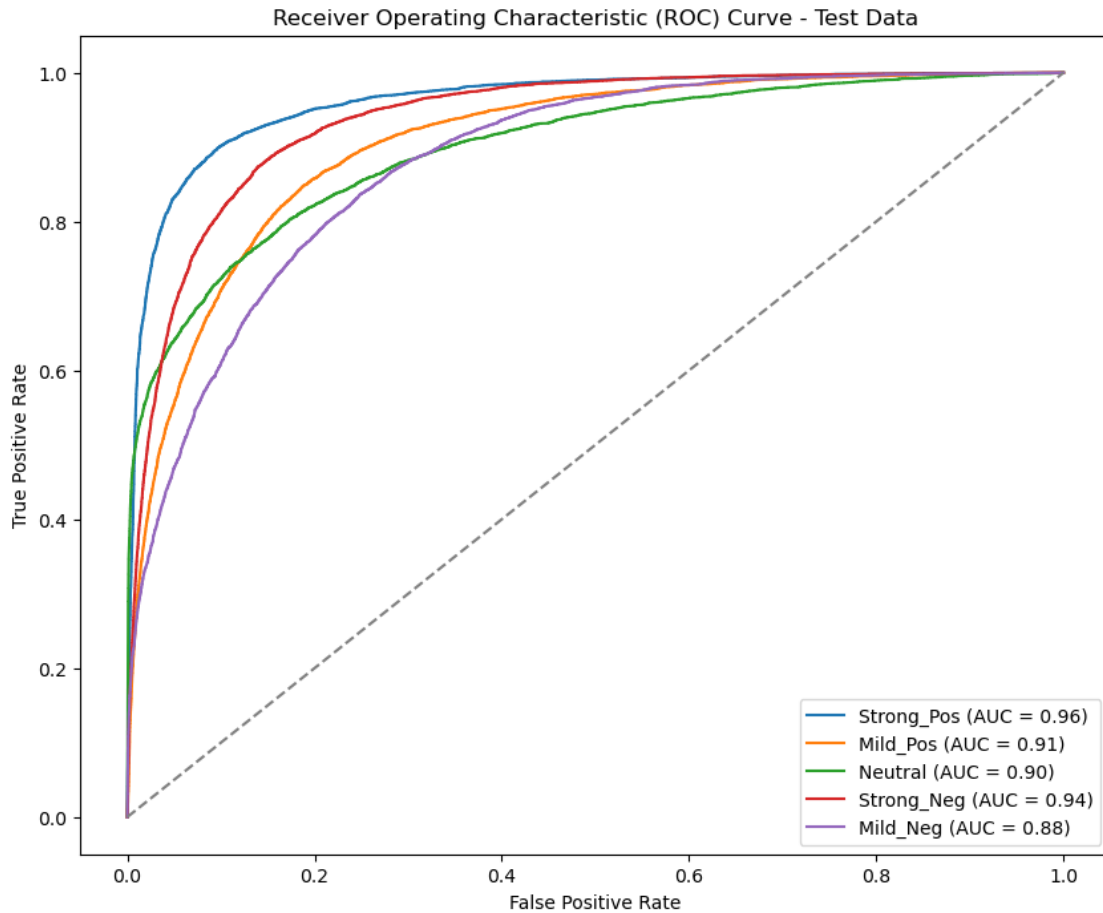
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_test[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```





```
[284]: lime_explanation_lstm(model, x_test[0], reverse_label_map, 50)
```

```
157/157          1s 5ms/step
<IPython.core.display.HTML object>
```

```
[284]: <lime.explanation.Explanation at 0x1ff8119acf0>
```

chatgpt prompt :- add learning rate scheduler and weight balancer

```
[298]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ReduceLROnPlateau
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
```

```

# Compute class weights
class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_train),
    y=y_train
)
class_weights = dict(enumerate(class_weights))

# Define Bidirectional LSTM Model
model = Sequential([
    # Embedding layer
    Embedding(input_dim=max_words, output_dim=128, input_length=max_len),

    # Bidirectional LSTM layer
    Bidirectional(LSTM(128, return_sequences=True)), # First Bi-LSTM
    Dropout(0.3),

    # Second Bi-LSTM layer
    Bidirectional(LSTM(128)),
    Dropout(0.3),

    # Fully connected layer
    Dense(64, activation='relu'),
    Dropout(0.3),

    # Output layer with 5 classes
    Dense(5, activation='softmax')
])

# Compile the Model with a Learning Rate Scheduler
initial_learning_rate = 0.001
optimizer = Adam(learning_rate=initial_learning_rate)

model.compile(loss='sparse_categorical_crossentropy', optimizer=optimizer,
    metrics=['accuracy'])

# Learning Rate Reducer (Reduce LR when validation loss plateaus)
lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2,
    verbose=1)

# Model Summary
model.summary()

start_time = time.time()
# Train the Model with class weights and learning rate scheduler
history = model.fit(
    x_train, y_train,

```

```

    epochs=10,
    batch_size=125,
    validation_data=(x_test, y_test),
    class_weight=class_weights, # Apply class weights
    callbacks=[lr_scheduler] # Learning rate adjustment
)
# Calculate training time
training_time = time.time() - start_time
print(f"Training time: {training_time:.2f} seconds")

```

C:\Users\Gaurav\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input\_length` is deprecated. Just remove it.  
 warnings.warn(

Model: "sequential\_11"

| Layer (type)                                       | Output Shape | Param #     |
|----------------------------------------------------|--------------|-------------|
| embedding_6 ( <a href="#">Embedding</a> )          | ?            | 0 (unbuilt) |
| bidirectional_14 ( <a href="#">Bidirectional</a> ) | ?            | 0 (unbuilt) |
| dropout_19 ( <a href="#">Dropout</a> )             | ?            | 0           |
| bidirectional_15 ( <a href="#">Bidirectional</a> ) | ?            | 0 (unbuilt) |
| dropout_20 ( <a href="#">Dropout</a> )             | ?            | 0           |
| dense_21 ( <a href="#">Dense</a> )                 | ?            | 0 (unbuilt) |
| dropout_21 ( <a href="#">Dropout</a> )             | ?            | 0           |
| dense_22 ( <a href="#">Dense</a> )                 | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/10  
1088/1088 134s 120ms/step  
- accuracy: 0.5050 - loss: 1.1878 - val\_accuracy: 0.6732 - val\_loss: 0.8321 -  
learning\_rate: 0.0010

Epoch 2/10  
1088/1088 131s 120ms/step  
- accuracy: 0.7083 - loss: 0.7603 - val\_accuracy: 0.6964 - val\_loss: 0.7767 -  
learning\_rate: 0.0010

Epoch 3/10  
1088/1088 131s 120ms/step  
- accuracy: 0.7587 - loss: 0.6376 - val\_accuracy: 0.7114 - val\_loss: 0.7634 -  
learning\_rate: 0.0010

Epoch 4/10  
1088/1088 131s 120ms/step  
- accuracy: 0.8006 - loss: 0.5368 - val\_accuracy: 0.7098 - val\_loss: 0.8145 -  
learning\_rate: 0.0010

Epoch 5/10  
1088/1088 0s 111ms/step -  
accuracy: 0.8306 - loss: 0.4632  
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.  
1088/1088 130s 120ms/step  
- accuracy: 0.8306 - loss: 0.4632 - val\_accuracy: 0.7126 - val\_loss: 0.8393 -  
learning\_rate: 0.0010

Epoch 6/10  
1088/1088 130s 120ms/step  
- accuracy: 0.8710 - loss: 0.3656 - val\_accuracy: 0.7109 - val\_loss: 0.9562 -  
learning\_rate: 5.0000e-04

Epoch 7/10  
1088/1088 0s 112ms/step -  
accuracy: 0.8945 - loss: 0.3031  
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.  
1088/1088 131s 120ms/step  
- accuracy: 0.8945 - loss: 0.3031 - val\_accuracy: 0.7067 - val\_loss: 1.0651 -  
learning\_rate: 5.0000e-04

Epoch 8/10  
1088/1088 131s 120ms/step  
- accuracy: 0.9146 - loss: 0.2500 - val\_accuracy: 0.7069 - val\_loss: 1.2398 -  
learning\_rate: 2.5000e-04

Epoch 9/10  
1088/1088 0s 112ms/step -  
accuracy: 0.9264 - loss: 0.2206  
Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.  
1088/1088 131s 120ms/step  
- accuracy: 0.9264 - loss: 0.2206 - val\_accuracy: 0.7046 - val\_loss: 1.3675 -  
learning\_rate: 2.5000e-04

Epoch 10/10  
1088/1088 131s 120ms/step  
- accuracy: 0.9381 - loss: 0.1859 - val\_accuracy: 0.7021 - val\_loss: 1.4724 -

learning\_rate: 1.2500e-04

Training time: 1309.71 seconds

```
[299]: # For Keras models, use predict() to get probabilities
y_train_pred_probs = model.predict(x_train)
y_test_pred_probs = model.predict(x_test)

# Check shapes of predictions and labels
print(y_train_pred_probs.shape)
print(y_test_pred_probs.shape)
print(y_train.shape)
print(y_test.shape)

# Convert predicted probabilities to class labels (for multi-class
↳classification)
if len(y_train_pred_probs.shape) > 1:
    # For multi-class classification, get the class with highest probability
    y_train_pred_classes = np.argmax(y_train_pred_probs, axis=1)
    y_test_pred_classes = np.argmax(y_test_pred_probs, axis=1)
else:
    # For binary classification (if the model output is a single probability),
↳round it to 0 or 1
    y_train_pred_classes = np.round(y_train_pred_probs).astype(int)
    y_test_pred_classes = np.round(y_test_pred_probs).astype(int)

# Since y_train and y_test are already class labels, no need to apply np.argmax
y_train_true = y_train
y_test_true = y_test

# Calculate accuracy and print reports
train_accuracy = accuracy_score(y_train_true, y_train_pred_classes)
test_accuracy = accuracy_score(y_test_true, y_test_pred_classes)

print(f"Train Accuracy: {train_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Extract class labels dynamically (assuming reverse_label_map is defined)
class_labels = [reverse_label_map[i] for i in sorted(reverse_label_map.keys())]

# Print classification report
print("\nClassification Report - Train Data")
print(classification_report(y_train_true, y_train_pred_classes,
↳target_names=class_labels))

print("\nClassification Report - Test Data")
print(classification_report(y_test_true, y_test_pred_classes,
↳target_names=class_labels))
```



```
# Plot confusion matrix for train and test data
plot_confusion_matrix(y_train_true, y_train_pred_classes, "Confusion Matrix - Train Data", reverse_label_map)
plot_confusion_matrix(y_test_true, y_test_pred_classes, "Confusion Matrix - Test Data", reverse_label_map)
```

4250/4250                      50s 12ms/step

1063/1063                     13s 12ms/step

(136000, 5)

(34000, 5)

(136000,)

(34000,)

Train Accuracy: 0.9492

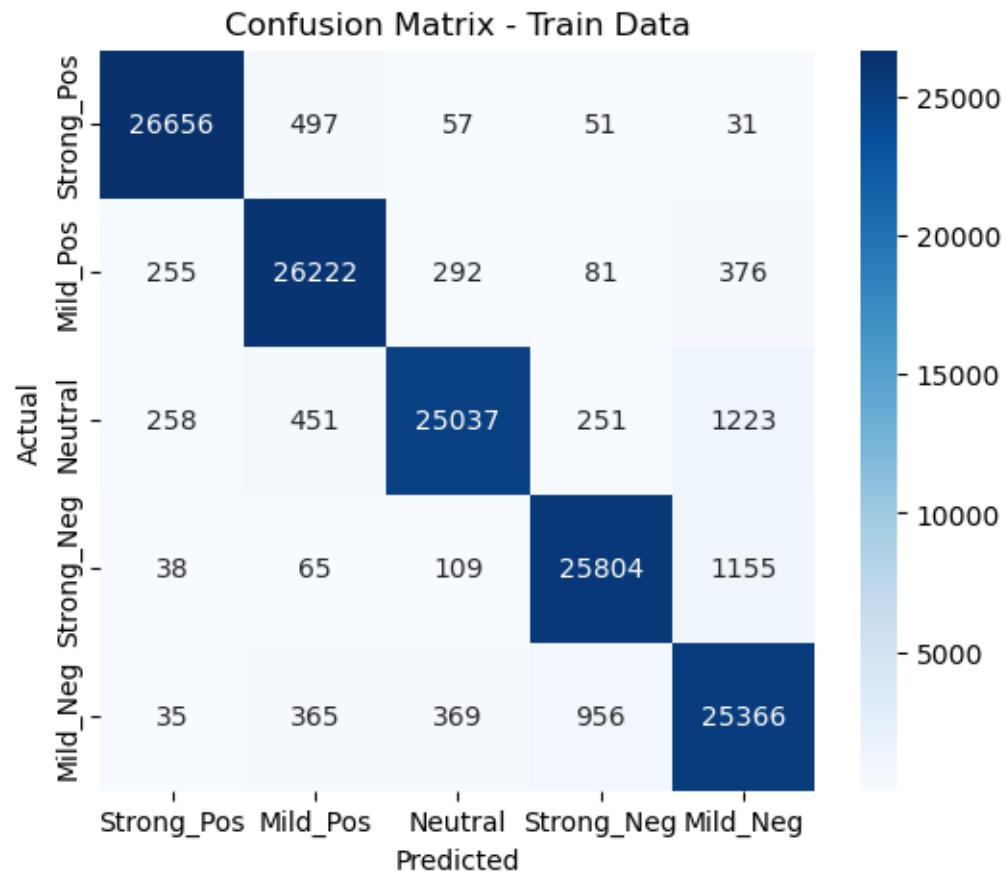
Test Accuracy: 0.7021

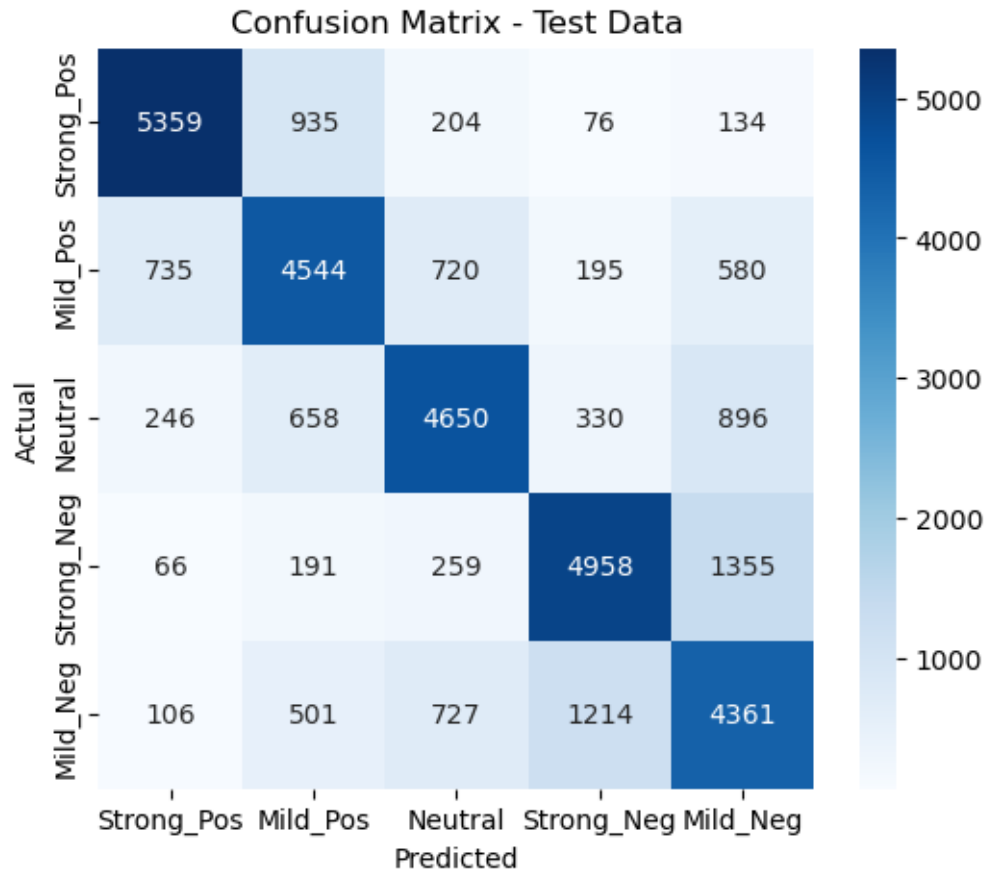
#### Classification Report - Train Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.98      | 0.98   | 0.98     | 27292   |
| Mild_Pos     | 0.95      | 0.96   | 0.96     | 27226   |
| Neutral      | 0.97      | 0.92   | 0.94     | 27220   |
| Strong_Neg   | 0.95      | 0.95   | 0.95     | 27171   |
| Mild_Neg     | 0.90      | 0.94   | 0.92     | 27091   |
| accuracy     |           |        | 0.95     | 136000  |
| macro avg    | 0.95      | 0.95   | 0.95     | 136000  |
| weighted avg | 0.95      | 0.95   | 0.95     | 136000  |

#### Classification Report - Test Data

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Strong_Pos   | 0.82      | 0.80   | 0.81     | 6708    |
| Mild_Pos     | 0.67      | 0.67   | 0.67     | 6774    |
| Neutral      | 0.71      | 0.69   | 0.70     | 6780    |
| Strong_Neg   | 0.73      | 0.73   | 0.73     | 6829    |
| Mild_Neg     | 0.60      | 0.63   | 0.61     | 6909    |
| accuracy     |           |        | 0.70     | 34000   |
| macro avg    | 0.70      | 0.70   | 0.70     | 34000   |
| weighted avg | 0.70      | 0.70   | 0.70     | 34000   |





```
[302]: import numpy as np

# Get the weights from the Embedding layer
embedding_weights = model.layers[0].get_weights()[0] # Embedding layer is the
↳ first layer (index 0)

# Get the word index mapping from the tokenizer
word_index = tokenizer.word_index

# Calculate the magnitude (norm) of each word vector in the embedding layer
norms = np.linalg.norm(embedding_weights, axis=1)

# Get the indices of the top 10 largest norms
top_10_indices = np.argsort(norms)[::-1][:10]

# Get the corresponding words from the word index
top_10_words = [word for word, index in word_index.items() if index in
↳ top_10_indices]
```

```
# Print the top 10 words with the highest weights
print("Top 10 Features with the Highest Weights:")
for i, word in enumerate(top_10_words, 1):
    print(f"{i}. {word} (Weight: {norms[word_index[word] - 1]:.4f})")
```

Top 10 Features with the Highest Weights:

1. kill (Weight: 0.5743)
2. without (Weight: 0.5689)
3. murder (Weight: 0.9139)
4. best (Weight: 0.7896)
5. suicid (Weight: 0.6859)
6. perfectli (Weight: 0.7781)
7. fucker (Weight: 1.1267)
8. cruel (Weight: 1.0625)
9. sockboymustgo (Weight: 0.9877)
10. raci (Weight: 1.2544)

```
[304]: import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize the true labels for multi-class classification
y_train_bin = label_binarize(y_train_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪ based on your classes
y_test_bin = label_binarize(y_test_true, classes=[0, 1, 2, 3, 4]) # Adjust
    ↪ based on your classes

# Initialize dictionaries for ROC curve and AUC calculations
fpr_train = {}
tpr_train = {}
roc_auc_train = {}
fpr_test = {}
tpr_test = {}
roc_auc_test = {}

# Compute ROC curve and AUC for each class (one-vs-rest)
for i in range(y_train_bin.shape[1]):
    fpr_train[i], tpr_train[i], _ = roc_curve(y_train_bin[:, i],
    ↪ y_train_pred_probs[:, i])
    roc_auc_train[i] = auc(fpr_train[i], tpr_train[i])

    fpr_test[i], tpr_test[i], _ = roc_curve(y_test_bin[:, i],
    ↪ y_test_pred_probs[:, i])
    roc_auc_test[i] = auc(fpr_test[i], tpr_test[i])

# Plot ROC curves for Train Data
plt.figure(figsize=(10, 8))
```

```

for i in range(y_train_bin.shape[1]):
    plt.plot(fpr_train[i], tpr_train[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_train[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

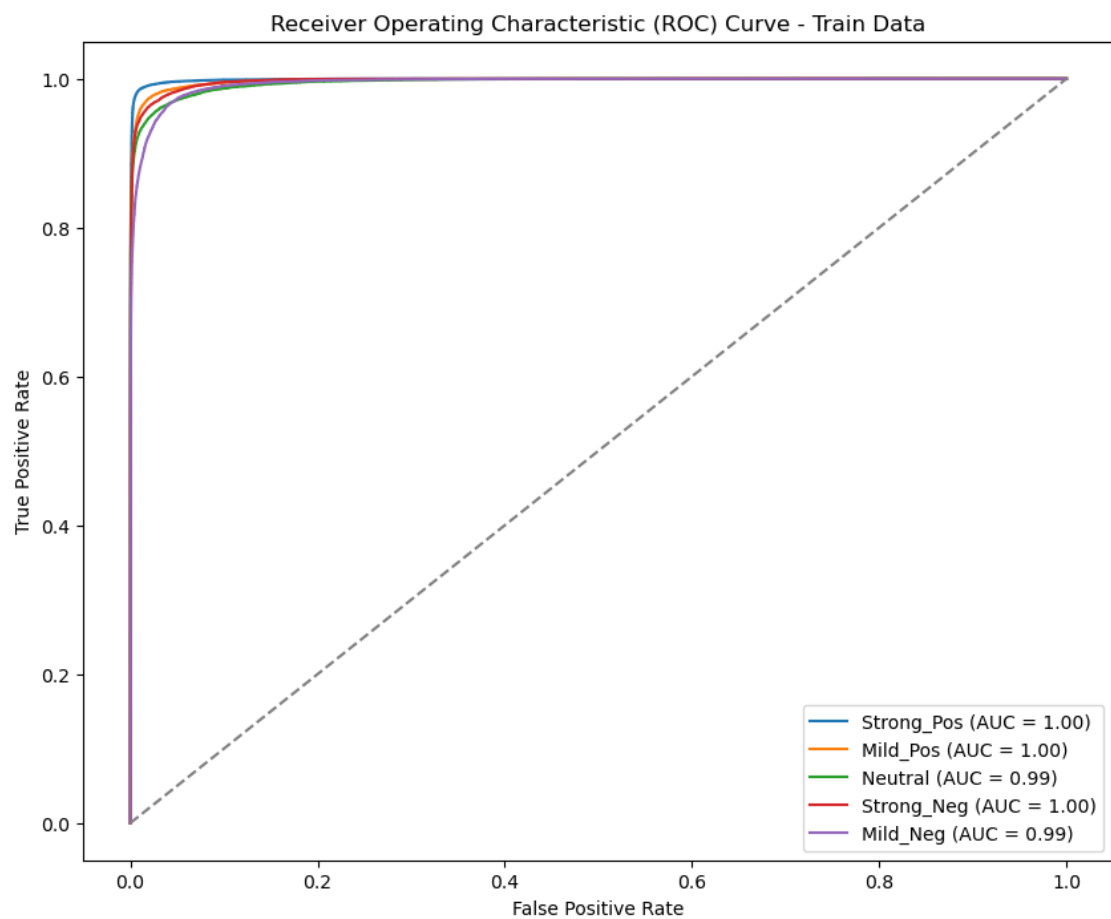
# Labels and title for Train Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Train Data')
plt.legend(loc="lower right")
plt.show()

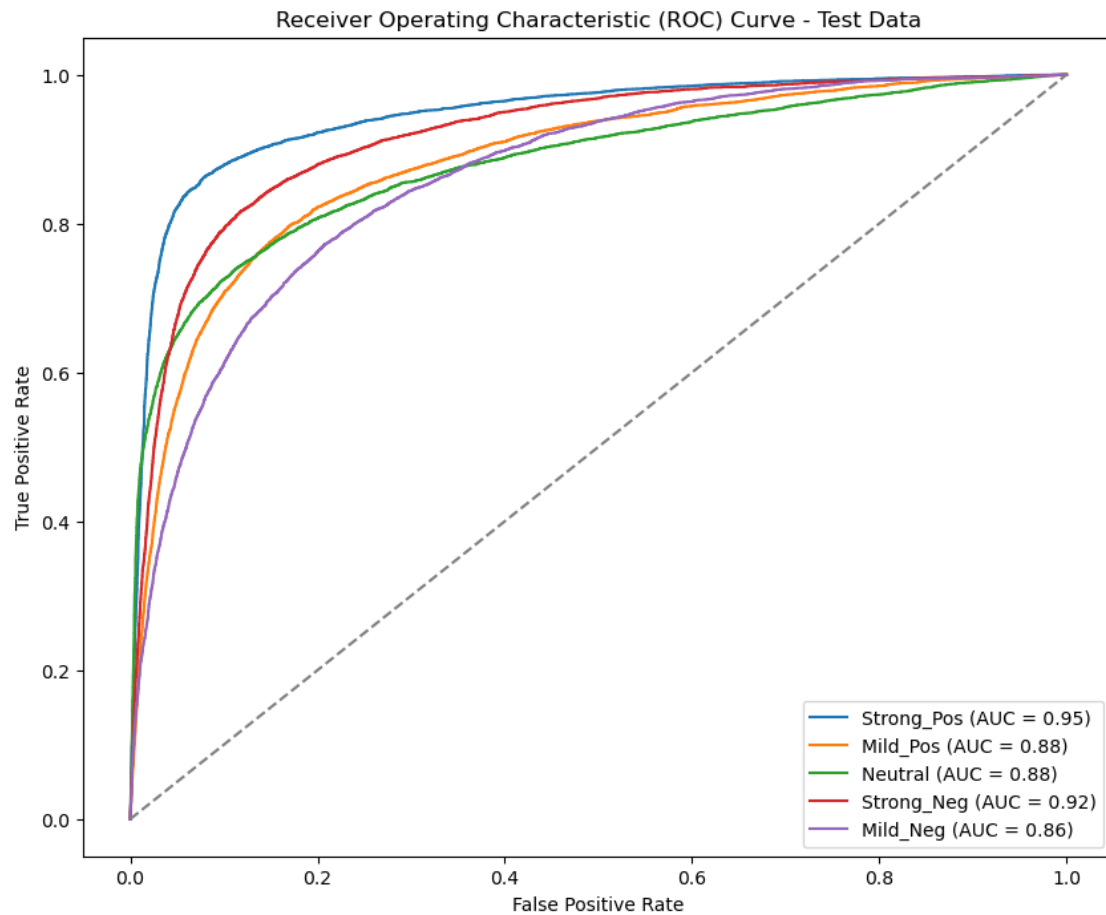
# Plot ROC curves for Test Data
plt.figure(figsize=(10, 8))
for i in range(y_test_bin.shape[1]):
    plt.plot(fpr_test[i], tpr_test[i], label=f'{reverse_label_map[i]} (AUC =_{
    ↪{roc_auc_test[i]:.2f})')

# Plot the diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='gray', linestyle='--')

# Labels and title for Test Data
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Test Data')
plt.legend(loc="lower right")
plt.show()

```





```
[306]: lime_explanation_lstm(model, x_test[0], reverse_label_map, 50)
```

```
157/157          2s 13ms/step
<IPython.core.display.HTML object>
```

```
[306]: <lime.explanation.Explanation at 0x1ff321d7b60>
```

```
[ ]:
```

```
[ ]:
```